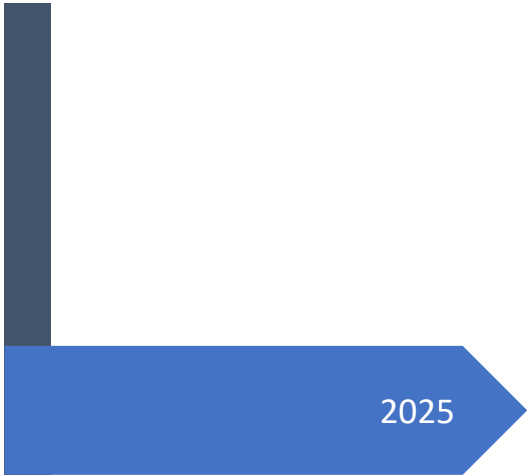
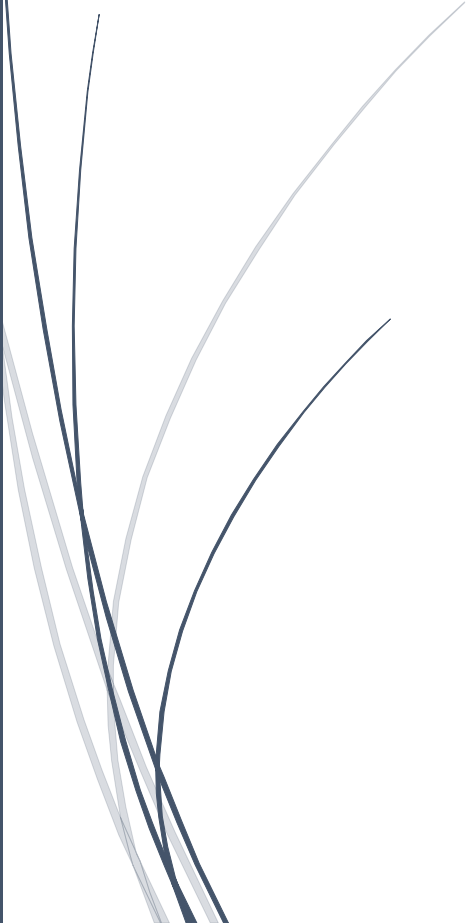


A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the year 2025.

2025

Методичка диффура

Часть 5. Высшая цифровая схемотехника

Several thin, curved lines in dark blue and light grey originate from the bottom left corner and sweep upwards and to the right.

Авторы: KeK, GaSoline

АДМИНЫ ТГ КАНАЛА "ДНЕВНИК БОТАЮЩЕГО ДРАКОШИ"

Содержание

Вступление	2
Лекция 1. АЛУ.....	3
Лекция 2. Регистровый файл.....	14
Лекция 3. Немного про память команд.....	24
Лекция 4. Простейший процессор.....	26
Лекция 5. Прошивка “Cobra”.....	34
Лекция 6. Простейший RISC-V.....	42
Лекция 7. Декодер инструкций RISC-V.....	49
Лекция 8. RISC-V на Verilog.....	66
Лекция 9. Прошивка RISC-V.....	74
Полезные ссылки (+источники).....	81

Вступление

Пытаясь разобраться в том, что понаписал Харис-Харис и другие уважаемые авторы по архитектурам процессора/высшей цифре я несколько осознал, что тема как высших базовых элементов (таких как АЛУ, регистровый файл, память команд, декодер инструкций), так и самого процессора RISC-V не раскрыта (большая часть авторов довольствуется красивыми картиночками без примеров, что весьма сильно мешало мне при изучении архитектуры процессора).

Поэтому автор (KeK) как раз и решил сделать краткое введение по базовым коробочкам, из которых состоит процессор, после которого уже будут более понятным и очевидным чтение других книг (и написание своего RISC-V).

Также, в качестве эксперимента данную методичку частично писал и другой автор (GaSoline), что было весьма забавным опытом (всегда хотелось удалить текст другого автора и переписать его так, как удобно мне)

Авторы специально делают много вставок из Verilog (кому хочется поиграться с ним — добро пожаловать на гитхаб) и RTL что синтезируется из него тем же Quartus, чтобы читатель сначала прочитал красивую теорию про какой-то блок, а потом увидел то, как он синтезируется на самом деле. (хотя бы в RTL).

Лекция 1. АЛУ.

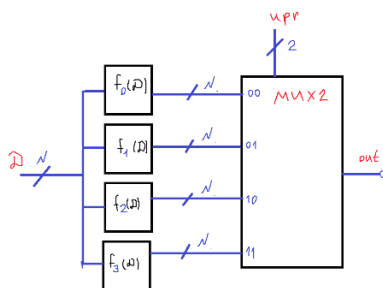
Некое вступление

В данной небольшой статье мы рассмотрим АЛУ и приведем тривиальные примеры (а потом уже читатель сам под нужную ему архитектуру сделает). На самом деле АЛУ и другие компоненты процессора это весьма очевидная штукавина.

Что такое АЛУ?

АЛУ – арифметико-логическое устройство, а если чуток более развернуто, то это штукавина, на которую приходит управляющий сигнал и некий входной сигнал, а она по управляющему сигналу (шине) выбирает операцию и исполняет ее над входными данными (еще, правда, есть такие штукавины как флаги, но про них будет в примерах на Verilog).

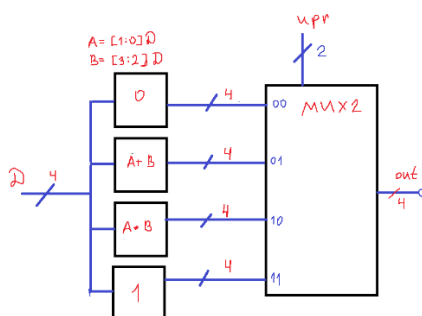
То есть, по данному описанию вырисовывается примерная схема АЛУ:



(Данный АЛУ умеет выполнять 4 функции над N-битным числом (элементами шины) и выдавать 1 из 4 функций по управляющему воздействию)

Как мы видим, это просто мультиплексор с некой комбинационной логикой на входах.

Небольшой пример.



Найти значение на выходе (в десятичной форме), если $upr = 2$, а $D = 13$

Так-как числа A и B это биты числа D в двоичной форме, то переведем D в этот вид (двоичный)

$$D = 13 = 1101_2$$



Тогда мы находим A и B:

$$A = 01_2 = 1$$

$$B = 11_2 = 3$$

$upr = 2 = 10_2$, то у нас над числами A и B будет выполняться умножение

ТЕ, на выходе будет:

$$out = (A * B) = 3 = 0011_2$$

Теперь, познакомившись более-менее с концепцией данного устройства, давайте реализовывать его на Verilog.

АЛУ номер 1. Переключатель между 4 операциями.

Данный пример будет вообще в отрыве от различных архитектур и систем, просто сделаем нечто, у которого можно выбрать операцию над входами. Числа пускай будут 32-битными (входы A, B и выход Out_ALU).

Таблица операций на АЛУ

Upr_ALU	Out_ALU
0	0
1	1
2	A B
3	A & B

Описание на Verilog (но RTL схема выходит некрасивая).

```

1 module ALU_1 (
2     input [1:0] Upr_ALU,
3     input [31:0] A,
4     input [31:0] B,
5     output [31:0] Out_ALU
6 );
7
8     assign Out_ALU = (Upr_ALU == 2'd0) ? 32'b0:
9                       (Upr_ALU == 2'd1) ? 32'b1:
10                      (Upr_ALU == 2'd2) ? A | B: A & B;
11
12 endmodule

```

Описание на Verilog с красивой RTL схемой (загнал операции ИЛИ, И в модули).

```

1 module ALU_1 (
2     input [1:0] Upr_ALU,
3     input [31:0] A,
4     input [31:0] B,
5     output [31:0] Out_ALU
6 );
7
8     wire [31:0] or_mux;
9     wire [31:0] and_mux;
10
11     assign Out_ALU = (Upr_ALU == 2'd0) ? 32'b0:
12                      (Upr_ALU == 2'd1) ? 32'b1:
13                      (Upr_ALU == 2'd2) ? or_mux: and_mux;
14
15     or_32_b or_ALU (
16         .A(A),
17         .B(B),
18         .out_or(or_mux)
19     );
20
21     and_32_b and_ALU (
22         .A(A),
23         .B(B),
24         .out_and(and_mux)
25     );
26
27 endmodule
28

```

```

1 module or_32_b (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out_or
5
6 );|
7
8 assign out_or = A | B;
9 endmodule

```

```

1 module and_32_b (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out_and
5
6 );|
7 assign out_and = A & B;
8 endmodule

```

Тестирование первого АЛУ

Тесты ниже не доказывают абсолютную работоспособность схемы, они лишь могут показать некие очевидные ошибки, если они есть, а для полной проверки схемы надо писать полный тестбенч (а тема данной статьи все-таки не совсем про Verilog и реализацию в нем тестбенчей). Ниже я привел быстрый тест схемы “на глазок”.

Тест операции 0

Upr_...	U 0	0				
A	U 3283417	3283417244	670383663	1583047648	2946927848	3645383503
B	U 1090458	109045818	1357000705	3607723804	2616441118	2237751728
Out_...	B 00000000	00000000000000000000000000000000				

Операция работает (выводит всегда 0).

Тест операции 1

Upr_...	U 1	1				
A	U 3283417	3283417244	670383663	1583047648	2946927848	3645383503
B	U 1090458	109045818	1357000705	3607723804	2616441118	2237751728
Out_...	B 00000000	00000000000000000000000000000001				

Операция работает (выводит всегда 1).

Тест операции 2

Name	0 ps	80,0 ns	160,0 ns	240,0 ns	320,0 ns	400,0 ns	480,0 ns	560,0 ns	640,0 ns	720,0 ns	800,0 ns	880,0 ns	960,0 ns
Upr_ALU	2												
A	0011101110001111111001011011101												
B	00000110011111111010000111010												
Out_ALU	00111111111111111110101111111												

Операция работает (делает А ИЛИ В)

Тест операции 3

Upr_ALU	3												
A	0011101110001111111001011011101												
B	00000110011111111010000111010												
Out_ALU	0000001000001111111000000011000												

Операция работает (делает А И В)

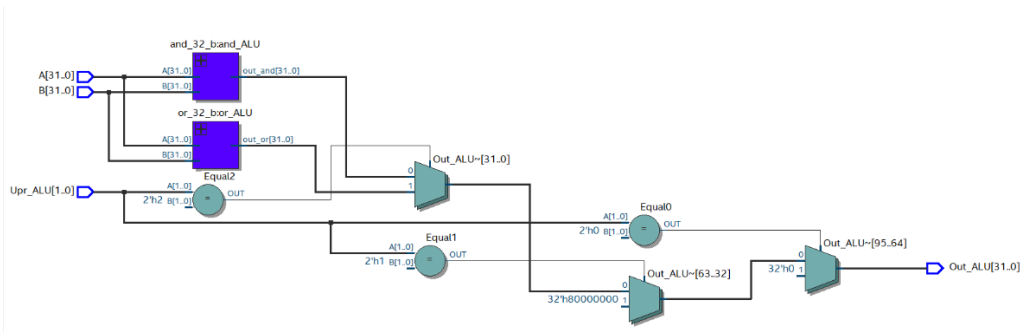
Да, в тесте 2 последних операций есть числа раскрытые не полностью (места не хватило), но автор их проверил, и на них АЛУ тоже работает верно

Тест всех операций разом

Upr_ALU	0	1	2	3	0
A	000110110111100101101000110101110	0110011011111111111110110110000010	00111111011011100011010001101111		
B	111110110100000000011000010111110	0000001011101110100001111010100	10100001010100000110111010100110		
Out_ALU	00000000000000000000000000000000	0110011011111111101101111010110	000000101111011010001110000000	00000000000000000000000000000000	

Как читатель может видеть, вроде бы все работает верно.

RTL схема



АЛУ номер 2. Переключатель между 8 операциями + бит сравнения C.

Если рассматривать АЛУ в отрыве от какой-либо (даже самой простой) архитектуры, то может быть непонятен смысл бита сравнения. Однако, если читатель поверит мне на слово, то концептуально простейший процессор/программируемое устройство должно уметь аппаратно реализовывать циклы и условные конструкции, бит C отвечает за верность выполнения условия (если C = 1 и должна выполняться проверка некоего условия (это зашито в командах, которыми мы прошиваем устройство), то выполняется нетрадиционный переход с одной строки памяти команд на другую (то есть, в обычном режиме у нас идет чтение 1 строчки кода, затем 2 строчки кода, затем 3 строчки кода и тд, но при выполнении неких условий (условная команда и C = 1) считыватель с памяти команд может перепрыгнуть на 7 строчек вверх по программе, или на 12 строчек вниз, что по сути своей и есть аппаратно реализованные условные конструкции и циклы).

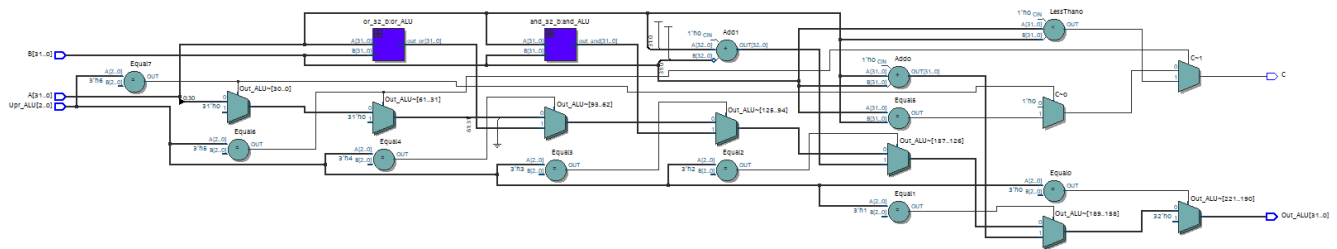
Отходя от большого и красивого обоснования наличия данного бита (ограничимся пока что им) накидаем относительно разумный список команд:

Upr_ALU	C	Операции на АЛУ
0	Все равно, но пускай 0	0
1	Все равно, но пускай 0	A+B
2	Все равно, но пускай 0	A-B
3	Все равно, но пускай 0	A & B
4	Все равно, но пускай 0	A B
5	1, если верно, иначе 0	A > B
6	1, если верно, иначе 0	A == B
7	Все равно, но пускай 0	A << 1 – сдвиг A на 1 влево

Описание на Verilog.

```
1 module ALU_2 (  
2     input [2:0] Upr_ALU,  
3     input [31:0] A,  
4     input [31:0] B,  
5     output C,  
6     output [31:0] out_ALU  
7 );  
8  
9 wire [31:0] or_mux;  
10 wire [31:0] and_mux;  
11 wire [31:0] A_shift_1;  
12 wire A_eq_B;  
13 wire A_more_B;  
14 assign A_shift_1 = {A[31:1], 1'b0};  
15  
16 assign Out_ALU = (Upr_ALU == 3'd0) ? 32'b0:  
17                 (Upr_ALU == 3'd1) ? A+B:  
18                 (Upr_ALU == 3'd2) ? A-B:  
19                 (Upr_ALU == 3'd3) ? and_mux:  
20                 (Upr_ALU == 3'd4) ? or_mux:  
21                 (Upr_ALU == 3'd5) ? 32'b0:  
22                 (Upr_ALU == 3'd6) ? 32'b0: A_shift_1;  
23  
24 assign A_more_B = (A > B) ? 1'b1 : 1'b0;  
25 assign A_eq_B = (A == B) ? 1'b1 : 1'b0;  
26 assign C = (Upr_ALU == 3'd5) ? A_more_B:  
27           (Upr_ALU == 3'd6) ? A_eq_B: 1'b0;  
28  
29 or_32_b or_ALU (  
30     .A(A),  
31     .B(B),  
32     .out_or(or_mux)  
33 );  
34  
35 and_32_b and_ALU (  
36     .A(A),  
37     .B(B),  
38     .out_and(and_mux)  
39 );  
40  
41 endmodule
```

RTL схема АЛУ.



Как читатель видит, устройство весьма нехило в размере выросло.

Тестирование второго АЛУ

Ну что-ж, давайте начнем сие увлекательное действие, состоящее из тестов выхода АЛУ и C для каждой операции. Также, так-как возможно переполнение, но данный флаг (бит переполнения при операции) нами при данном АЛУ не предусмотрен, то будем давать тесты, при которых переполнения не будет.

Тест операции 0 (выдает 0)

Upr_...	U 0	0				
A	U 1555273	1555273705	3701359245	1200712224	2666777510	2016296541
B	U 3376135	337613518	4292456068	1449873350	2934833074	2073938296
C	B 0					
Out_...	B 0000000	00000000000000000000000000000000				

Тест операции 1 (выдает $A + B$)

Upr_...	U 1	1									
A	S 227	227	X	239	X	251	X	263	X	275	
B	S -120	-120	X	-63	X	-6	X	51	X	108	
C	B 0										
Out_...	S 107	107	X	176	X	245	X	314	X	383	

Тест операции 2 (выдает A – B)

[illegible]

Тест операции 3 (Логическое И)

[illegible]

Тест операции 4 (логическое ИЛИ)

[illegible]

Тест операции 5 ($A > B$, работает только для беззнаковых десятичных, желательно свой компаратор писать)

Upr_	U 5											5																																							
A	U 227	227										239										251										263										275									
B	U 4294967	4294967176										4294967233										4294967290										51										108									
C	U 0																																																		
Out_	U 0	0																																																	

Тест операции 6 ($A = B$)

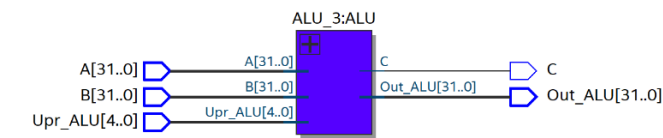
Upr...	U 6	6				
A	S 227	227	239	251	263	275
B	S -120	-120	239	-6	51	275
C	S 0					
Out ...	S 0	0				

Тест операции 7 (сдвиг на 1 разряд числа А влево (для положительных чисел это примерно умножение на 2))

	Upr...	U 7					
	A	S 227	227	239	251	263	275
	B	S -120	-120	239	-6	51	275
	C	S 0					
	Out...	S 454	454	478	502	526	550

Как читатель видит, АЛУ реализовано примерно так, как нам надо (только компаратор можно свой поставить, чтобы он знаковые числа обрабатывал (или встроенный модифицировать))

АЛУ номер 3. Весьма близко к реальному RISC-V.



Для выхода АЛУ (у нас вместо result_0 выход Out_ALU

Операция	={cmp, mod, opcode}	Выражение	Действие
ADD	0 0 000	result_o = a_i + b_i	Сложение
SUB	0 1 000	result_o = a_i - b_i	Вычитание
SLL	0 0 001	result_o = a_i << b_i	Сдвиг влево
SLTS	0 0 010	result_o = a_i < b_i	Знаковое сравнение
SLTU	0 0 011	result_o = a_i < b_i	Беззнаковое сравнение
XOR	0 0 100	result_o = a_i ^ b_i	Побитовое исключающее ИЛИ
SRL	0 0 101	result_o = a_i >> b_i	Сдвиг вправо
SRA	0 1 101	result_o = a_i >>> b_i	Арифметический сдвиг вправо (операнд a_i — знаковый)
OR	0 0 110	result_o = a_i b_i	Побитовое логическое ИЛИ
AND	0 0 111	result_o = a_i & b_i	Побитовое логическое И

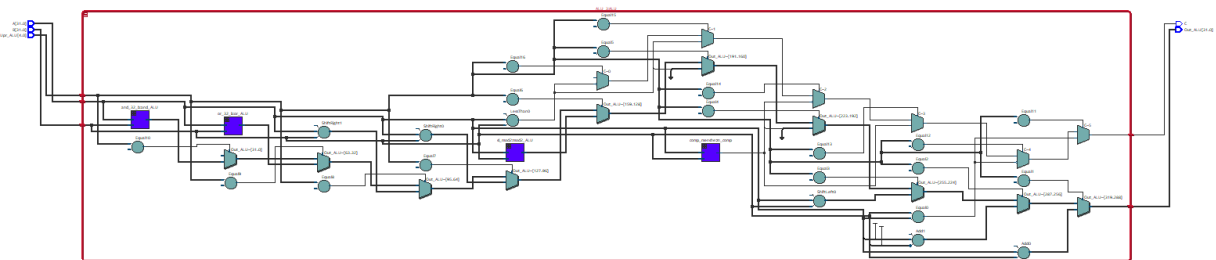
Для некоего характерного бита (флага), у нас, как и в прошлом АЛУ это будет бит C (результат логической операции)

Операция	={cmp, mod, opcode}	Выражение	Действие
EQ	1 1 000	flag_o = (a_i == b_i)	Выставить флаг, если равны
NE	1 1 001	flag_o = (a_i != b_i)	Выставить флаг, если не равны
LTS	1 1 100	flag_o = a_i < b_i	Знаковое сравнение <
GES	1 1 101	flag_o = a_i ≥ b_i	Знаковое сравнение ≥
LTU	1 1 110	flag_o = a_i < b_i	Беззнаковое сравнение <
GEU	1 1 111	flag_o = a_i ≥ b_i	Беззнаковое сравнение ≥

В общем-то, 2 примера АЛУ выше должны были подвести нас к данной штуковине и, в общем-то, показать ее очевидность. Как и в прошлом АЛУ, ставим 0 на выходы, которые мы не используем.

Из новых моментов по сравнению с предыдущим АЛУ скорее большее внимание биту C и оформлению кода (теперь, например, вместо кода операции сложения 5'd0 будем использовать ADD с помощью конструкции parameter, которая будет сама вместо ADD подставлять 5'd0). Автор не будет управляющий вход АЛУ разделять на 3 части, просто сделав его 5-битным.

RTL схема АЛУ.



Описание АЛУ на Verilog (главный модуль).

```
1 module ALU_3 (
2     input [4:0] Upr_ALU,
3     input [31:0] A,
4     input [31:0] B,
5     output C,
6     output [31:0] Out_ALU
7 );
8     parameter ADD = 5'b00000;
9     parameter SUB = 5'b01000;
10    parameter SLL = 5'b00001;
11    parameter SLTS = 5'b00010;
12    parameter SLTU = 5'b00011;
13    parameter XOR = 5'b00100;
14    parameter SRL = 5'b00101;
15    parameter SRA = 5'b01101;
16    parameter OR = 5'b00110;
17    parameter AND = 5'b00111;
18    parameter EQ = 5'b11000;
19    parameter NE = 5'b11001;
20    parameter LTS = 5'b11100;
21    parameter GES = 5'b11101;
22    parameter LTU = 5'b11110;
23    parameter GEU = 5'b11111;
24
25    wire [31:0] out_or;
26    wire [31:0] out_and;
27    wire [31:0] out_add;
28    wire [31:0] out_sub;
29    wire [31:0] out_xor;
30    wire [31:0] out_sll;
31    wire [31:0] out_slts;
32    wire [31:0] out_sltu;
33    wire [31:0] out_srl;
34    wire [31:0] out_sra;
35    wire out_A_menshe_B_zn;
36    wire out_A_ne_menshe_B_zn;
37    wire out_A_menshe_B;
38    wire out_A_ne_menshe_B;
39    wire out_equal;
40    wire out_notequal;
41
42    assign out_A_ne_menshe_B_zn = ~out_A_menshe_B_zn;
43    assign out_A_ne_menshe_B = ~out_A_menshe_B;
44    assign out_equal = (A==B) ? 1'b1 : 1'b0;
45    assign out_A_menshe_B = (A<B) ? 1'b1 : 1'b0;
46    assign out_notequal = ~out_equal;
47
48    assign out_add = A+B;
49    assign out_sub = A-B;
50    assign out_sll = A << B;
51    assign out_slts = {{31{1'b0}},out_A_menshe_B_zn};
52    assign out_sltu = {{31{1'b0}},out_A_menshe_B};
53    assign out_srl = A >> B;
54    assign out_sra = A >>> B;
55
56
57    assign Out_ALU = (Upr_ALU == ADD) ? out_add:
58                    (Upr_ALU == SUB) ? out_sub:
59                    (Upr_ALU == SLL) ? out_sll:
60                    (Upr_ALU == SLTS) ? out_slts:
61                    (Upr_ALU == SLTU) ? out_sltu:
62                    (Upr_ALU == XOR) ? out_xor:
63                    (Upr_ALU == SRL) ? out_srl:
64                    (Upr_ALU == SRA) ? out_sra:
65                    (Upr_ALU == OR) ? out_or:
66                    (Upr_ALU == AND) ? out_and:32'b0;
67
68    assign C = (Upr_ALU == EQ) ? out_equal:
69              (Upr_ALU == NE) ? out_notequal:
70              (Upr_ALU == LTS) ? out_A_menshe_B_zn:
71              (Upr_ALU == GES) ? out_A_ne_menshe_B_zn:
72              (Upr_ALU == LTU) ? out_A_menshe_B:
73              (Upr_ALU == GEU) ? out_A_ne_menshe_B:1'b0;
74
75
76    or_32_b or_ALU (
77        .A(A),
78        .B(B),
79        .out_or(out_or)
80    );
81
82
83    and_32_b and_ALU (
84        .A(A),
85        .B(B),
86        .out_and(out_and)
87    );
88
89    sl_mod2 mod2_ALU (
90        .A(A),
91        .B(B),
92        .out(out_xor)
93    );
94
95    comp_menshe zn_comp (
96        .A(A),
97        .B(B),
98        .out(out_A_menshe_B_zn)
99    );
100 endmodule
```

Описание АЛУ на Verilog (модули).

```

1 module s1_mod2 (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out
5 );
6
7 assign out = A ^ B;
8
9 endmodule

```

```

1 module or_32_b (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out_or
5 );
6
7
8 assign out_or = A | B;
9 endmodule

```

```

1 module and_32_b (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out_and
5 );
6
7 assign out_and = A & B;
8
9 endmodule

```

Тест АЛУ.

В силу того, что операций весьма много, будем тестировать сразу несколько штук за тест (такой метод позволит найти, увы, только очевиднейшие ошибки).

Операции ADD, SUB, SLTS (сложение, вычитание, знаковое сравнение (выход на АЛУ)).

	Upr_ALU	0			8			2				
	A	-100	-77	-54	-31	-8	15	38	61	84	107	
	B	-600	-500	-400	-300	-200	-100	0	30	200	-100	
	C											
	Out_ALU	-700	-577	-454	-331	269	192	115	38	0	1	0

Операции XOR, OR, AND (сложение по модулю 2, ИЛИ, И (везде побитовое, выход на АЛУ)).

Upr_ALU	6						7						4																	
A	000111010110000001110000111100						01101001000111111000101010001010						00000111110110100110111101001011						00000000000000000000000000000000						10000100010010101110001011011101					
B	1110110010110110010001111111000						00011100100010001110101111001100						11110010101001000101110010011011						101001100001001000100100000011011						00111011011001000011001001100101					
C																														
Out_ALU	111111011111011001101111111100						011111010001111110110111001110						00000010100000000100110000001011						101001100001001000100100000011011						1011111001011101101000010111000					

Операции SLL, SRL, SRA (сдвиг влево, сдвиг направо, арифметический сдвиг направо на В позиций).

Upr_	U 1	1		5		13					
A	B.0001110	000111010110000001110000111100		0110100100011111100010101001010		00000111110110100110111101001011		00000000000000000000000000000000		10000100010010101110001011011101	
B	U 1	1		2		1					
C	S 0										
Out_	B.0011101	0011101011000000011100001111000		10100100011111100010101000101000		0000001111011010011011110100101		00000000000000000000000000000000		01000010001001010111000101101110	

Операция SLTS (беззнаковое сравнение $A < B$).

Upr_...	U 3	3				
A	U 4928451	492845116	1763674762	131755851	0	2219500253
B	U 1372872	137287247	4040284435	3023470043	520616384	645961084
C	S 0					
Out_...	U 0	0	1	0		

Беззнаковое оно, так-как не работает на отрицательных числах (ниже будет пример).

Upr_...	U 3	3				
A	S 4928451	492845116	1763674762	131755851	0	-2075467043
B	S 1372872	137287247	-254682861	-1271497253	520616384	645961084
C	S 0					
Out_...	U 0	0	1	0		

Тест флага С для АЛУ.

Операции EQ, NE ($A == B$, $A != B$, в, если выражение верно, то выводить 1)

Upr_...	U 24	24				25			
A	S 4928451	492845116	-123456789	1763674762	25	0	-2075467043	98765432	
B	S 1372872	137287247	-123456789	-254682861	25	520616384	645961084	98765432	
C	S 0								
Out_...	S 0	0							

Операции LTS, GES (знаковое сравнение “меньше”, “больше или равно”).

Upr_...	U 28	28				29			
A	S 4928451	492845116	-123456789	1763674762	25	0	-2075467043	987654321	
B	S 1372872	137287247	-123456788	-254682861	25	520616384	645961084	987654321	
C	S 0								
Out ...	S 0	0							

Операции LTU, GEU.

Upr_...	U 28	30				31			
A	U 4928451	492845116	4171510507	1763674762	25	0	2219500253	3307312975	
B	U 1372872	137287247	4171510508	4040284435	25	520616384	645961084	3307312975	
C	U 0								
Out ...	U 0	0							

Про реализацию знакового компаратора (необязательное дополнение).

Пусть у нас есть сравнитель (операция “меньше”) для беззнаковых десятичных чисел, необходимо реализовать данную операцию для знаковых десятичных чисел (потому как встроенная операция для беззнаковых).

Реализуем данную операцию путем вычитания из A числа B, результат записав в S ($S = A - B$). Рассмотрим теперь ситуации, когда $S < 0$ (то есть, $A < B$).

Когда $A < 0$ (знаковый бит = 1), $B > 0$ (знаковый бит = 0).

Учтем данное условие выражением $A[31] \& \overline{B[31]}$

Когда $A > 0$ (знаковый бит = 0), $B > 0$ (знаковый бит = 0), $S < 0$ (знаковый бит = 1)

Учтем данное условие выражением $\overline{A[31]} \& \overline{B[31]} \& S[31]$

Когда $A < 0$ (знаковый бит = 1), $B < 0$ (знаковый бит = 1), $S > 0$ (знаковый бит = 0)

Учтем данное условие выражением $A[31] \& B[31] \& \overline{S[31]}$

В итоге, получим выражение для знакового сравнения:

$$comp_{zn\ A<B} = A[31] \& \overline{B[31]} \mid \overline{A[31]} \& \overline{B[31]} \& S[31] \mid A[31] \& B[31] \& S[31]$$

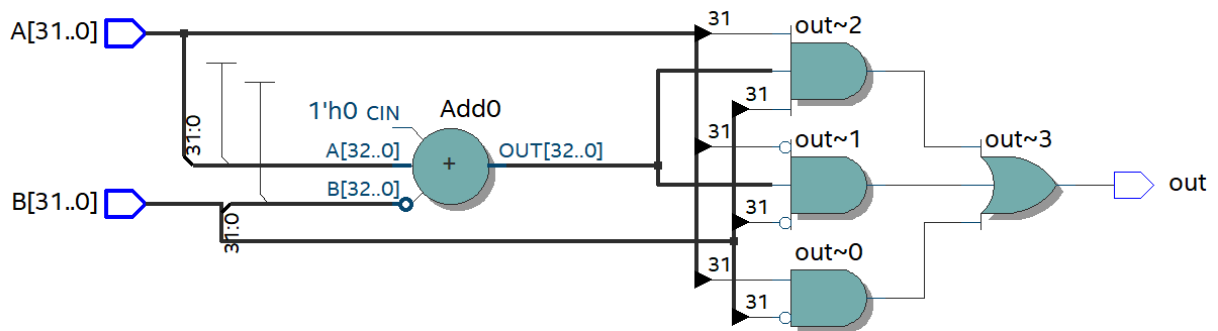
Код на Verilog, описывающий компаратор знакового сравнения “меньше”.

```

1 module comp_menshe (
2     input [31:0] A,
3     input [31:0] B,
4     output out
5 );
6
7 wire [31:0] S;
8 assign S = A-B;
9 assign out = (A[31] & ~B[31]) | (~A[31] & ~B[31] & S[31]) | (A[31] & B[31] & S[31]);
10
11 endmodule

```

RTL схема знакового компаратора “А меньше В”



Тест знакового компаратора.

Name	Value	0 ps	80,0 ns	160,0 ns	240,0 ns	320,0 ns	400,0 ns	480,0 ns	560,0 ns	640,0 ns	720,0 ns	800,0 ns	880,0 ns	960,0 ns
A	S -1000	-1000	-1000	-1000	-1000	-1000	-1000	-1000	-1000	-1000	-1000	-1000	-1000	-1000
B	S -850	-850	-850	-850	-850	-850	-850	-850	-850	-850	-850	-850	-850	-850
out	S 1	1	1	1	1	1	1	1	1	1	1	1	1	1

Таким образом, у нас есть встроенный компаратор в Verilog, который реализует беззнаковое “меньше” и сейчас мы описали компаратор на Verilog, который выполняет такую же операцию “меньше”, но уже с знаковыми числами. Операцию “равно” будем использовать встроенную в Verilog, а операцию “больше” реализуем как “меньше”, но с другими входными данными ($A > B$ это же $B < A$).

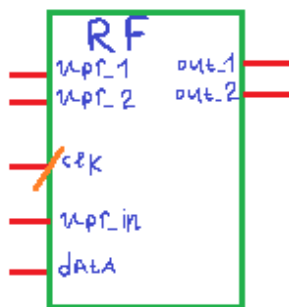
Лекция 2. Регистровый файл.

Вступление

В Харис-Харисе мне больше всего не нравится его резкий переход от средней цифры (триггеров, мультиплексоров, дешифраторов) к высшей цифре (регистровый файл, АЛУ, расширитель знака и тд). В данной небольшой статье поговорим мы про регистровый файл (по факту, встроенная быстрая память процессора) и его устройство.

Концепция регистрового файла

Итак, нам надо сделать вот такую штуковину (упрощенный регистровый файл):



Как она должна работать:

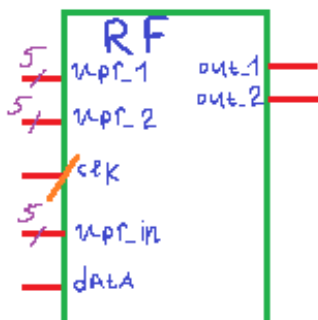
По управляющей шине мы выбираем номер ячейки памяти, значение с которой мы хотим вывести на выход (соответственно, вход `upr_1` управляет `out_1`, а вход `upr_2` управляет `out_2`)

Вход `upr_in` задает нам ячейку памяти, в которую надо записывать информацию из `data`.

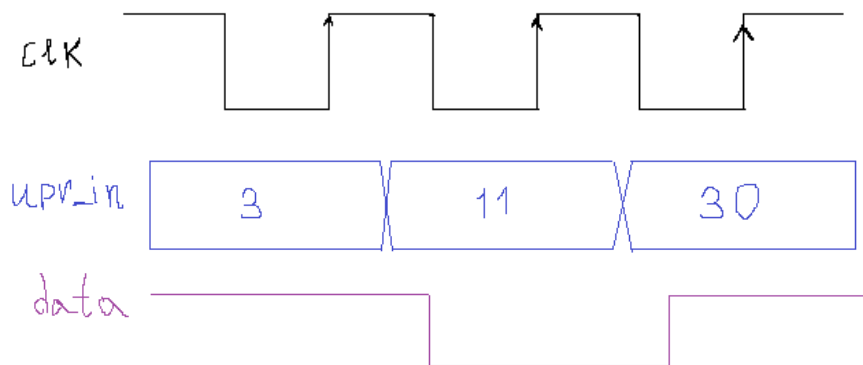
Запись в ячейку памяти и ее считывание будет по фронту `clk` из 0 в 1

Пример регистрового файла и его работы:

Давайте сделаем регистровый файл на 32 однобитные ячейки. Тогда, в 3 управляющих шинах (`upr_1`, `upr_2`, `upr_in`) будет по 5 проводов. Так-как ячейки однобитные, то `data` и `out_1`, `out_2` будут содержать в себе 1 провод.

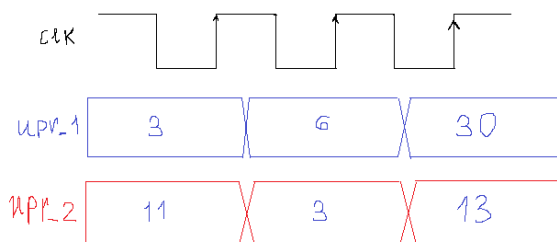


Запись информации



Как мы видим, идет запись "1" по clk в 3 ячейку памяти, "0" по clk в 11 ячейку памяти и "1" по clk в 30 ячейку памяти.

Считывание информации с ячеек памяти



Давайте найдем значение на выходах после 3 event (3 переходов clk с 0 в 1)

$event_1$; out_1 = память₃ = "1", out_2 = память₁₁ = "0"

$event_2$; out_1 = память₆ = "0", out_2 = память₃ = "1"

$event_3$; out_1 = память₃₀ = "1", out_2 = память₁₃ = "0"

Пример работы регистрового файла (WE = 1 – запись с data в ячейку номером upr_in разрешена)

clk	U 0														
▶ upr_1	U 3	<table><tr><td>3</td><td>2</td><td>3</td><td>1</td><td>3</td><td>2</td><td>1</td><td>2</td><td>3</td><td colspan="2">1</td><td>2</td></tr></table>	3	2	3	1	3	2	1	2	3	1		2	
3	2	3	1	3	2	1	2	3	1		2				
▶ upr_2	U 2	<table><tr><td>2</td><td>3</td><td>1</td><td>3</td><td>1</td><td>0</td><td>1</td><td>0</td><td colspan="2">2</td><td>0</td><td>2</td><td>0</td></tr></table>	2	3	1	3	1	0	1	0	2		0	2	0
2	3	1	3	1	0	1	0	2		0	2	0			
▶ upr_in	U 0	<table><tr><td>0</td><td>1</td><td>3</td><td>0</td><td colspan="2">3</td><td colspan="2">1</td><td>2</td><td>0</td><td>2</td><td>0</td><td>2</td></tr></table>	0	1	3	0	3		1		2	0	2	0	2
0	1	3	0	3		1		2	0	2	0	2			
WE	U 1														
data	U 0														
out_1	U 0														
out_2	U 0														

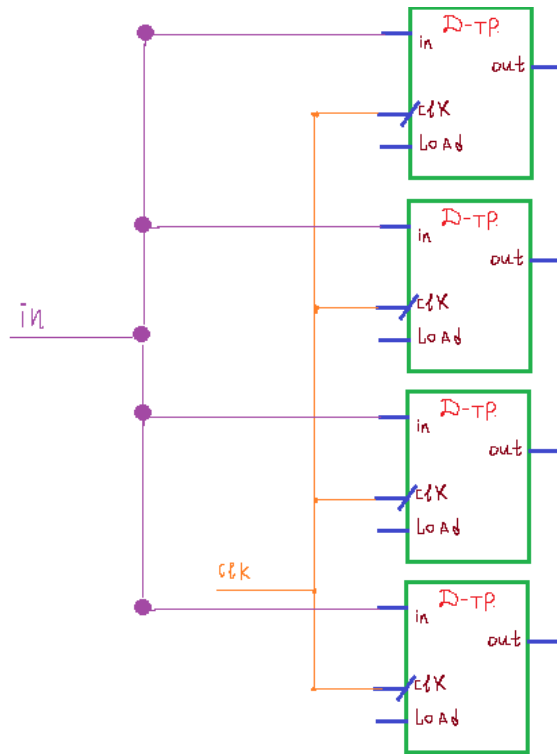
Ну что, работать с ним мы научились более-менее (точнее, поняли очевидность работы с ним), давайте теперь его соберем из триггеров (4 ячейки памяти, каждая ячейка содержит 1 бит).

Построение 4-битного регистрового файла на D-триггерах.

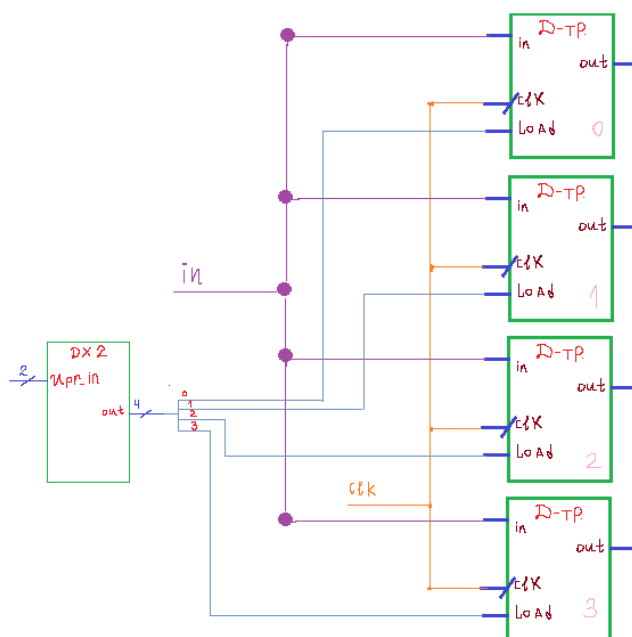
Хоть тут и будет построен только 4-битный регистровый файл с 1 выходом, у которого 1-битная ячейка памяти, но принцип построения 32-битного регистра, у которого 32-битная ячейка памяти будет таким же.

Ну что, давайте строить схему

Основа – 4 D-триггера с общим clk и in



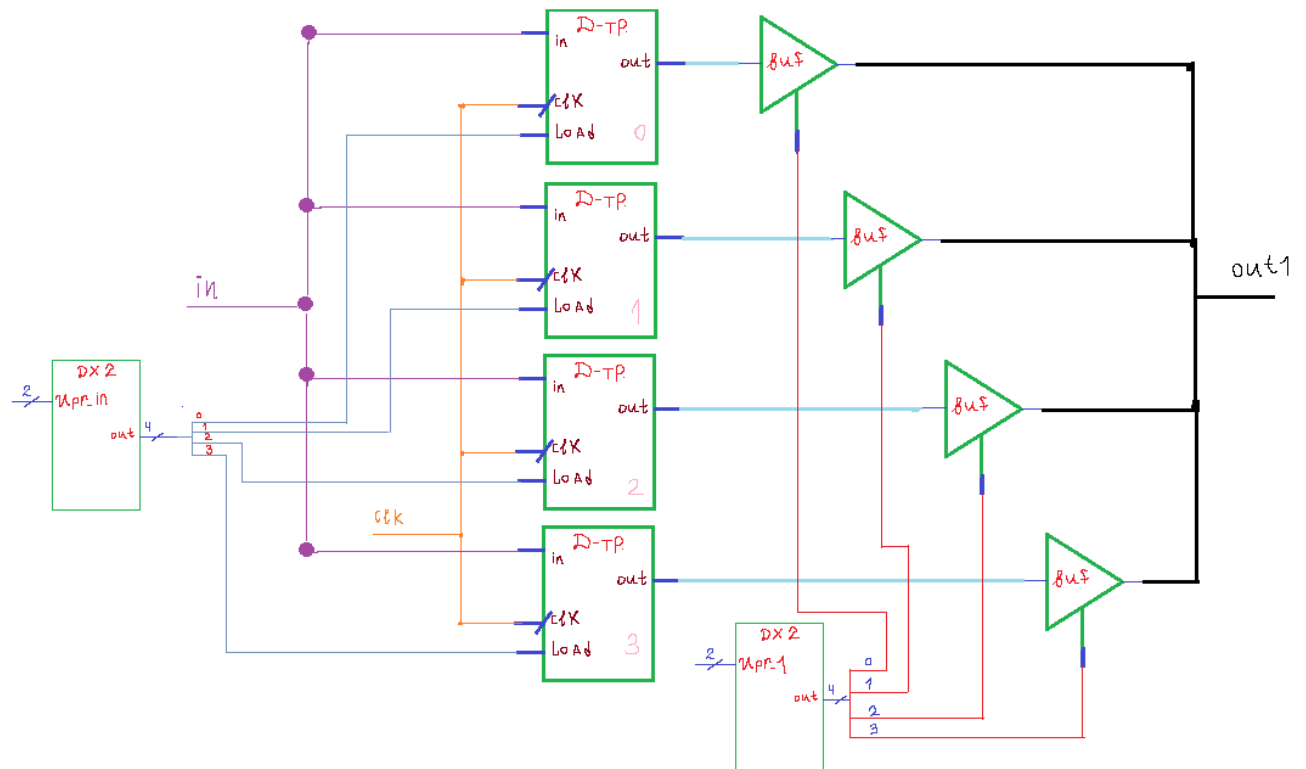
Выбор ячейки для записи будет осуществляться дешифратором на сигналы Load. Также, пронумеруем ячейки памяти



Ну что, вот мы реализовали запись в наш регистровый файл (по управляющей шине приходит 1 на выбранный триггер, и он записывает это состояние), давайте реализуем считывание.

Для считывания будем применять описанные выше буферы с 3 состояниями, которыми управлять будем с помощью дешифратора (позднее я узнал, что для выхода RF предпочтительнее мультиплексор вместо буферов).

Реализация считывания:



Таким образом, нами был реализован вот такой вот упрощенный регистровый файл:



Описание на Verilog регистрового файла вручную (не через встроенную ячейку памяти).

Способ описания на Verilog, представленный ниже, является несколько избыточным (потому как регистровый файл можно строчек на 20 кода реализовать с помощью встроенной синхронной ячейки памяти, но методически верный (если я уж наговорил красивой абстрактной теории про реализацию RF через триггеры, которыми управляют дешифраторы, то хотелось бы данную теорию увидеть в практической реализации и протестировать). Сделаем, для простоты и компактности RF всего-то на 4 бита.

План реализации регистрового файла:

- 1) D триггер с сигналом Load
- 2) буферы с 3 состояниями для выходов регистрового файла
- 3) Дешифратор DX2
- 4) Все соединяем и делаем Write Enable (WE)
- 5) Тестируем

D-триггер с сигналом Load

По смыслу, если Load = 1, то запись в триггер разрешена (out = data), если Load = 0, то запись в триггер запрещена (out = out). Тогда, мы можем записать логическое выражение для out через data и Load:

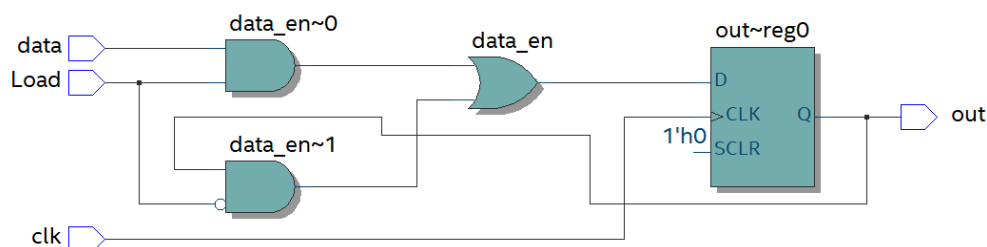
$$out = data \& Load \mid out \& \overline{Load}$$

Опишем данное устройство на Verilog:

Код на Verilog

```
1 module D_trig_s_en (  
2     input data,  
3     input Load,  
4     input clk,  
5     output reg out  
6 );  
7  
8  
9 wire data_en;  
10 assign data_en = data & Load | out & ~Load;  
11  
12 always @(posedge clk) begin  
13     out <= data_en;  
14 end  
15  
16 endmodule
```

Схема



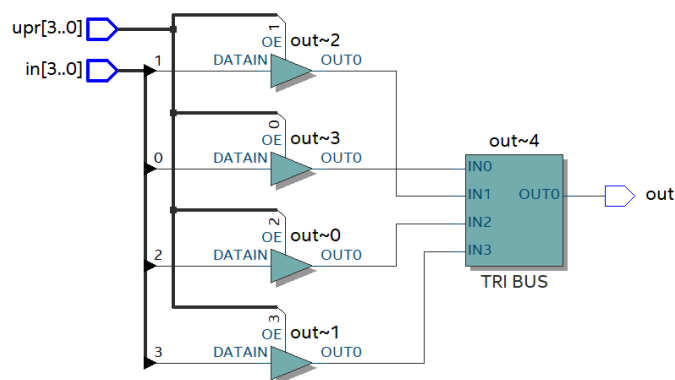
Буфер с 3 состояниями

Вместо буферов с 3 состоянием можно было бы воспользоваться просто мультиплексором, но я решил использовать их.

Код на Verilog.

```
1 module buf_z (
2     input [3:0] in,
3     input [3:0] upr,
4     output out
5 );
6
7 assign out = (upr [0] == 1) ? in [0] : 1'bz;
8 assign out = (upr [1] == 1) ? in [1] : 1'bz;
9 assign out = (upr [2] == 1) ? in [2] : 1'bz;
10 assign out = (upr [3] == 1) ? in [3] : 1'bz;
11
12 endmodule
```

RTL схема.



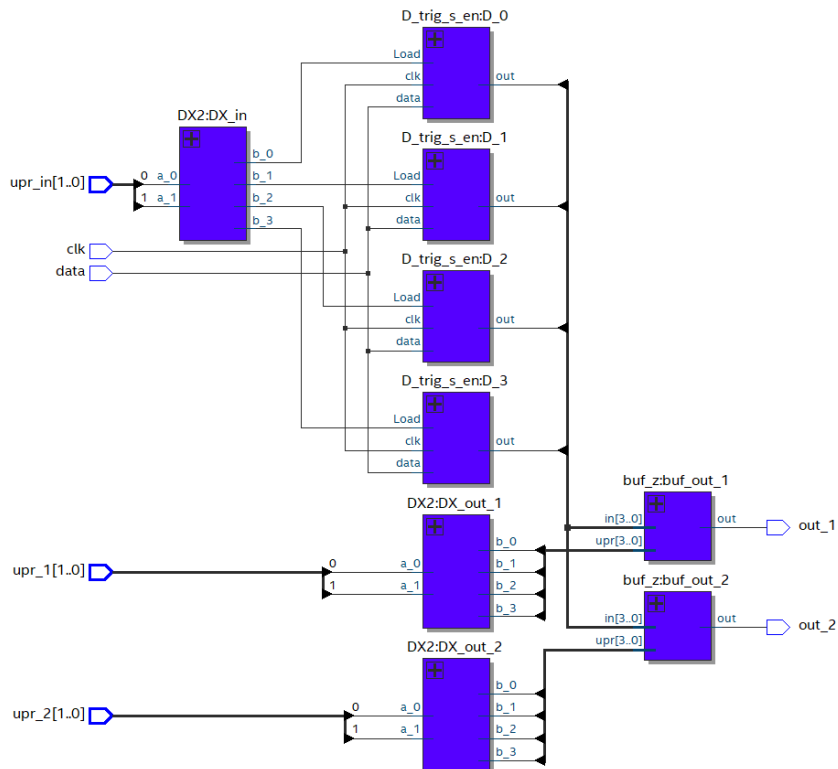
Дешифратор DX2 (с enable и без enable).

Раз уж решил описывать каждый модуль, то и DX2 оставлю тут.

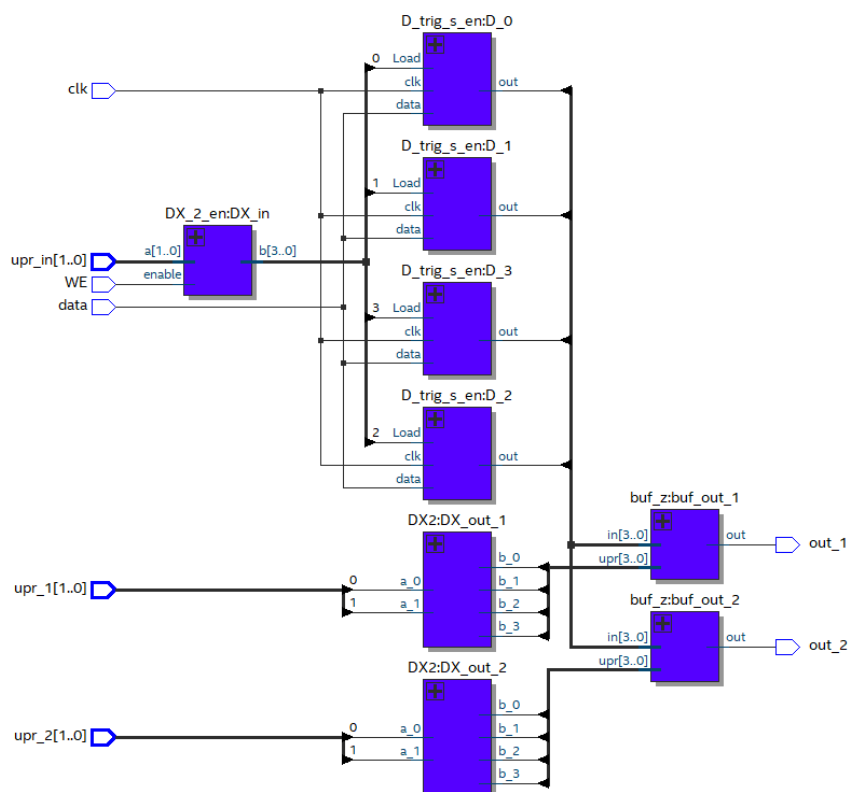
Код на Verilog

```
1 module DX2(
2     input a_0,
3     input a_1,
4     output b_0,
5     output b_1,
6     output b_2,
7     output b_3
8 );
9
10 assign b_0 = ~a_0&~a_1;
11 assign b_1 = a_0&~a_1;
12 assign b_2 = ~a_0&a_1;
13 assign b_3 = a_0&a_1;
14
15 endmodule
16
17 module DX_2_en (
18     input [1:0] a,
19     input enable,
20     output [3:0] b
21 );
22
23 assign b[0] = ~a[0] & ~a[1] & enable;
24 assign b[1] = a[0] & ~a[1] & enable;
25 assign b[2] = ~a[0] & a[1] & enable;
26 assign b[3] = a[0] & a[1] & enable;
27
28 endmodule
```

Схема регистрового файла без WE



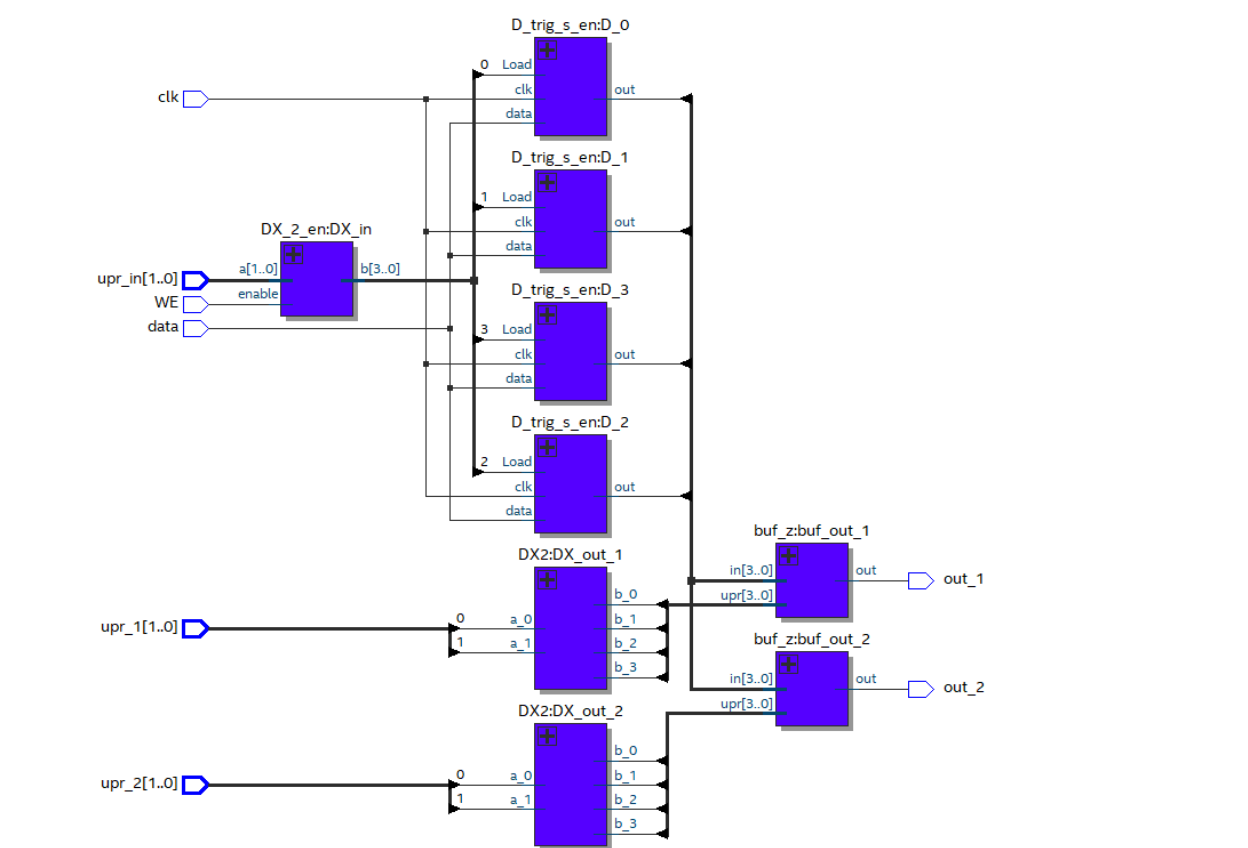
Для реализации сигнала Write-Enable просто поставим вместо обычного входного дешифратора дешифратор с enable (тогда при WE = 0 он выдает все 0 на выходе и ячейки памяти не будут активными для записи)



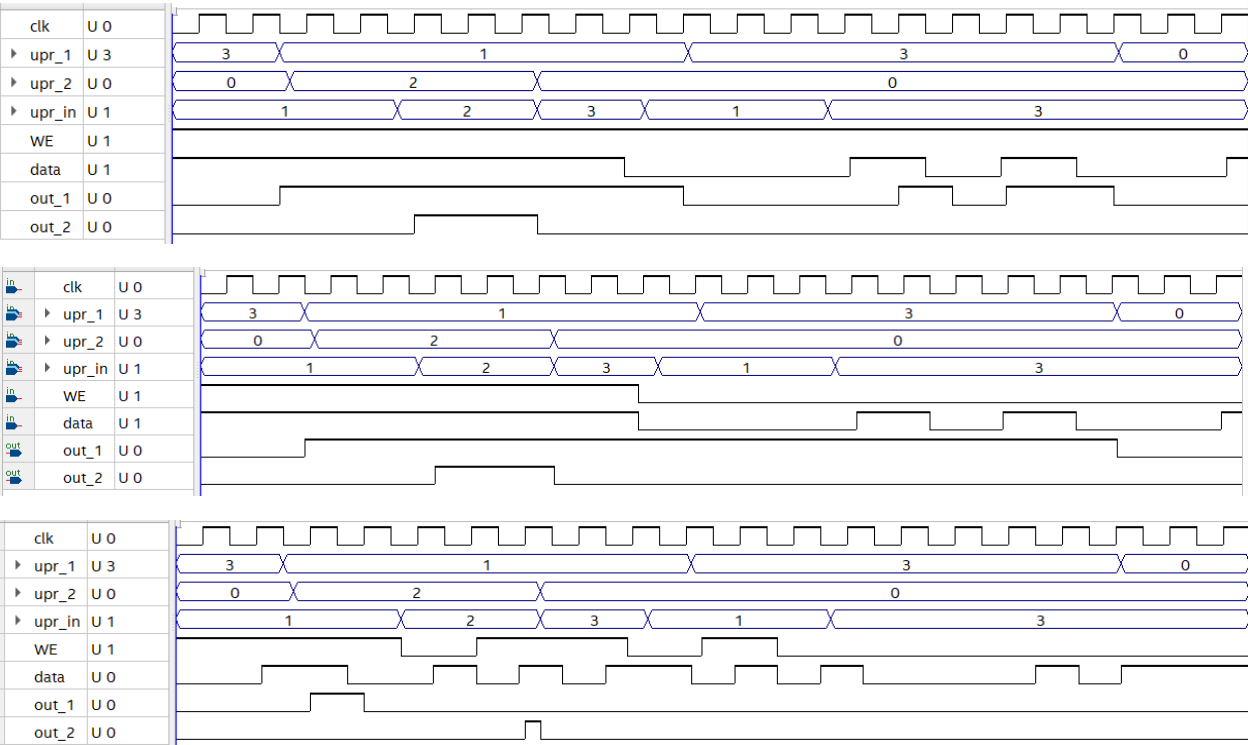
Код на Verilog.

```
1  module my_RF (
2      input [1:0] upr_1,
3      input [1:0] upr_2,
4      input clk,
5      input [1:0] upr_in,
6      input data,
7      input WE,
8      output out_1,
9      output out_2
10 );
11
12 wire [3:0] out_dx_in;
13 wire [3:0] out_dx_1;
14 wire [3:0] out_dx_2;
15 wire [3:0] out_dx_upr_buf_1;
16 wire [3:0] out_dx_upr_buf_2;
17 wire [3:0] in_buf;
18
19
20 buf_z buf_out_1(
21     .in(in_buf),
22     .upr(out_dx_1),
23     .out(out_1)
24 );
25
26 buf_z buf_out_2(
27     .in(in_buf),
28     .upr(out_dx_2),
29     .out(out_2)
30 );
31
32 DX_2_en DX_in (
33     .a(upr_in),
34     .enable(WE),
35     .b(out_dx_in)
36 );
37
38 DX2 DX_out_1 (
39     .a_0(upr_1[0]),
40     .a_1(upr_1[1]),
41     .b_0(out_dx_1[0]),
42     .b_1(out_dx_1[1]),
43     .b_2(out_dx_1[2]),
44     .b_3(out_dx_1[3])
45 );
46
47 DX2 DX_out_2 (
48     .a_0(upr_2[0]),
49     .a_1(upr_2[1]),
50     .b_0(out_dx_2[0]),
51     .b_1(out_dx_2[1]),
52     .b_2(out_dx_2[2]),
53     .b_3(out_dx_2[3])
54 );
55
56 D_trig_s_en D_0(
57     .data(data),
58     .Load(out_dx_in[0]),
59     .clk(clk),
60     .out(in_buf[0])
61 );
62
63 D_trig_s_en D_1(
64     .data(data),
65     .Load(out_dx_in[1]),
66     .clk(clk),
67     .out(in_buf[1])
68 );
69
70 D_trig_s_en D_2(
71     .data(data),
72     .Load(out_dx_in[2]),
73     .clk(clk),
74     .out(in_buf[2])
75 );
76
77 D_trig_s_en D_3(
78     .data(data),
79     .Load(out_dx_in[3]),
80     .clk(clk),
81     .out(in_buf[3])
82 );
83
84 endmodule
```

Схема регистрового файла с WE.



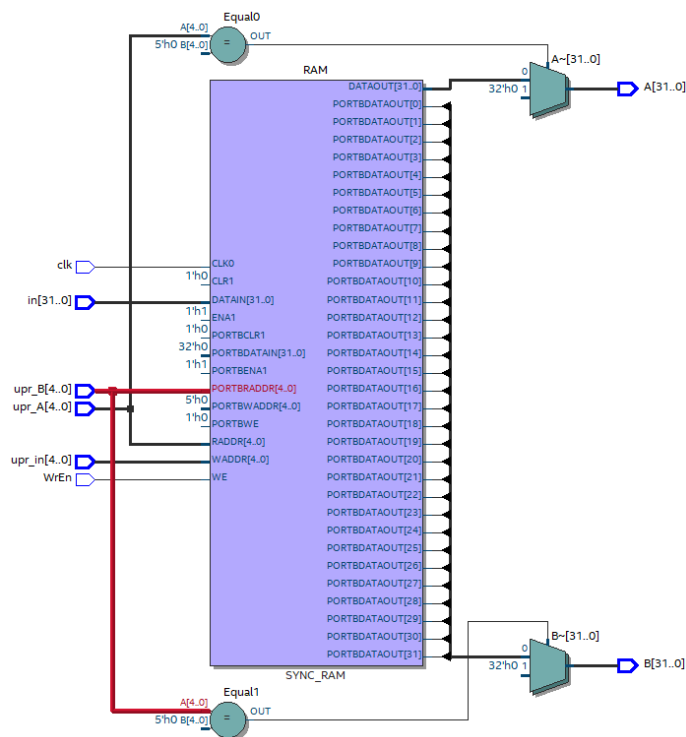
Тестирование регистрового файла.



Регистровый файл через встроенную ячейку памяти на Verilog (32 ячейки, 32 бита каждая, в нулевой ячейке всегда 0).

Данный способ куда более быстрый и генерирует просто готовый регистровый файл, именно данным способом обычно быстро описывают Register File как отдельный блок при проектировании процессора

```
1 module Register_file(  
2     input [4:0] upr_A,  
3     input [4:0] upr_B,  
4     input [4:0] upr_in,  
5     input [31:0] in,  
6     output [31:0] A,  
7     output [31:0] B,  
8     input clk,  
9     input WrEn  
10 );  
11  
12 reg [31:0] RAM [0:31];  
13  
14 assign A = (upr_A == 5'd0) ? 32'd0: RAM [upr_A];  
15 assign B = (upr_B == 5'd0) ? 32'd0: RAM [upr_B];  
16  
17 always @(posedge clk) begin  
18     if (WrEn) begin  
19         RAM[upr_in] <= in;  
20     end  
21 end  
22 endmodule
```



Заключение

В данной статье была разобрана тривиальность как применения регистрового файла, так и его устройства (Д-триггеры и 2 дешифратора), чего, увы, нет в том же Харис-Харисе, из-за чего я сильно тупил над данной темой.

Лекция 3. Немного про память команд.

Вступление

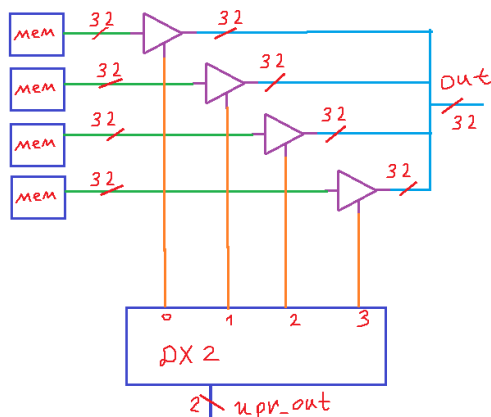
В данной небольшой заметке хотел бы затронуть такой примитив высшей цифровой схемотехники как память команд (играет роль хранителя прошивки для процессора). По сути, это просто постоянная память (ROM), которая по управляющему входу задает выход (это не всегда так, но в довольно простых процессорах данное утверждение верно (а в более сложных структурах иногда эту память заменяют на аналог регистрового файла с некоторыми нюансами)).

Про схему памяти команд

Как мы поняли ранее в вступлении, память команд – постоянная память (TE ROM, например, Flash-память), так что реализовывать будем ее с “волшебными” ячейками памяти и рассмотрим, собственно, саму организацию ROM.

ROM с 4 32-битными ячейками памяти

Разумеется, на выход можно было бы поставить MUX, но в микросхемах памяти предпочитают использовать буферы с 3 состоянием (опять-таки, позднее автор узнал, что для малых элементов памяти мультиплексор предпочтительнее. Про то, как реализуются сами ячейки памяти, мы не думаем — это не наш уровень абстракции.

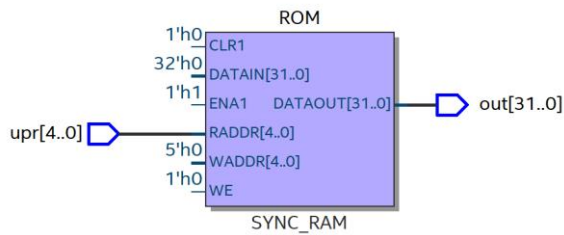


Ну что, абстрактными красивыми словами поговорили, давайте теперь напишем код на Verilog и посмотрим, во что он синтезируется.

Код на Verilog

```
1 module command_memory (
2
3     input  [4:0] upr,
4     output [31:0] out
5
6 );
7
8
9     reg [31:0] ROM [0:31];
10
11     initial $readmemb ("Memory.txt",ROM);
12     assign out = ROM[upr];
13
14
15 endmodule
```

Схема



Данная строчка просто задает значения для памяти (которые лежат в текстовике)

```
initial $readmemb ("Memory.txt",ROM);
```

Содержимое текстовика (памяти команд)

```
00101000000000000000000000000000
00101000000000000000000000000000
00100000000000000000000000000001
00110000000000000000000000000000
00110000100010001000000000000000
00000000000011000000000000000000
01000110000000000000000000000000
```

Тестирование памяти команд.

upr	U 0	0			1			2			
out	B 0010100	00101000000000000000000000000000			00101000000000000000000000000000			00100000000000000000000000000001			
upr	U 3	3			4			5			
out	B 0011000	0011000000000100011000000000			001100001000100010000001000000000			00000000000011000000000000000000			
upr	U 6	6			7			8			9
out	B 0100011	01000110000000000100000011111101			00000000000000000000000000000000						

Разумеется, можно вбивать числа и в 16-ричной системе в текстовик, в коде же вместо буквы b (binary) надо будет поставить букву h (hexadecimal).

Как мы видим, ничего особо не поменялось.

```
1 module command_memory (
2
3     input [4:0] upr,
4     output [31:0] out
5
6 );
7
8 reg [31:0] ROM [0:31];
9
10 initial $readmemb ("HMemory.txt",ROM);
11 assign out = ROM[upr];
12
13
14 endmodule
```

Содержимое текстовика (памяти команд).

```
AAAAAAA
BBBBBBB
01234567
9ABCDEF0
11223344
```

Тестирование памяти команд

upr	0	1	2	3	4	5	6	7	8	9
out	AAAAAAA	BBBBBBB	01234567	9ABCDEF0	11223344					00000000

Лекция 4. Простейший процессор.

Введение

Автор в данной статье расскажет про весьма сильно упрощенный процессор “Кобра”, архитектура которого что-то около RISC-V, а методически он весьма неплохо показывает устройство процессора и принцип его работы (мне понравилось, потому как сразу в лицо кидать декодер инструкций, конвейеры, параллелизм и тд такое себе). Если же добавить немного спойлеров из следующих лекций, то данный процессор функционально – это RISC-V, реализующий I_1, B, R.

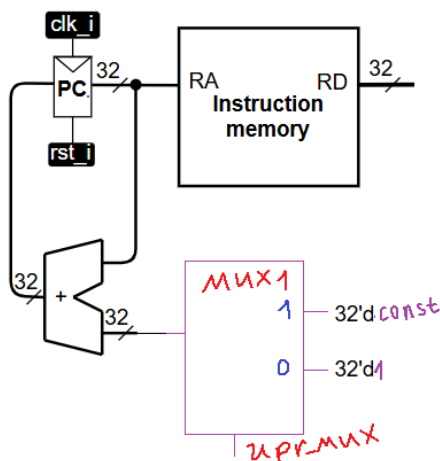
Про микроархитектуру процессора.

При рассказе про различные микроархитектуры есть 2 разных подхода: нарисовать картинку красивую и потом объяснять, что она делает, или же поставить создание процессора/блока процессора в качестве задачи и вместе с читателем решать ее. Я пойду этим путем, и построю архитектуру процессора из того, что он должен делать.

При таком подходе реализуются сначала куски архитектуры, которые потом будут соединяться между собой (но это потом, а сейчас давайте не сильно задумываться про то, откуда мы берем различные управляющие сигналы и константы)

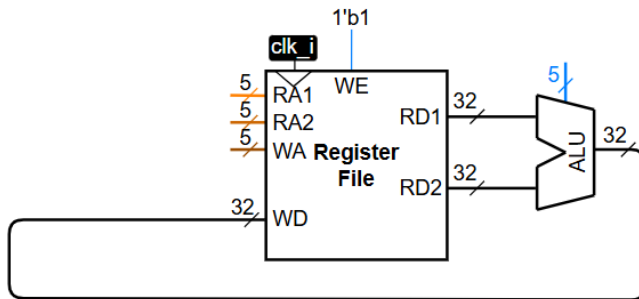
1) Он должен уметь перескакивать с 1 строчки программы (содержания памяти команд) на произвольную другую по некоему условию, а если нет, то просто идти сверху вниз (аппаратно реализованные циклы и if-else)

Реализуем сначала аппаратно возможность перескакивать на разные инструкции с помощью мультиплексора.



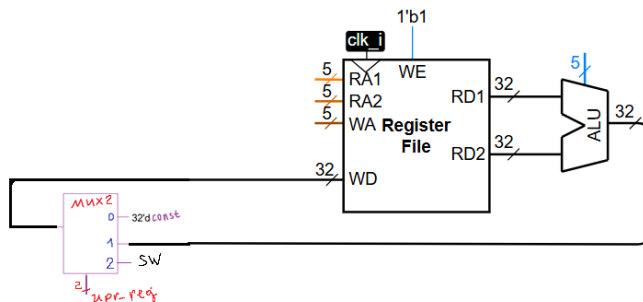
Также, отложим у себя в голове, что надо где-то завести провод, который будет отвечать за переключение мультиплексора (назовем данный провод условным, так-как он будет отвечать за то, работает ли чтение программы в типичном режиме, или же мы перескакиваем куда-то вверх или вниз по выполнению условия).

2) Он должен делать какие-то вычисления и куда-то результаты вычислений складывать, то есть надо АЛУ и память (в качестве которой будет выступать регистровый файл).



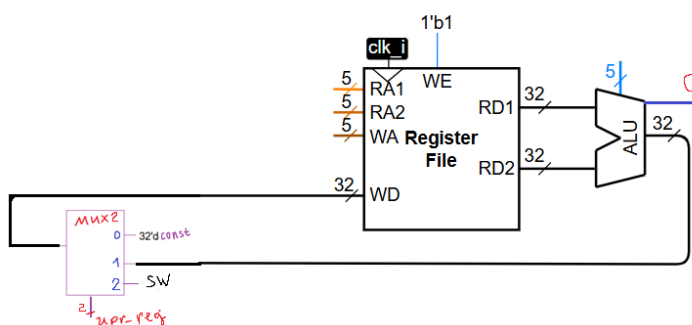
На рисунке выше изображена условная ситуация, хотелось бы записывать данные в регистровый файл не только с АЛУ, но также константу и с внешних проводов (это вход SW)

Так что более-менее адекватно будет выглядеть такая конфигурация:



Разумеется, сюда можно добавить еще как минимум 1 вход (например, константный 0), но это уже на усмотрение читателя.

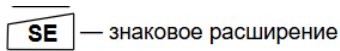
3) Не будем также забывать о том, что нам нужен некий индикатор в АЛУ, например, сравнения 2 чисел, который я назову C (да, тот самый флаг сравнения).



4) Надо откуда-то брать константу, предлагается первые 8 бит команды отдать под константу, но, так-как мы использовали 32-битное представление константы выше, то нам надо расширить константу с 8 бит до 32 (и сохранить знак, так-как константа может быть отрицательным числом).

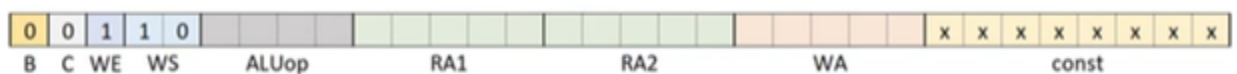
Решается это весьма просто: мы просто припаиваем 24 провода к 7 (считаем с 0 провода), возможно поставив буфер для более быстрой перезарядки 25 проводов 1 выходом. Таким образом и получается 32-битная константа с сохранением знака.

Данное действие выполняет блок SE – знаковое расширение



5) Также, надо какие-то биты в команде отдать и под управление переходами в программе (если равен единице бит безусловного перехода, то у нас независимо от результата на АЛУ счетчик должен измениться на константу), а если равен единице бит условного перехода, то уже скачок на константу будет только если на АЛУ индикатор C = 1

Таким образом, получаем некое разбиение 32-битной команды:



Что нового относительно прошлых лекций:

Разумеется, можно было бы разбить и по-другому, но так будет более-менее близко к реальному процессору.

B – бит безусловного перехода (если он 1, то, например, на 3 инструкции вверх (если константа равна -3))

C – бит условного перехода (если он 1 и C на АЛУ 1, то у нас, опять-таки, на 3 инструкции вверх (const = -3))

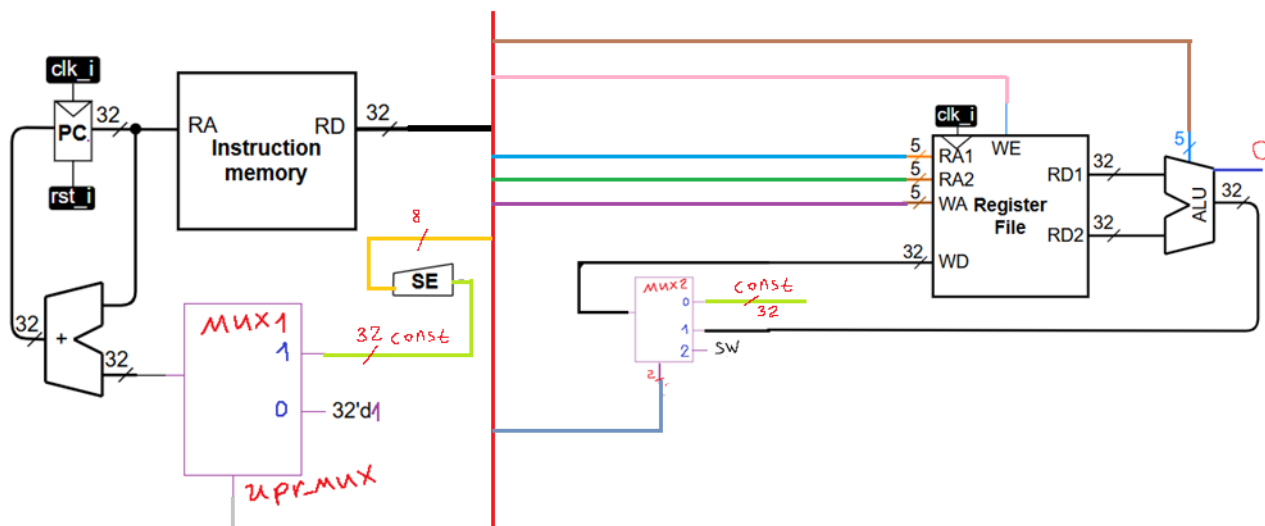
WE – записываем/не записываем результат АЛУ в регистровый файл

WS – управление входом в регистровый файл (что записываем: константу, выход АЛУ, с внешних источников?)

Остальные биты инструкции отвечают за выбором операции на АЛУ, за тем, какую ячейку памяти вывести на каждый из 2 выходов регистрового файла, в какую ячейку регистрового файла записываем, 8 первых бит - константа.

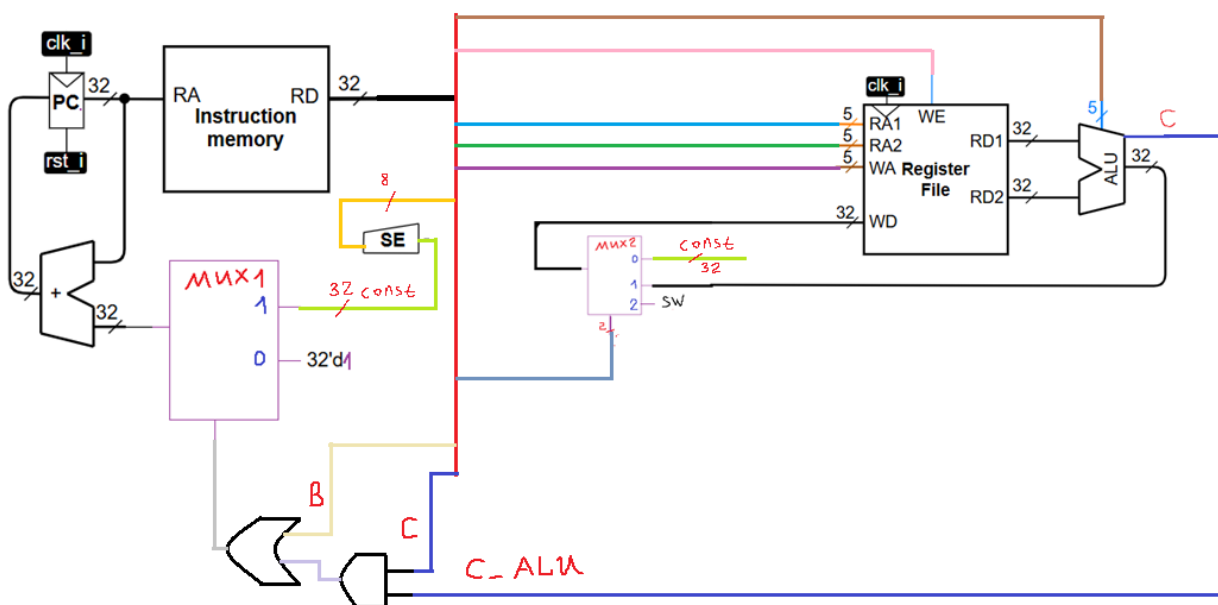
Соединяем блоки

Далее мы просто берем и различные провода из прошивки (памяти команд) присоединяем к регистровому файлу, АЛУ, мультиплексорам и тд:



Читатель сам, при желании, может расставить по разбиению команды номера проводов, которые идут от памяти инструкций к логике.

Мы же сейчас займемся последним штришком: будем реализовывать условный и безусловный перенос. Достигается это весьма просто : ставим И между C и C_ALU, а на вход upr_mux подаем В ИЛИ (C и C_ALU) (что полностью удовлетворяет требованиям условного и безусловного перехода).



Вот и все, программируемое устройство, весьма сильно напоминающее процессор готово!

Далее пойдет уже мой код на Verilog в котором я его описываю:

Мультиплексор на входе в регистровый файл:

```
1 module mux_RF (
2     input [1:0] upr_data,
3     input [31:0] SW,
4     input [31:0] work_const,
5     input [31:0] OUT_ALU,
6     output reg [31:0] data_RF
7 );
8 always @(*) begin
9     case (upr_data[1:0])
10        2'd0: data_RF = work_const;
11        2'd1: data_RF = SW;
12        2'd2: data_RF = OUT_ALU;
13        default: begin
14            data_RF = 0;
15        end
16    endcase;
17 end
18 endmodule
```

Регистровый файл:

```
1 module Register_file(
2     input [4:0] upr_A,
3     input [4:0] upr_B,
4     input [4:0] upr_in,
5     input [31:0] in,
6     output [31:0] A,
7     output [31:0] B,
8     input clk,
9     input WrEn
10 );
11
12 reg [31:0] RAM [0:31];
13
14 assign A = (upr_A == 5'd0) ? 32'd0: RAM [upr_A];
15 assign B = (upr_B == 5'd0) ? 32'd0: RAM [upr_B];
16
17 always @(posedge clk) begin
18     if (WrEn) begin
19         RAM[upr_in] <= in;
20     end
21 end
22 endmodule
```

Модули АЛУ (кроме знакового компаратора, но он в конце статьи есть)

```
1 module sl_mod2 (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out
5 );
6
7 assign out = A ^ B;
8
9 endmodule
```

```
1 module or_32_b (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out_or
5 );
6
7 assign out_or = A | B;
8
9 endmodule
```

```
1 module and_32_b (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out_and
5 );
6
7 assign out_and = A & B;
8
9 endmodule
```

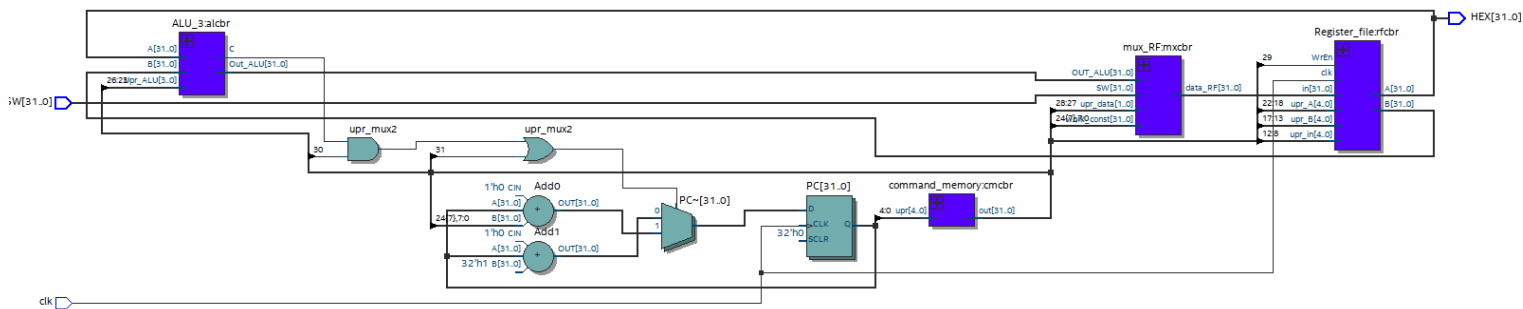
АЛУ (заметьте, оно больше процессора)

```
1 module ALU_3 (
2     input [3:0] Upr_ALU,
3     input [31:0] A,
4     input [31:0] B,
5     output C,
6     output [31:0] Out_ALU
7 );
8
9 parameter ADD = 4'd0;
10 parameter SUB = 4'd1;
11 parameter SLL = 4'd2;
12 parameter SLTS = 4'd3;
13 parameter SLTU = 4'd4;
14 parameter XOR = 4'd5;
15 parameter SRL = 4'd6;
16 parameter SRA = 4'd7;
17 parameter OR = 4'd8;
18 parameter AND = 4'd9;
19 parameter EQ = 4'd10;
20 parameter NE = 4'd11;
21 parameter LTS = 4'd12;
22 parameter GES = 4'd13;
23 parameter LTU = 4'd14;
24 parameter GEU = 4'd15;
25
26 wire [31:0] out_or;
27 wire [31:0] out_and;
28 wire [31:0] out_add;
29 wire [31:0] out_sub;
30 wire [31:0] out_xor;
31 wire [31:0] out_sll;
32 wire [31:0] out_slts;
33 wire [31:0] out_sltu;
34 wire [31:0] out_srl;
35 wire [31:0] out_sra;
36 wire out_A_menshe_B_zn;
37 wire out_A_ne_menshe_B_zn;
38 wire out_A_menshe_B;
39 wire out_A_ne_menshe_B;
40 wire out_equal;
41 wire out_notequal;
42
43 assign out_A_ne_menshe_B_zn = ~out_A_menshe_B_zn;
44 assign out_A_ne_menshe_B = ~out_A_menshe_B;
45 assign out_equal = (A==B) ? 1'b1 : 1'b0;
46 assign out_A_menshe_B = (A<B) ? 1'b1 : 1'b0;
47 assign out_notequal = ~out_equal;
48
49 assign out_add = A+B;
50 assign out_sub = A-B;
51 assign out_sll = A << B;
52 assign out_slts = {{31{1'b0}},out_A_menshe_B_zn};
53 assign out_sltu = {{31{1'b0}},out_A_menshe_B};
54 assign out_srl = A >> B;
55 assign out_sra = A >>>B;
56
57
58 assign Out_ALU = (Upr_ALU == ADD) ? out_add:
59                 (Upr_ALU == SUB) ? out_sub:
60                 (Upr_ALU == SLL) ? out_sll:
61                 (Upr_ALU == SLTS) ? out_slts:
62                 (Upr_ALU == SLTU) ? out_sltu:
63                 (Upr_ALU == XOR) ? out_xor:
64                 (Upr_ALU == SRL) ? out_srl:
65                 (Upr_ALU == SRA) ? out_sra:
66                 (Upr_ALU == OR) ? out_or:
67                 (Upr_ALU == AND) ? out_and:32'b0;
68
69 assign C = (Upr_ALU == EQ) ? out_equal:
70           (Upr_ALU == NE) ? out_notequal:
71           (Upr_ALU == LTS) ? out_A_menshe_B_zn:
72           (Upr_ALU == GES) ? out_A_ne_menshe_B_zn:
73           (Upr_ALU == LTU) ? out_A_menshe_B:
74           (Upr_ALU == GEU) ? out_A_ne_menshe_B:1'b0;
75
76
77 or_32_b or_ALU (
78     .A(A),
79     .B(B),
80     .out_or(out_or)
81 );
82
83
84 and_32_b and_ALU (
85     .A(A),
86     .B(B),
87     .out_and(out_and)
88 );
89
90
91 sl_mod2 mod2_ALU (
92     .A(A),
93     .B(B),
94     .out(out_xor)
95 );
96
97 comp_menshe zn_comp (
98     .A(A),
99     .B(B),
100    .out(out_A_menshe_B_zn)
101 );
102 endmodule
```

Кобра, описание на Verilog:

```
1 module cobra (
2     input clk,
3     input [31:0] SW,
4     output [31:0] HEX
5 );
6
7
8 wire [4:0] upr_in;
9 wire [4:0] upr_1;
10 wire [4:0] upr_2;
11 wire [31:0] A;
12 wire [31:0] B;
13 wire WrEn;
14 reg [31:0] data_RF;
15 wire [31:0] pr_data_RF;
16 wire [31:0] instr;
17 wire [3:0] UPR_ALU;
18 wire [31:0] OUT_ALU;
19 reg [31:0] PC = 32'b0;
20 wire c;
21 wire [31:0] work_const;
22 wire upr_mux2;
23 wire [1:0] upr_data;
24
25
26 assign upr_in = instr[12:8];
27 assign upr_1 = instr[22:18];
28 assign upr_2 = instr[17:13];
29 assign UPR_ALU = instr[26:23];
30 assign WrEn = instr[29];
31 assign upr_mux2 = (c&instr[30])|instr[31];
32 assign work_const = {{24{instr[7]}}}, instr[7:0] };
33 assign upr_data = instr[28:27];
34 assign pr_data_RF = data_RF;
35 assign HEX = A;
36
37 Register_file rfcbr (
38     .upr_A(upr_1),
39     .upr_B(upr_2),
40     .upr_in(upr_in),
41     .in(data_RF),
42     .A(A),
43     .B(B),
44     .clk(clk),
45     .WrEn(WrEn)
46 );
47
48 always @(posedge clk) begin
49     if (upr_mux2==1) begin
50         PC <= PC + work_const;
51     end
52
53     else begin
54         PC <= PC + 1;
55     end
56 end
57
58 mux_RF mxcbr (
59     .upr_data(upr_data),
60     .SW(SW),
61     .work_const(work_const),
62     .OUT_ALU(OUT_ALU),
63     .data_RF(pr_data_RF)
64 );
65
66 ALU_3 alcbr (
67     .Upr_ALU(UPR_ALU),
68     .A(A),
69     .B(B),
70     .C(c),
71     .Out_ALU(OUT_ALU)
72 );
73
74 command_memory cmcbr (
75     .upr(PC),
76     .out(instr)
77 );
78
79
80 endmodule
```

RTL cxema Cobra:



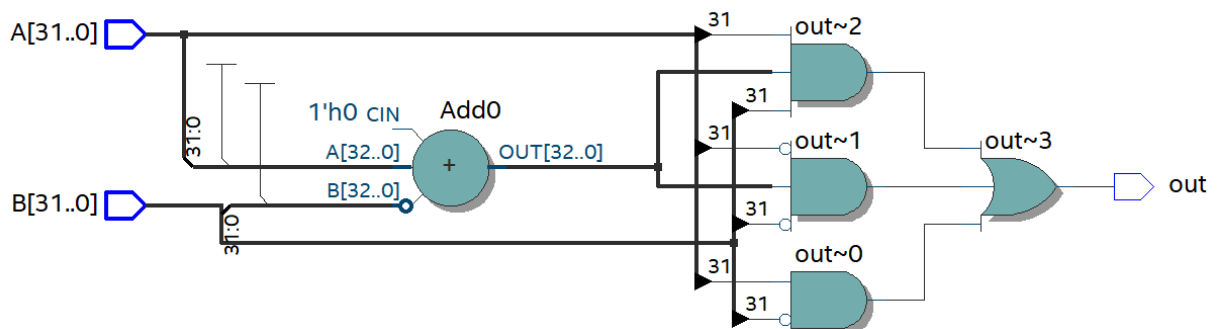
Вывод

Мы теоретически разобрали простейший недопроцессор и даже описали его на Verilog и посмотрели, во что он синтезируется (а еще поняли, что процессор = очевидная штукавина). Да, для реального процессора это примерно скелет, нету даже элементарного декодера инструкций, но как ступенька для развития вполне себе подходит.

Дополнение: компаратор знакового сравнения “меньше” (к АЛУ).

```
1 module comp_menshe (
2     input [31:0] A,
3     input [31:0] B,
4     output out
5 );
6
7 wire [31:0] S;
8 assign S = A-B;
9 assign out = (A[31] & ~B[31]) | (~A[31] & ~B[31] & S[31]) | (A[31] & B[31] & S[31]);
10
11 endmodule
```

RTL схема знакового компаратора “А меньше В”

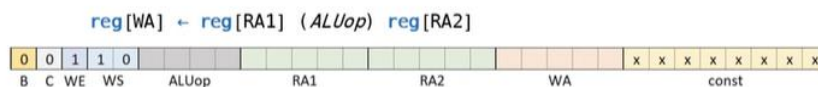


Лекция 5. Прошивка “Cobra”

Введение

В данной статье мы немного поиграем с написанным ранее процессором, прошивая его разными способами.

Операция на АЛУ над регистрами RA1 и RA2, результат записывается в WA



Загрузка константы в регистр WA



Пример 1. Найдём сумму 7 и 8. Номер операции ADD (ALU_OP = 0)

```
parameter ADD = 4'd0;
```

Код на Verilog, который отвечает за то, что будет загружено в регистровый файл (upr_data = WS).

```
case (upr_data[1:0])
  2'd0: data_RF = work_const;
  2'd1: data_RF = SW;
  2'd2: data_RF = OUT_ALU;
  default: begin
    data_RF = 0;
  end
endcase;
```

Строчка 1. Загрузка 7 в регистр 1

00 1 00 0000 00000 00000 00001 00000111

00000111₂ = 7 = константа, которую мы загружаем

00001₂ = 1 — регистр, в который мы записываем константу

1 — разрешаем запись в регистровый файл

Строчка 2. Загрузка 8 в регистр 2

00 1 00 0000 00000 00000 00010 00001000

00001000₂ = 8 = константа, которую мы загружаем

00010₂ = 2 — регистр, в который мы записываем константу

1 — разрешаем запись в регистровый файл

Строчка 3. Загрузка в 3 регистр суммы чисел из 1 и 2 регистра.

00 1 10 0000 00010 00001 00011 00000000

$10_2 = 2$ – на вход регистрового файла подается операция с АЛУ

$0000_2 = 0$ – номер операции на АЛУ

$00001_2 = 1$ – с этой ячейки памяти берем RA2

$00010_2 = 2$ – с этой ячейки памяти берем RA1

$00011_2 = 3$ – регистр, в который мы записываем результат операции на АЛУ

1 – разрешаем запись в регистровый файл

Строчка 4. Выводим значение с 3 ячейки регистрового файла (получаем результат)

00 0 00 0000 00011 00000 00000 00000000

$00011_2 = 3$ – выводим значение с 3 ячейки, тк снимаем данные с выхода 1 RF

Итоговая прошивка:

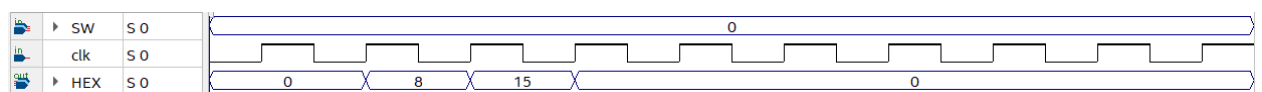
```
0010000000000000000000000100000111
00100000000000000000000001000001000
00110000000001000000100011000000000
00000000000001100000000000000000000
```

То, что должно быть по clk:

- 0) Загрузка 7 в регистр 1, на выходе RF (RA1) будет 0 ячейка, соответственно, покажет 0.
- 1) Загрузка 8 в регистр 2, на выходе RF (RA1) будет 0 ячейка, соответственно, покажет 0.
- 2) Загрузка суммы из 1 и 2 регистра в 3 регистр, (RA1 = 1), на выходе RF будет значение в 1 ячейке (то есть 8)
- 3) Вывод результата суммы, что записана в 3 ячейке (RA1 = 3), на выходе RF будет результат суммы 7 и 8 = 15

Далее же PC идет +1 и в IM есть только 32'b0 строчки, поэтому регистровый файл выводит значение с 0 ячейки (то есть 0)

Результат симуляции cobra с нашей прошивкой.



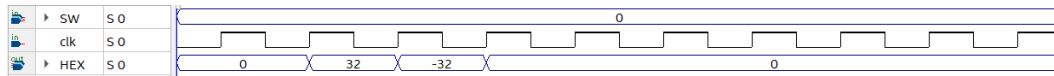
Как читатель видит, вышло ровно так, как и предполагалось.

Пример 2. Найдём 32-64

```
parameter SUB = 4'd1;
assign out_sub = A-B;
```

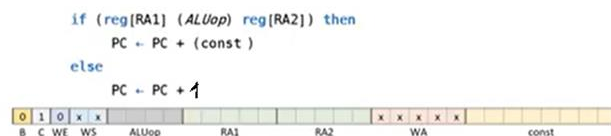

Далее же PC идет +1 и в IM есть только 32'b0 строчки, поэтому регистровый файл выводит значение с 0 ячейки (то есть 0)

Результат симуляции sobra с нашей прошивкой.



Поиграемся теперь с условным переносом (бит C).

В общем-то, условный перенос просто заставляет счетчик, который идеи по программе прибавлять не 1, а значение, заданное в константе, таким образом можно как идти наверх по программе, так и вниз, реализуя при этом циклы и операции if/else.



Код на Verilog, описывающий PC (с – флаг сравнения АЛУ).

```

assign upr_mux2 = (c&instr[30])|instr[31];
if (upr_mux2==1) begin
    PC <= PC + work_const;
end
else begin
    PC <= PC + 1;
end

```

Пример 3. Умножение 7 на 3 через константы.

Строчка 1. Загрузка 7 в регистр 1

00 1 00 0000 00000 00000 00001 00000111

00000111₂ = 7 = константа, которую мы загружаем

00001₂ = 1 – регистр, в который мы записываем константу

1 – разрешаем запись в регистровый файл

Строчка 2. Загрузка 3 в регистр 2

00 1 00 0000 00000 00000 00010 00000011

00000011₂ = 3 = константа, которую мы загружаем

00010₂ = 2 – регистр, в который мы записываем константу

1 – разрешаем запись в регистровый файл

Строчка 3. Загрузка 1 в регистр 4

00 1 00 0000 00000 00000 00100 00000001

00000001₂ = 1 = константа, которую мы загружаем

00100₂ = 4 – регистр, в который мы записываем константу

1 – разрешаем запись в регистровый файл

[illegible]

- 1) 0, 1, 2 clk – на выходе 0 (так-как идет запись в память)
- 2) На 3 clk выдаст 7, так-как оно лежит в регистре 1, к которому обращаются в команде 4.
- 3) На 4 clk выдаст значения счетчика (сколько раз надо сложить).
- 4) На 5 clk идет вывод значения суммы
- 5) Далее, пока счетчик больше 0, переходим к пункту 3 (но сначала выдаем 0, так-как $A = 0$)
- 6) После всего процесса выдаем 0.

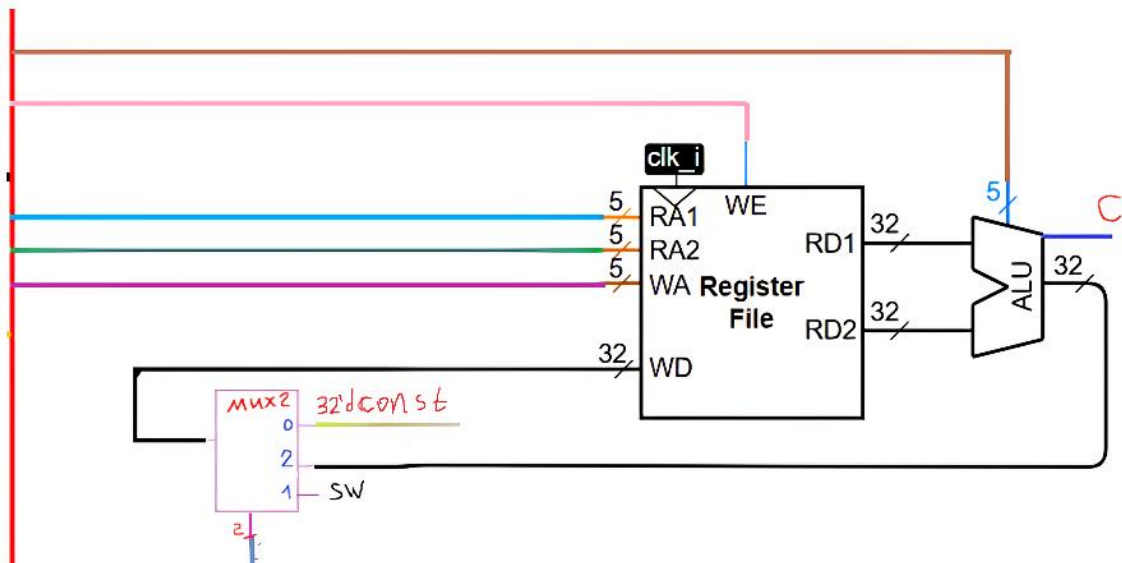
SW	S0	0
clk	S0	
HEX	S0	0 7 3 7 0 7 2 14 0 7 1 21 0

```
0010000000000000000000000000100011111
00100000000000000000000000001000000011
00100000000000000000000000001000000001
00110000000001000110001100000000
00110000100010001000001000000000
00000000000001100000000000000000
01000110000000000100000011111101
```

SW	S0	0
clk	S0	
HEX	S0	0 31 3 31 0 31 2 62 0 31 1 93 0

99

Для примера давайте рассмотрим теперь программу умножения 7 на 3, но 7 и 3 загружаются с входа SW. На самом деле, отличаться будет только управляющий сигнал на MUX у входа в регистровый файл (из рисунка видно, что надо туда поставить 2 (upr_data = WS = 1))



Описание этого элемента на Verilog в cobra.

```
case (upr_data[1:0])
  2'd0: data_RF = work_const;
  2'd1: data_RF = SW;
  2'd2: data_RF = OUT_ALU;
default: begin
  data_RF = 0;
end
```

В итоге, команда для записи значения с внешних выводов выглядит так:



Строчка 1. Загрузка числа 1 (в SW выставим 7) в регистр 1

00 1 01 0000 00000 00000 00001 00000000

00001₂ = 1 – регистр, в который мы записываем константу

01₂ = 1 – показывает, что запись данных будет с SW

1 – разрешаем запись в регистровый файл

Строчка 2. Загрузка числа 2 (в SW выставим 3) в регистр 2

00 1 01 0000 00000 00000 00010 00000000

00010₂ = 2 – регистр, в который мы записываем константу

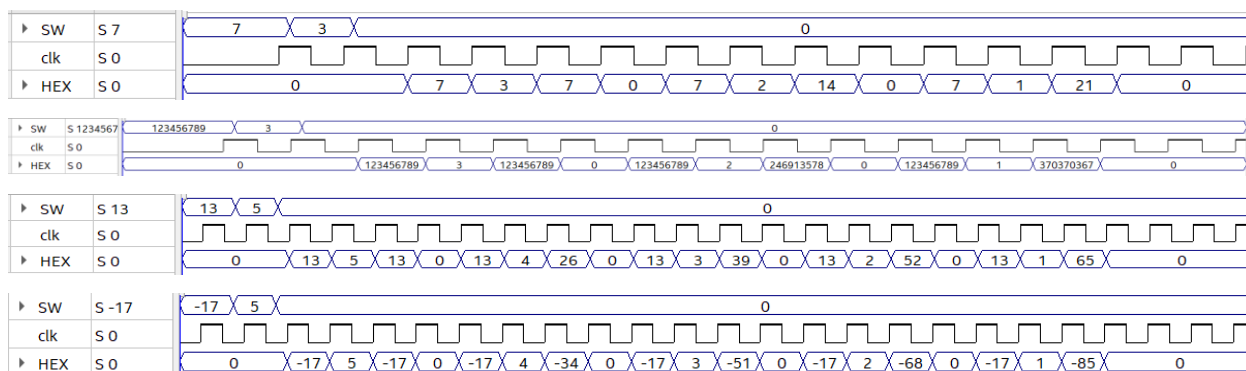
01₂ = 1 – показывает, что запись данных будет с SW

1 – разрешаем запись в регистровый файл

Далее же код будет полностью аналогичен коду с примера 3.

[illegible]

Результаты симуляции на соба с нашей прошивкой.



SW	S-17	-17	-5	0		
clk	S0	[Clock Signal]				
HEX	S0	0	-17	-5	-17	0

Print (S)

Итак, разделим инструкции на типы. Тип инструкции зашьём в её последние 7 бит и назовём это дело опкодом (opcode), т.е. кодом операции. Далее, выделим первые 7 бит на конкретизацию операции. То есть, здесь мы можем сказать, какую именно инструкцию данного типа мы хотим выполнить. Это поле func7. Аналогично поступим ещё с 3 битами, и их назовём func3. risc-v модульный, а мы делаем пока самую простую его версию, поэтому func7 использоваться в этом смысле практически не будет, а вот func3 есть почти во всех инструкциях. Добавим к этому набору адреса регистров чтения и записи (15 бит), получим суммарно 7+7+3+15=32. После текстового описания давайте ознакомимся с картинкой из мануала к risc-v.

31	27	26	25	24	20	19	15	14	12	11	7	6	0							
func7				rs2		rs1		func3		rd		opcode		R-type						
imm[11:0]						rs1		func3		rd		opcode		I-type						
imm[11:5]				rs2		rs1		func3		imm[4:0]		opcode		S-type						
imm[12:10:5]				rs2		rs1		func3		imm[4:1:11]		opcode		B-type						
imm[31:12]										rd		opcode		U-type						
imm[20:10:11:19:12]										rd		opcode		J-type						

RV32I Base Instruction Set

imm[31:12]				rd		0110111		LUI
imm[31:12]				rd		0010111		AUIPC
imm[20:10:11:19:12]				rd		1101111		JAL
imm[11:0]				rs1	000	rd	1100111	JALR
imm[12:10:5]		rs2	rs1	000	imm[4:1:11]	1100011		BEQ
imm[12:10:5]		rs2	rs1	001	imm[4:1:11]	1100011		BNE
imm[12:10:5]		rs2	rs1	100	imm[4:1:11]	1100011		BLT
imm[12:10:5]		rs2	rs1	101	imm[4:1:11]	1100011		BGE
imm[12:10:5]		rs2	rs1	110	imm[4:1:11]	1100011		BLTU
imm[12:10:5]		rs2	rs1	111	imm[4:1:11]	1100011		BGEU
imm[11:0]			rs1	000	rd	0000011		LB
imm[11:0]			rs1	001	rd	0000011		LH
imm[11:0]			rs1	010	rd	0000011		LW
imm[11:0]			rs1	100	rd	0000011		LBU
imm[11:0]			rs1	101	rd	0000011		LHU
imm[11:5]		rs2	rs1	000	imm[4:0]	0100011		SB
imm[11:5]		rs2	rs1	001	imm[4:0]	0100011		SH
imm[11:5]		rs2	rs1	010	imm[4:0]	0100011		SW
imm[11:0]			rs1	000	rd	0010011		ADDI
imm[11:0]			rs1	010	rd	0010011		SLTI
imm[11:0]			rs1	011	rd	0010011		SLTIU
imm[11:0]			rs1	100	rd	0010011		XORI
imm[11:0]			rs1	110	rd	0010011		ORI
imm[11:0]			rs1	111	rd	0010011		ANDI
0000000		shamt	rs1	001	rd	0010011		SLLI
0000000		shamt	rs1	101	rd	0010011		SRLI
0100000		shamt	rs1	101	rd	0010011		SRAI
0000000		rs2	rs1	000	rd	0110011		ADD
0100000		rs2	rs1	000	rd	0110011		SUB
0000000		rs2	rs1	001	rd	0110011		SLL
0000000		rs2	rs1	010	rd	0110011		SLT
0000000		rs2	rs1	011	rd	0110011		SLTU
0000000		rs2	rs1	100	rd	0110011		XOR
0000000		rs2	rs1	101	rd	0110011		SRL
0100000		rs2	rs1	101	rd	0110011		SRA
0000000		rs2	rs1	110	rd	0110011		OR
0000000		rs2	rs1	111	rd	0110011		AND
fm	pred	succ	rs1	000	rd	0001111		FENCE
000000000000			00000	000	00000	1110011		ECALL
000000000001			00000	000	00000	1110011		EBREAK

RV32/RV64 Zicsr Standard Extension

csr	rs1	001	rd	1110011	CSRRW
csr	rs1	010	rd	1110011	CSRRS
csr	rs1	011	rd	1110011	CSRRC
csr	uimm	101	rd	1110011	CSRRWI
csr	uimm	110	rd	1110011	CSRRSI
csr	uimm	111	rd	1110011	CSRRCI

Trap-Return Instructions

0011000	00010	00000	000	00000	1110011	MRET
---------	-------	-------	-----	-------	---------	------

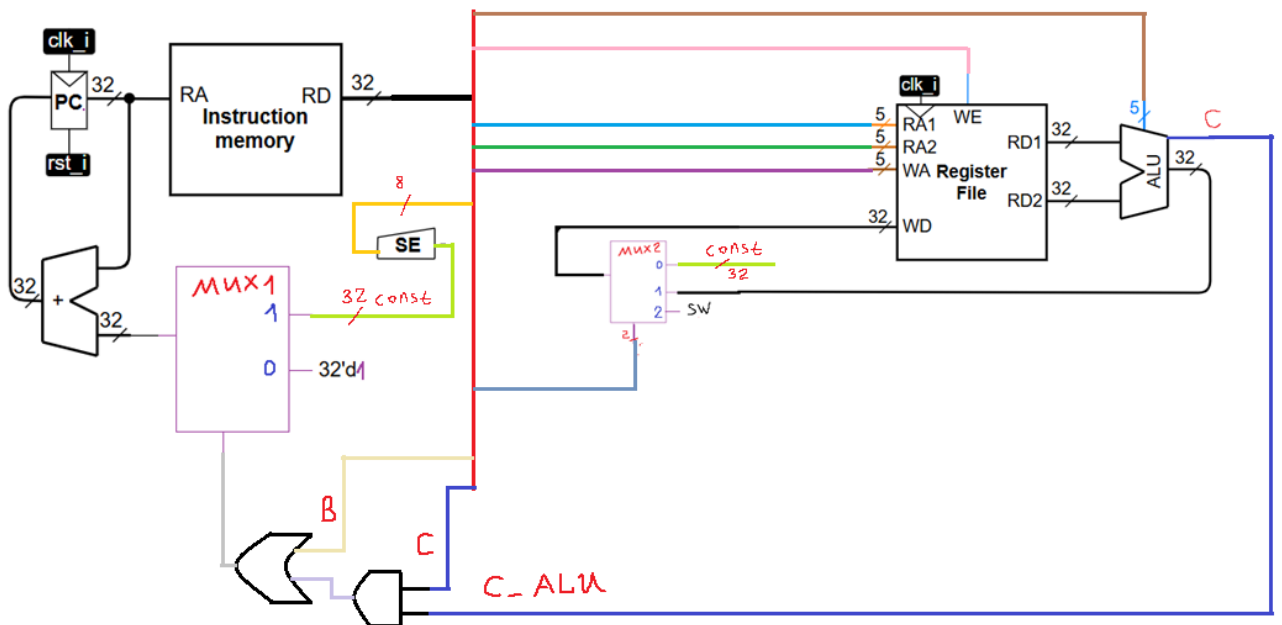
табл. 1. команды RISC-V [rvdoc]

Первая табличка – деление инструкций на типы по опкоду. Видно, что неиспользуемые поля, вроде func7 и rs2 (то же, что RA2), заменены более нужной информацией – битами константы imm. Далее перечисляются базовые инструкции из набора RV32I и дополнения Zicsr, их назначение можно рассмотреть позже. Имеет смысл сразу добавить табличку, в которой описано назначение всех этих инструкций.

Instr	Название	Функция	Описание	Формат	opcode	func3	func7	Пример использования
add	ADDITION	Сложение	$rd = rs1 + rs2$	R	0110011	0x0	0x00	<code>op rd, rs1, rs2</code>
sub	SUBtraction	Вычитание	$rd = rs1 - rs2$			0x0	0x20	
xor	eXclusive OR	Побитовое Исключающее ИЛИ	$rd = rs1 \wedge rs2$			0x4	0x00	
or	OR	Побитовое ИЛИ	$rd = rs1 \vee rs2$			0x6	0x00	
and	AND	Побитовое И	$rd = rs1 \& rs2$			0x7	0x00	
sll	Shift Left Logical	Логический сдвиг влево	$rd = rs1 \ll rs2$			0x1	0x00	
srl	Shift Right Logical	Логический сдвиг вправо	$rd = rs1 \gg rs2$			0x5	0x00	
sra	Shift Right Arithmetic	Арифметический сдвиг вправо	$rd = rs1 \ggg rs2$			0x5	0x20	
slt	Set Less Than	Результат сравнения $A < B$	$rd = (rs1 < rs2) ? 1 : 0$			0x2	0x00	
sltu	Set Less Than Unsigned	Беззнаковое сравнение $A < B$	$rd = (rs1 < rs2) ? 1 : 0$			0x3	0x00	
addi	ADDITION Immediate	Сложение с константой	$rd = rs1 + imm$	I	0010011	0x0	0x00	<code>op rd, rs1, imm</code>
xori	eXclusive OR Immediate	Побитовое Исключающее ИЛИ с константой	$rd = rs1 \wedge imm$			0x4	0x00	
ori	OR Immediate	Побитовое ИЛИ с константой	$rd = rs1 \vee imm$			0x6	0x00	
andi	AND Immediate	Побитовое И с константой	$rd = rs1 \& imm$			0x7	0x00	
slli	Shift Left Logical Immediate	Логический сдвиг влево	$rd = rs1 \ll imm$	I*	0010011	0x1	0x00	<code>addi x6, x3, -12</code>
srl	Shift Right Logical Immediate	Логический сдвиг вправо	$rd = rs1 \gg imm$			0x5	0x00	
srai	Shift Right Arithmetic Immediate	Арифметический сдвиг вправо	$rd = rs1 \ggg imm$			0x5	0x20	
slti	Set Less Than Immediate	Результат сравнения $A < B$	$rd = (rs1 < imm) ? 1 : 0$	I	0010011	0x2	0x00	<code>ori x3, x1, 0x8F</code>
sltiu	Set Less Than Immediate Unsigned	Беззнаковое сравнение $A < B$	$rd = (rs1 < imm) ? 1 : 0$			0x3	0x00	
lb	Load Byte	Загрузить байт из памяти	$rd = SE(Mem[rs1 + imm][7:0])$	I	0000011	0x0	0x00	<code>op rd, imm(rs1)</code>
lh	Load Half	Загрузить полуслово из памяти	$rd = SE(Mem[rs1 + imm][15:0])$			0x1	0x00	
lw	Load Word	Загрузить слово из памяти	$rd = SE(Mem[rs1 + imm][31:0])$			0x2	0x00	
lbu	Load Byte Unsigned	Загрузить беззнаковый байт из памяти	$rd = Mem[rs1 + imm][7:0]$			0x4	0x00	
lhu	Load Half Unsigned	Загрузить беззнаковое полуслово из памяти	$rd = Mem[rs1 + imm][15:0]$	S	0100011	0x0	0x00	<code>op rs2, imm(rs1)</code>
sb	Store Byte	Сохранить байт в память	$Mem[rs1 + imm][7:0] = rs2[7:0]$			0x1	0x00	
sh	Store Half	Сохранить полуслово в память	$Mem[rs1 + imm][15:0] = rs2[15:0]$			0x2	0x00	
sw	Store Word	Сохранить слово в память	$Mem[rs1 + imm][31:0] = rs2[31:0]$			0x2	0x00	
beq	Branch if Equal	Перейти, если $A == B$	$if (rs1 == rs2) PC += imm$	B	1100011	0x0	0x00	<code>comp rs1, rs2, imm</code>
bne	Branch if Not Equal	Перейти, если $A \neq B$	$if (rs1 \neq rs2) PC += imm$			0x1	0x00	
blt	Branch if Less Than	Перейти, если $A < B$	$if (rs1 < rs2) PC += imm$			0x4	0x00	
bge	Branch if Greater or Equal	Перейти, если $A \geq B$	$if (rs1 \geq rs2) PC += imm$			0x5	0x00	
bltu	Branch if Less Than Unsigned	Перейти, если $A < B$ беззнаковое	$if (rs1 < rs2) PC += imm$			0x6	0x00	
bgeu	Branch if Greater or Equal Unsigned	Перейти, если $A \geq B$ беззнаковое	$if (rs1 \geq rs2) PC += imm$			0x7	0x00	
jal	Jump And Link	Переход с сохранением адреса возврата	$rd = PC + 4; PC += imm$	J	1101111	-	-	<code>jal x1, offset</code>
jalr	Jump And Link Register	Переход по регистру с сохранением адреса возврата	$rd = PC + 4; PC = rs1 + imm$	I	1100111	0x0	-	<code>jalr x1, 0(x5)</code>
lui	Load Upper Immediate	Загрузить константу в сдвинутую на 12	$rd = imm \ll 12$	U	0110111	-	-	<code>lui x3, 0xFFFFF</code>
auipc	Add Upper Immediate to PC	Сохранить счетчик команд в сумме с константой $\ll 12$	$rd = PC + (imm \ll 12)$	I	0010111	-	-	<code>auipc x2, 0x000FF</code>
fence	FENCE	Синхронизация инструкций с потоками данных	Воспринимать как <code>nop</code>	I	0001111	0x0	-	<code>fence</code>
ecall	Environment CALL	Передача управления операционной системе	<code>illegal_instr = 1</code>	I**	1110011	0x0	-	<code>ecall</code>
ebreak	Environment BREAK	Передача управления отладчику	<code>illegal_instr = 1</code>	I**	1110011	0x0	-	<code>ebreak</code>
mret	Machine mode RETURN	Возврат управления от обработчика прерывания/исключения	$PC = mepc$	I**	1110011	0x0	-	<code>mret</code>
csrrw	Control Status Register Read Write	Записать в регистровый файл значение CS-регистра. Обновить значение CS-регистра в соответствии с вызываемой инструкцией (см. описание).	$csr = rs1; rd = csr$	I	1110011	0x1	0x00	<code>csrrw rd, csr, rs1</code>
csrrs	Control Status Register Read Set		$csr \vee= rs1; rd = csr$			0x2	0x00	
csrrci	Control Status Register Read Clear		$csr \&= \sim rs1; rd = csr$			0x3	0x00	
csrrwi	Control Status Register Read Write Immediate		$csr = imm; rd = csr$			0x5	0x00	
csrrci	Control Status Register Read Clear Immediate		$csr \&= \sim imm; rd = csr$			0x6	0x00	

табл. 2. описание инструкций и их запись в ассемблере [гитхабМПСУ]

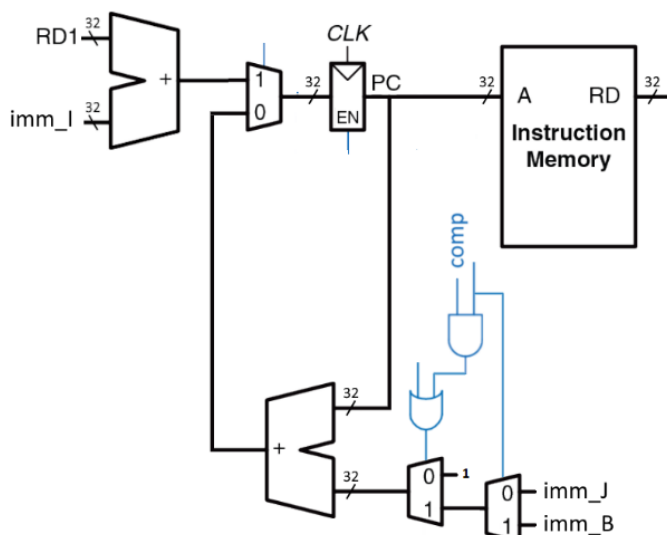
Часть 2. Получение RISC-V из Cobra (+GaSoline)



Выше нарисована схема Cobra, которую мы собрали в прошлых статьях. По сути, если в АЛУ есть все операции из таблички (и они закодированы так как надо), то RISV-V можно построить, просто приделав декодер инструкций и мультиплексы на каждый из выходов регистрового файла и чуток модифицировав PC, чем мы и займемся в данном разделе.

Из-за того, что у нас константа закодирована по-разному в разных типах команд (это неудобство, которым мы платим за большую эффективность процессора), то у нас на PC необходимо поставить возможность перехода по константам разных типов (в зависимости от типа инструкции), то есть, появится несколько лишних мультиплексоров. Задачей определения типов команд и расстановкой всех управляющих сигналов у нас займется декодер, так что будем забивать голову этим уже в следующей статье.

Структурная схема счетчика с поддержкой jalr

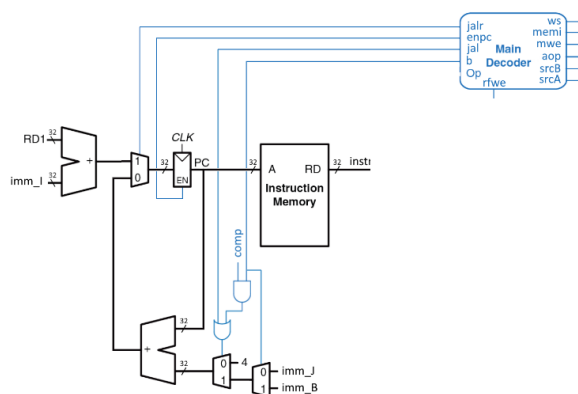


Побайтовая адресация в памяти команд

Также скажем пару слов про побайтовую адресацию в памяти команд. На самом деле, конкретно для памяти команд в вакууме это не имеет какого-то особого смысла именно концептуально, побайтовая адресация важна для памяти данных для работы с отдельными байтами, а на память команд ее наклеили просто ради единообразия и стыковки с внешними элементами (тем же программатором) и для поддержки сжатых инструкций (например, 16-битных), где не надо заново переписывать память команд, а просто прибавить не 4 (ниже поясню, почему 4), а 2 (для 16-битных). Если же делать процессор for fun, то заморачиваться с этим пунктом особо не стоит (но все же желательно, если не хотите писать прошивку вручную, а использовать компиляторы, которые уже есть) и можно просто прибавлять 1 (описание памяти команд на Verilog будет другим и 4 на 1 в остальных тух поменять придется)

Откуда же я взял волшебные числа 4 и 2? Все просто: 32-битная команда – это 4 байта, а 16-битная команда – это 2 байта, поэтому для перехода между командами 32-битными надо перескакивать через 4 байта, а 16-битными – через 2 байта.

Итоговая реализация счетчика (1 заменили на 4).



Память команд с побитовой адресацией:

```

1 module command_memory (
2
3     input [31:0] upr,
4     output [31:0] out
5
6
7 );
8
9 reg [31:0] ROM [0:31];
10
11 initial $readmemb ("Memory.txt",ROM);
12 assign out = ROM[upr[31:2]];
13
14
15 endmodule

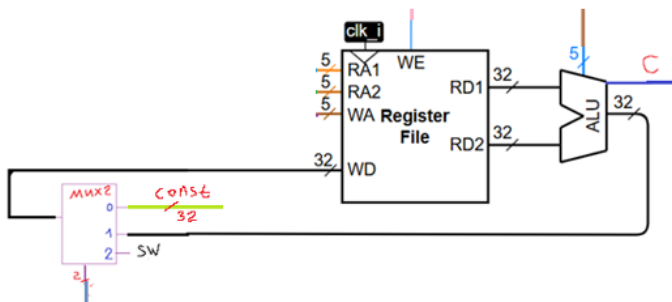
```

Модификация системы регистровый файл + АЛУ + память

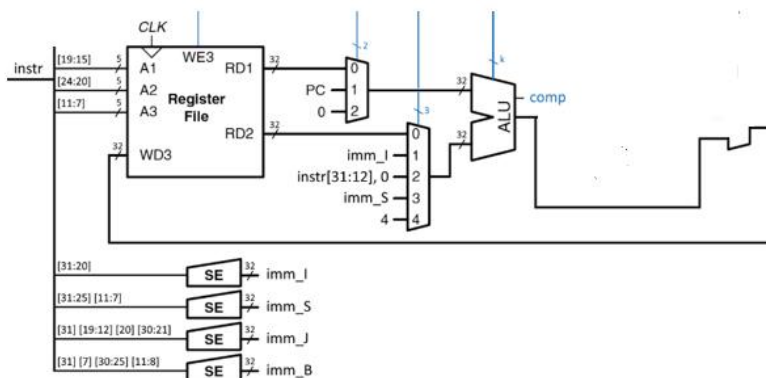
Шаг 1. Переставим мультиплексоры

В Cobra система АЛУ/Регистровый файл была реализована просто и прямолинейно: выходы регистрового файла подаются сразу на АЛУ, а вход мультиплексируется, у нас же будет чутко по-другому: входы АЛУ мультиплексируются, а выход АЛУ идет на регистровый файл (по сути своей это примерно одно и тоже)

Было:



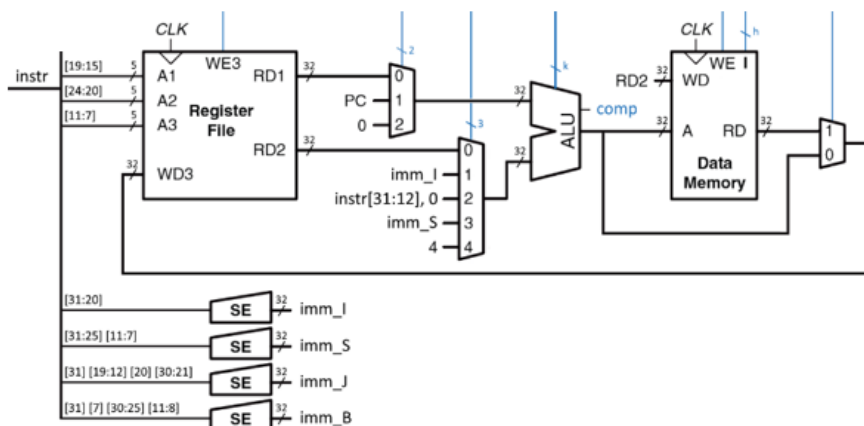
Стало (просто переставили мультиплексоры и чуток расширили входы):



Из относительно нового у нас несколько разных констант, также есть возможность на АЛУ работать с PC (это надо для работы с всякими jal, jalr и подобными командами, где нужно сохранить куда-то значение PC или же PC +4 (адрес возврата на строчку памяти команд)).

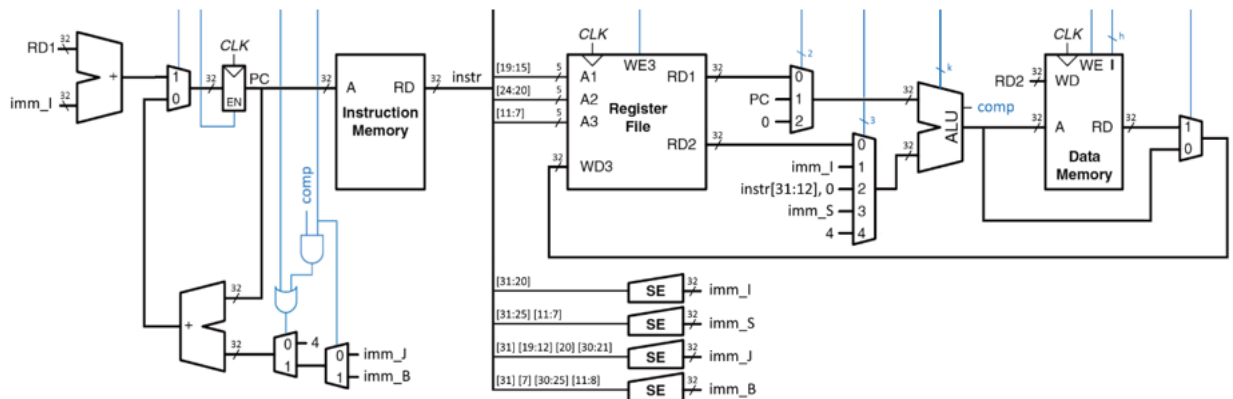
Шаг 2. Добавляем Data Memory

Если внимательно посмотреть на рисунок выше и подумать, то относительно очевидным будет решение просто поставить data memory между регистровым файлом и АЛУ и просто мультиплексировать выход (если надо, то данные идут на data memory, иначе данные идут в регистровый файл). Очевидным же решение я назвал потому, что у нас регистровый файл и память команд играют примерно одинаковую роль.

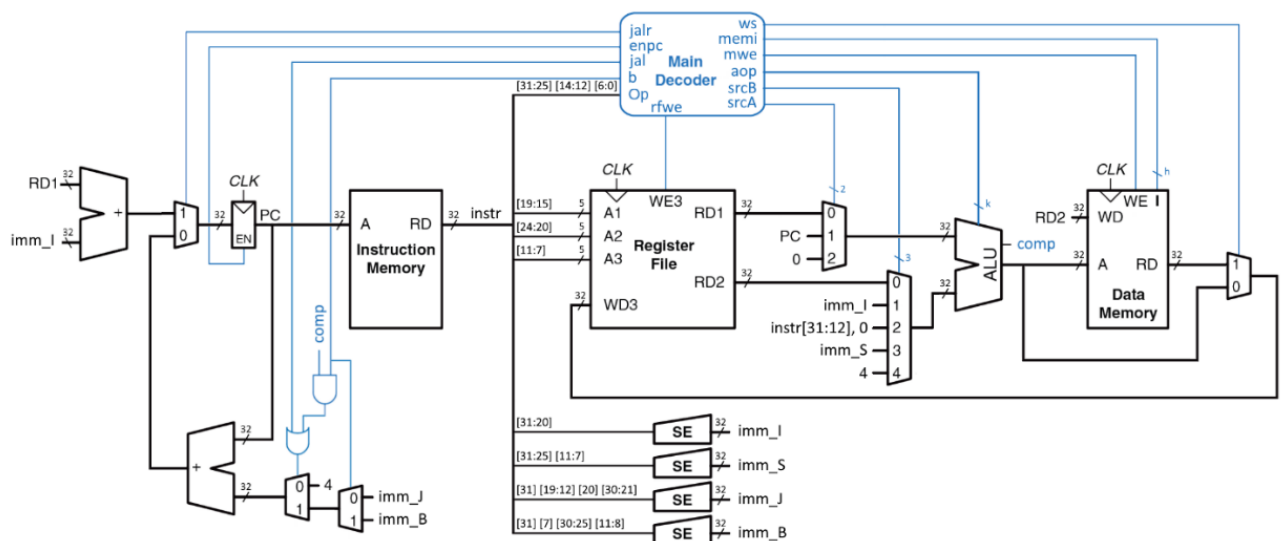


Шаг 3. Добавляем управлитор к мультиплексорам.

Выше мы уже собрали вот такой вот процессор, в котором невооружённым взглядом видны торчащие провода управления от мультиплексоров и всякие enable:



Теперь нужно реализовать управлитор (декодер инструкций), который по opcode, funct3, funct7 будет выдавать необходимые значения на управляющие входы АЛУ, Data Memory, WE и тд. В данной статье будем считать его волшебной коробочкой, которую подробно (и долго) будем собирать в следующей лекции (то есть, воспользуемся проектирование сверху вниз, когда мы сначала говорим, то волшебная штука у нас есть, а потом долго и мучительно ее реализуем).



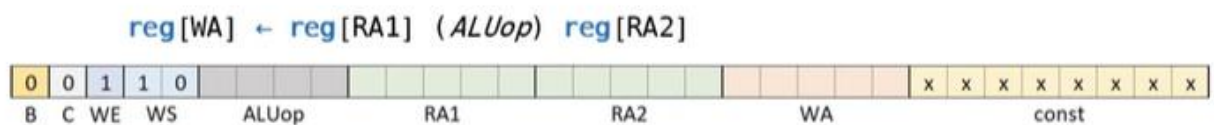
Ну что, все, RISC-V готов, осталось собрать декодер (следующая статья), соединить все блоки между собой (8 лекция, где будет преимущественно описание на Verilog процессора) и можно тестировать будет (код на C перегоняем в ассемблер RISC-V, а его в машинный код).

Лекция 7. Декодер инструкций RISC-V.

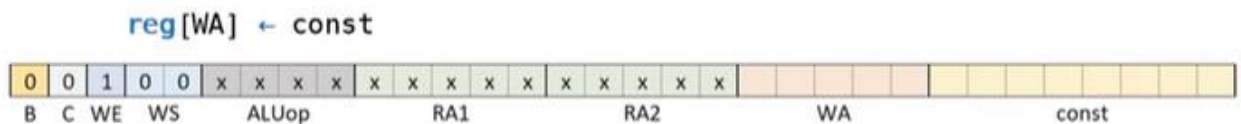
Большое вступление (на всю страничку).

Как читатель мог заметить на примере упрощенного процессора Cobra, у нас в инструкции всегда очень много “лишних” проводов, и с их более разумным использованием можно было бы расширить возможности самого процессора.

Пример операции на АЛУ Cobra, когда 8 бит никак полезно не используются



Загрузка константы. Не используется 14 бит.



То-есть, подводя итог примерам выше, у нас есть неиспользуемые биты в каждом конкретном типе операций, так что весьма разумно сделать так, чтобы инструкции обладали большим полезным содержанием. Авторы RISC-V, скорее всего подумали также и несколько модифицировали команды, сделав не как в Cobra их прямое исполнение (то есть, конкретный бит команды идет напрямую на АЛУ или управляющий мультиплексор регистрового файла), а через посредника, биты команды же играют роль управляющих сигналов мультиплексора, которые с помощью декодера инструкций (нашего посредника) уже “собираются” в новую команду, которая поступит на устройства.

Таким образом, возрастает эффективность команд устройства (например, мы теперь имеем возможность расширить список операций на АЛУ из-за такого варианта кодирования)

Про кодировку команд, схожесть и отличия с Cobra.

Главным отличием команд RISC-V от своей упрощенной версии (Cobra) являются поля funct3, funct7, opcode и непостоянство принадлежности битов к своему полю (для некоторых типов команд биты с 7 по 11 играют роль rd (upr_in в кодировке Cobra для регистрового файла), а для других являются частью imm (константы). Кстати, в J-типе константа выглядит как монстр Франкенштейна из-за того, что процессор должен поддерживать 8 битные инструкции (поэтому пришлось так нашинковать изначальную команду и собрать из нее константу).

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Разберем теперь каждое поле по отдельности

Funct3, funct7.

Называются так они из-за их размеров (funct3 – 3 бита, funct7 – 7 бит). Используются данные поля для выбора команды на АЛУ (практически всегда только funct3, funct7 уже выступает в роли дополнительных битов).

Opcode – распределитель ролей.

Данная штука кодирует тип инструкции (а тип инструкции отвечает за то, как именно декодер распределит управляющие биты на все блоки процессора по их видам и за наличие, собственно, самих типов команд).

Type R (Register/Register), opcode = 7'd51.

В данном типе операций данные берутся с регистров, производится операция на АЛУ и результат записывается в регистр. Использует регистры для управления регистровым файлом: RS1 (у меня это upr_1), RS2 (у меня это upr_2), rd (у меня это upr_in). Пример команд: ADD (сложение), SUB (вычитание), OR (логическое ИЛИ).

Type I (Immediate), opcode = 7'd19, 7'd3, 7'd103

Данный тип инструкций можно назвать регистр-константа (есть регистр, с которого берем данные, есть регистр, в который данные записываем, есть константа, с которой выполняем операции на АЛУ). Пример команд: ADDI (сложение регистра с константой), загрузка данных из памяти, условные переходы. Из нового относительно прошлого типа – константа Imm.

На самом деле, тип Immediate имеет целых 3 opcode, так-как это объединенный тип операции (когда есть 3 типа операций, но выполняют они все примерно одинаковые вещи). Ниже как раз-таки представлено это разделение.

Opcode (7 бит)	Группа инструкций	Примеры
0010011 (0x13)	Арифметика с константой	ADDI , ORI , ANDI , SLLI
0000011 (0x03)	Загрузка из памяти (Load)	LW , LH , LB , LBU , LHU
1100111 (0x67)	Условный переход (JALR)	JALR

Type S (Store), opcode = 7'd35

Данный тип инструкции – запись в память (именно в data memory, а не в регистровый файл). Данный тип инструкций не возвращает результат в регистр (это отличие его от Immediate). Пример команд: SW (запись 32 бит (слова)), SH (запись 16 бит (полуслова)), SB (запись 8 бит (байта))

Type B (Branch), opcode = 7'd99

Данный тип инструкций реализует условные переходы (if/else), но в отличие от типа Immediate никуда не записывает данные. По разбивке команды весьма сильно похож на Store, хотя и имеет разный с ним функционал. Примеры команд: BEQ (перейти, если равно), BNE (перейти, если не равно), но imm интерпретируется как шаг перехода (сколько прибавить к счетчику, если команда верна).

Type U (Upper Immediate), opcode = 7'd23, 7'd55

В данном типе инструкций 2 opcode по очень простой причине: в команде нет поля funct3, а значит роль мультиплексора выполняет только opcode

Данный тип инструкций работает с старшими битами константы (с 31 по 12), которые мы используем как хотим (используется для формирования 32-битных адресов (LUI) и для прибавления к PC константы, в которой мы задаем только старшие биты (а с 11 по 0 бит записываются 0)).

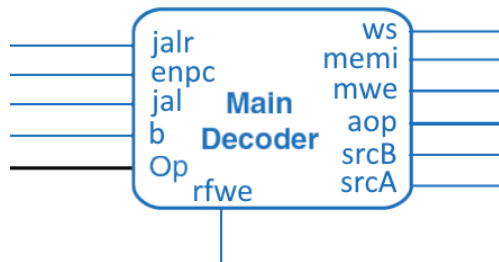
Type J (Jump), opcode = 7'd111

Данный тип инструкций отвечает за переход с сохранением адреса возврата, используется при вызове функций и возврате из них. Константа определяет относительный адрес перехода (сколько надо прибавить к PC, чтобы перейти на нужную нам строчку кода).

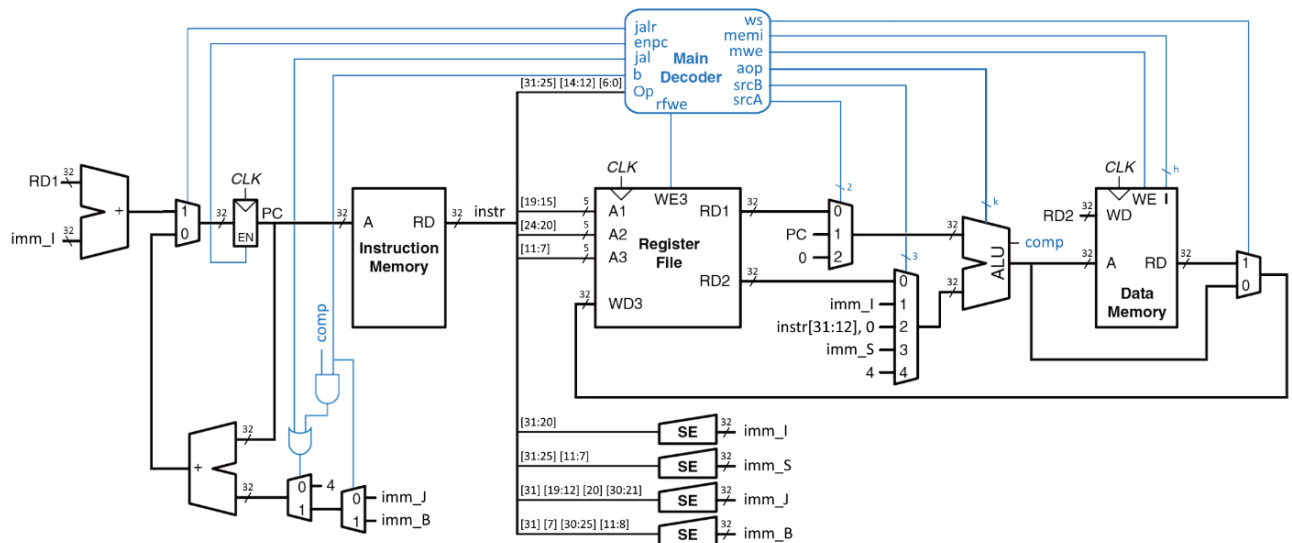
Декодер инструкций как функциональный блок.

Итак, мы более-менее разобрались с кодировкой инструкции по типам, присвоили им opcode, но ничего не сказали про декодер инструкций (точнее, что в него выходит, а что выходит). Хотелось бы сказать, что в данное устройство входит вся команда, но оказывается можно обойтись лишь “дополнительными битами” по отношению к Cobra (opcode, funct3, funct7), а с остальным разберутся мультиплексоры и выходы декодера.

Декодер инструкций. Черный провод – вход (funct3, funct7, opcode), синие провода – выходы декодера



Декодер инструкций в RISC-V (однотактном)



Реализация декодера инструкций.

На самом деле, декодер сложен скорее не концептуально, а объемно (150+ строк кода на Verilog), поэтому в данной статье читатель должен набраться терпения и мужества ради чтения страниц так 10 практически однообразных действий.

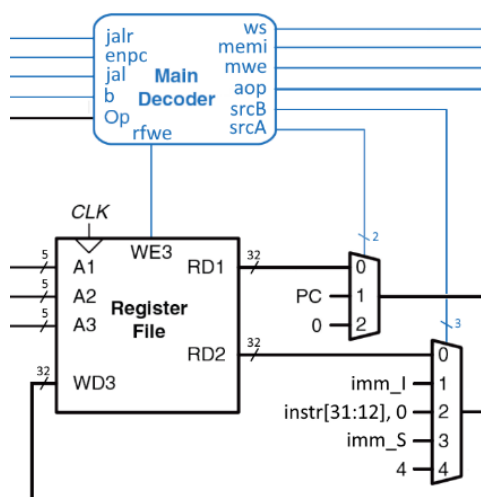
Инструкция типа R (Register/Register)

31	27	26	25	24	20	19	15	14	12	11	7	6	0		
funct7				rs2		rs1		funct3		rd			opcode		R-type

Инструкция	Название	Функция	Формат	Opcode (hex)	func3 (hex)	func7 (hex)	Пример
add	ADD	$rd = rs1 + rs2$	R	0x33	0x0	0x00	<code>add x1, x2, x3</code>
sub	SUB	$rd = rs1 - rs2$	R	0x33	0x0	0x20	<code>sub x4, x5, x6</code>
xor	XOR	$rd = rs1 \wedge rs2$	R	0x33	0x4	0x00	<code>xor x7, x8, x9</code>
or	OR	$rd = rs1 \vee rs2$	R	0x33	0x6	0x00	<code>or x10, x11, x12</code>
and	AND	$rd = rs1 \& rs2$	R	0x33	0x7	0x00	<code>and x13, x14, x15</code>
sll	Shift Left Logical	$rd = rs1 \ll rs2$	R	0x33	0x1	0x00	<code>sll x16, x17, x18</code>
srl	Shift Right Logical	$rd = rs1 \gg rs2$ (логический)	R	0x33	0x5	0x00	<code>srl x19, x20, x21</code>
sra	Shift Right Arithmetic	$rd = rs1 \gg rs2$ (арифметический)	R	0x33	0x5	0x20	<code>sra x22, x23, x24</code>
slt	Set Less Than	$rd = (rs1 < rs2) ? 1 : 0$ (знаковое)	R	0x33	0x2	0x00	<code>slt x25, x26, x27</code>
sltu	Set Less Than Unsigned	$rd = (rs1 < rs2) ? 1 : 0$ (беззнаковое)	R	0x33	0x3	0x00	<code>sltu x28, x29, x30</code>

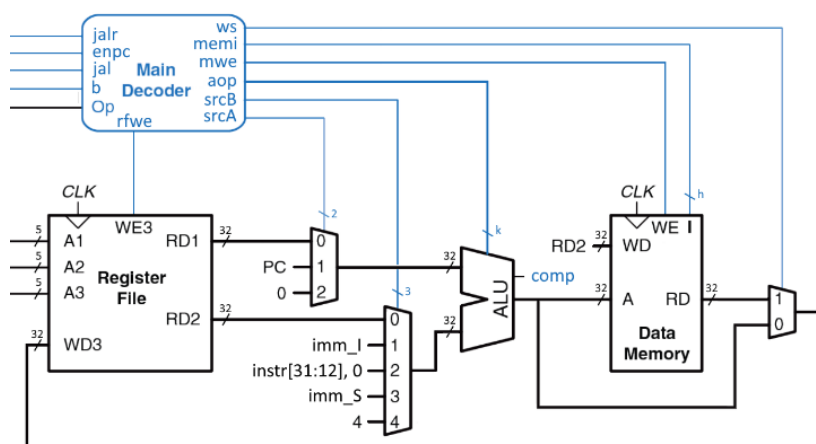
Опишем комбинационную схему, которая при opcode = 51 реализует функционал, для типа R, описанный выше.

Биты srcA и srcB для инструкции типа R



Так-как в данном типе инструкций мы берем данные с регистрового файла и в него же их записываем после результата операции на АЛУ, то **srcA = 2'd0, srcB = 3'd0** (смотри картинку выше).

Биты ws, memi, mwe для инструкции типа R



Так-как данные должны записаться в регистровый файл, то **ws = 0** (без записи в память Data Memory), **mwe = 0** (еще вырубим Data Memory, чтобы данные не записывала). Выход декодера memi можно сделать произвольным (так-как Data Memory не работает, а memi отвечает за тип операции с памятью (запись числа знакового, запись числа беззнакового, чтение и тд, у нас memi будет 5-битным), поэтому выставим **memi = 5'd0**

Таблица про то, что делает memi в Data Memory

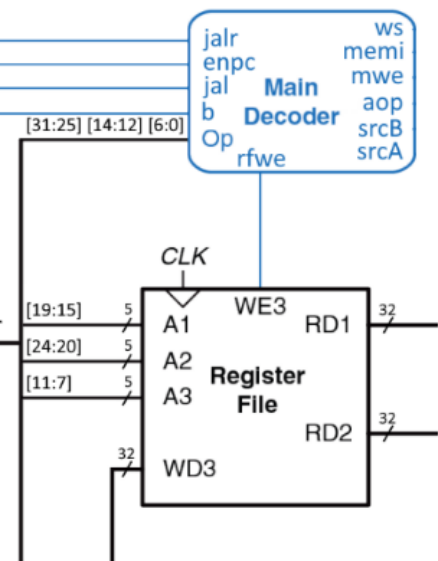
Биты	Название	Значения	Описание
0	mem_read	0 – нет чтения, 1 – чтение	Активирует чтение из памяти (для LW, LB, LH и т. д.).
1	mem_write	0 – нет записи, 1 – запись	Активирует запись в память (для SW, SB, SH).
2–3	mem_size	00 – byte (8 бит), 01 – half (16), 10 – word (32)	Определяет размер данных для загрузки/сохранения.
4 (опц.)	mem_unsigned	0 – знаковое, 1 – беззнаковое	Для загрузки (LBU, LHU – беззнаковое расширение).

Бит aop (операция на АЛУ) для инструкции типа R.

Из-за нашей специальной кодировки АЛУ (смотри 3 АЛУ из статьи про АЛУ) мы получаем весьма простое выражение для его управляющих битов:

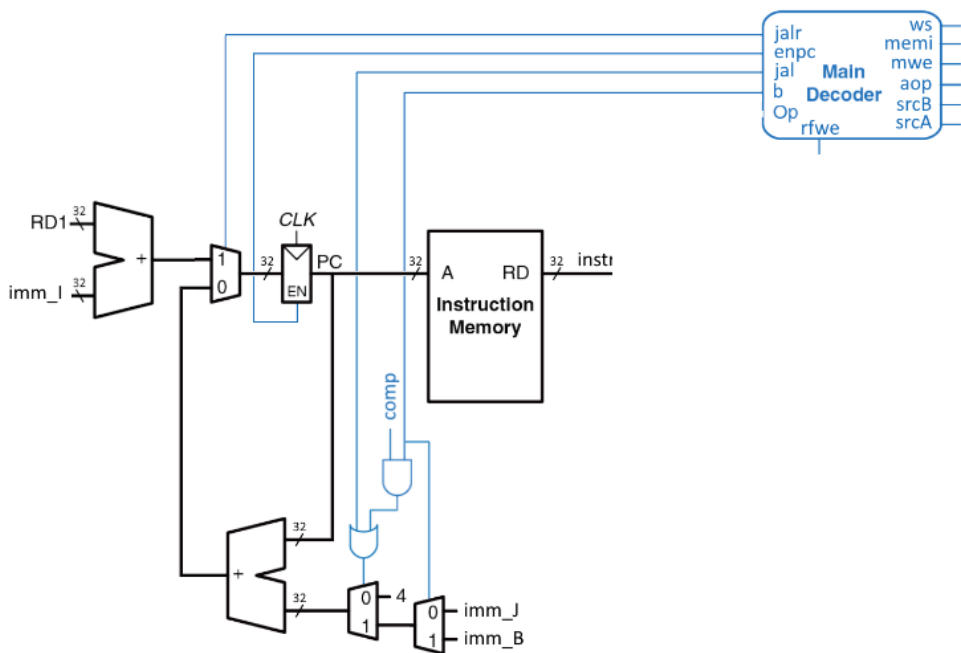
$$aop = \{funct7[6:5], funct3\} \text{ – шина из 2 последних проводов } funct7 \text{ и всего } funct3$$

Бит rfwe



В операции типа R мы будем записывать данные в регистровый файл, поэтому ставим **rfwe = 1'd1**.

Биты enpc, jal, jalr, b.



Далее весь анализ делается на основе картинки выше.

Бит jal отвечает за безусловный перенос, в операции типа R данного переноса нет, поэтому **jal = 1'd0**

Бит enpc отвечает за работу счетчика (счетчик работать должен как обычно), поэтому **enpc = 1'd1**

Бит jalr отвечает за работу счетчика в обычном режиме, или же резкий переход на какой-то адрес, в инструкции типа R, как нетрудно догадаться, **jalr = 1'd0**

Бит b отвечает за условные переходы, в инструкции типа R такого не происходит, поэтому **b = 1'd0**

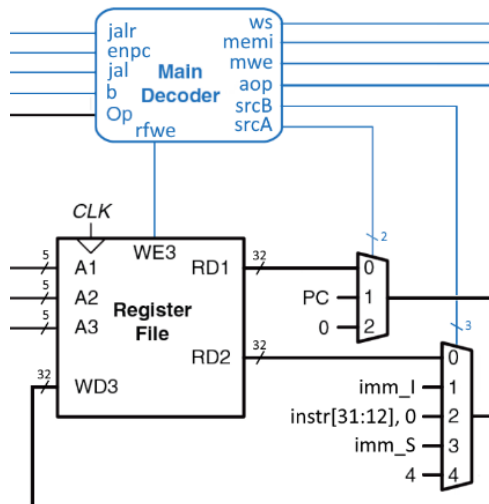
Инструкции типа I (Immediate) (opcode = 7'd19).

imm[11:0]	rs1	funct3	rd	opcode	I-type
-----------	-----	--------	----	--------	--------

Опишем комбинационную схему, которая при opcode = 19 реализует функционал, для типа I, описанный выше.

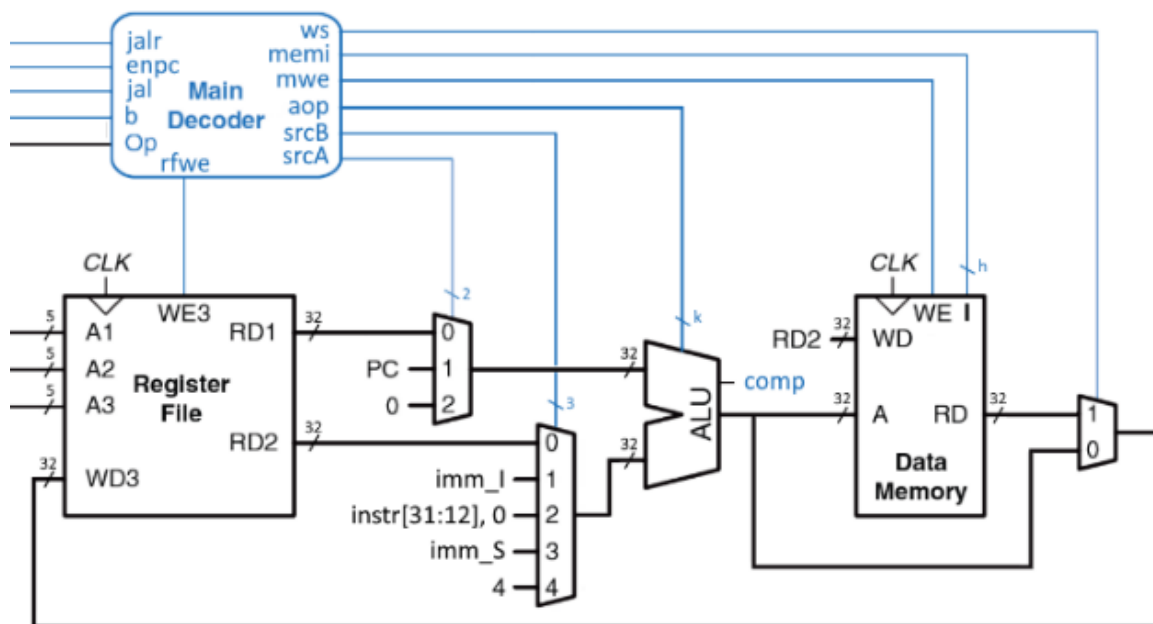
addi	ADDITION Immediate	Сложение с константой	rd = rs1 + imm	I	0010011	0x0
xori	eXclusive OR Immediate	Побитовое Исключающее ИЛИ с константой	rd = rs1 ^ imm			0x4
ori	OR Immediate	Побитовое ИЛИ с константой	rd = rs1 imm			0x6
andi	AND Immediate	Побитовое И с константой	rd = rs1 & imm			0x7
slti	Set Less Than Immediate	Результат сравнения A < B	rd = (rs1 < imm) ? 1 : 0	I		0x2
sltiu	Set Less Than Immediate Unsigned	Беззнаковое сравнение A < B	rd = (rs1 < imm) ? 1 : 0			0x3

Биты srcA и srcB для инструкции типа I



Так-как в данном типе инструкций мы берем данные с регистрового файла, с константы и в регистровый файл их записываем после результата операции на АЛУ, то $srcA = 2'd0$, $srcB = 3'd1$ (смотри картинку выше).

Биты ws, memi, mwe для инструкции типа I (opcode – 19)



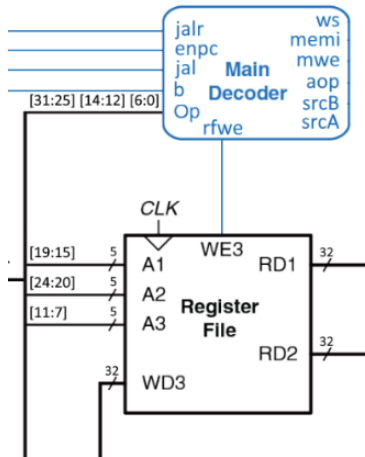
Так-как данные должны записаться в регистровый файл, то $ws = 0$, $mwe = 0$, $memi = 5'd0$

Бит aop (операция на АЛУ) для инструкции типа R.

Из-за нашей специальной кодировки АЛУ мы получаем:

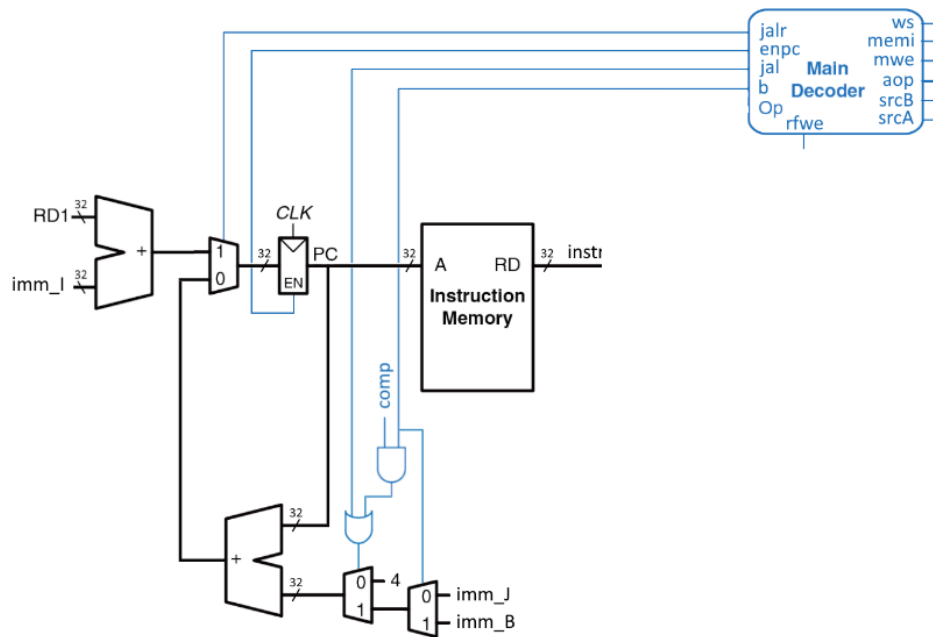
$aop = \{2'd0, funct3\}$

Бит rfwe



В операции типа I мы будем записывать данные в регистровый файл, поэтому ставим **rfwe = 1'd1**.

Биты enpc, jal, jalr, b.



Далее весь анализ делается на основе картинки выше.

Бит jal отвечает за безусловный перенос, в операции типа I данного переноса нет, поэтому **jal = 1'd0**

Бит enpc отвечает за работу счетчика (счетчик работать должен как обычно), поэтому **enpc = 1'd1**

Бит jalr отвечает за работу счетчика в обычном режиме, или же резкий переход на какой-то адрес, в инструкции типа I, как нетрудно догадаться, **jalr = 1'd0**

Бит b отвечает за условные переходы, в инструкции типа I такого не происходит, поэтому **b = 1'd0**

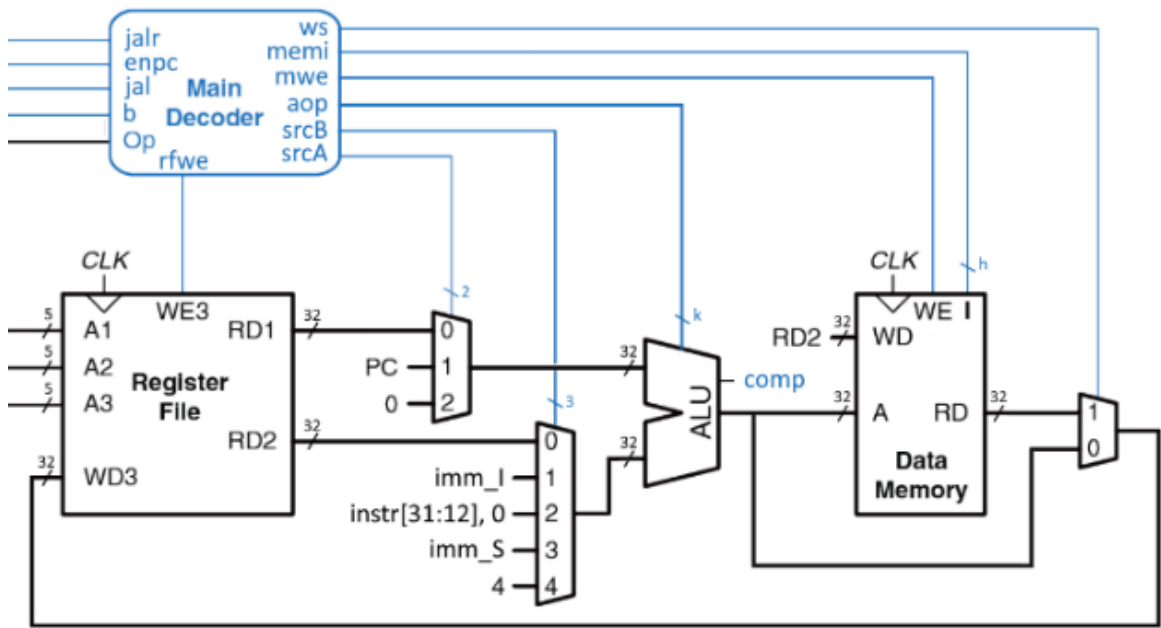
Инструкции типа I (Immediate) (opcode =7'd3).

Отличием данного типа инструкций от I с opcode = 19 является работа с памятью

lb	Load Byte	Загрузить байт из памяти	$rd = SE(Mem[rs1 + imm][7:0])$	I	0000011	0x0	-
lh	Load Half	Загрузить полуслово из памяти	$rd = SE(Mem[rs1 + imm][15:0])$			0x1	
lw	Load Word	Загрузить слово из памяти	$rd = SE(Mem[rs1 + imm][31:0])$			0x2	
lbu	Load Byte Unsigned	Загрузить беззнаковый байт из памяти	$rd = Mem[rs1 + imm][7:0]$			0x4	
lhu	Load Half Unsigned	Загрузить беззнаковое полуслово из памяти	$rd = Mem[rs1 + imm][15:0]$			0x5	

То-есть, все выходы декодера будут такими же, как и у opcode = 19 кроме битов, отвечающих за работу с памятью.

Биты ws, memi, mwe для инструкции типа I (opcode =3)



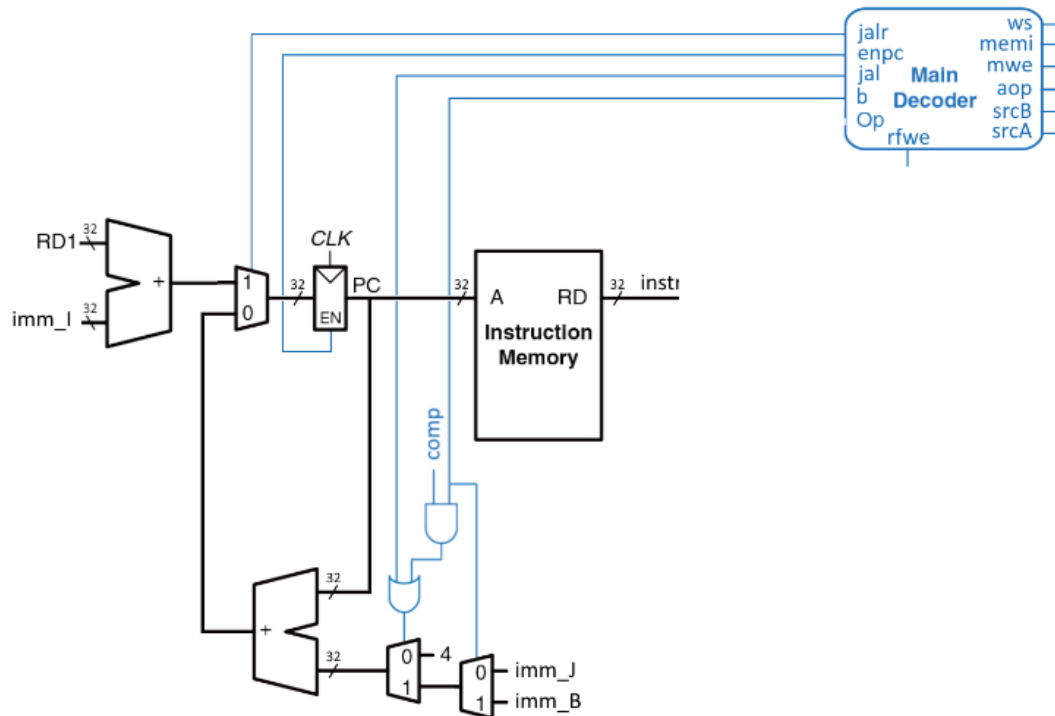
Биты	Название	Значения	Описание
0	mem_read	0 – нет чтения, 1 – чтение	Активирует чтение из памяти (для LW, LB, LH и т. д.).
1	mem_write	0 – нет записи, 1 – запись	Активирует запись в память (для SW, SB, SH).
2–3	mem_size	00 – byte (8 бит), 01 – half (16), 10 – word (32)	Определяет размер данных для загрузки/сохранения.
4 (опц.)	mem_unsigned	0 – знаковое, 1 – беззнаковое	Для загрузки (LBU, LHU – беззнаковое расширение).

Так-как данные должны считаться из памяти и записаться в регистровый файл, то из картинки выше следует, что надо поставить **ws = 1, mwe = 1, Memi[0] = 1, Memi[1] = 0, Memi[4:2] = funct3.**

Инструкции типа I (Immediate) (opcode = 7'd103).

jalr	Jump And Link Register	Переход по регистру с сохранением адреса возврата	$rd = PC + 4; PC = rs1 + imm$	I	1100111	0x0	-	jalr x1, 0(x5)
------	------------------------	---	-------------------------------	---	---------	-----	---	----------------

Биты aop, ws, memi, mwe, rwe, srcA, srcB для инструкции типа I (opcode – 103)



Mwe = 1'd0, Memi = 5'd0, ws = 1'd0, rfwe = 1'd1, (работаем с регистровым файлом, а не с памятью)

srcA = 2'd1, srcB = 3'd4, aop = 5'd0 (берем значение с PC и 4, делаем сложение (0 операция на АЛУ) и записываем в RF)

Jalr = 1'd1 (так-как надо перейти на значение в регистре 1 + константа)

Остальные биты такие же, как и при других opcode инструкции типа I

Описание на Verilog декодера, реализующего декодирование инструкций типа R, I

```

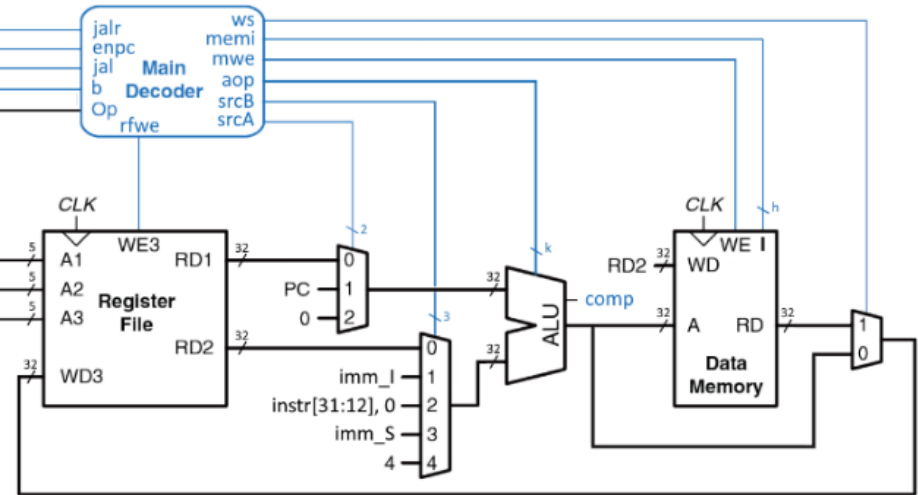
1  module Decoder_R (
2      input [6:0] opcode,
3      input [2:0] func3,
4      input [6:0] func7,
5      output jalr,
6      output enpc,
7      output jal,
8      output b,
9      output ws,
10     output [4:0] memi,
11     output mwe,
12     output rfwe,
13     output [4:0] aop,
14     output [2:0] srcB,
15     output [1:0] srcA
16 );
17
18 parameter opcode_R = 7'd51;
19 parameter opcode_I_1 = 7'd19;
20 parameter opcode_I_2 = 7'd3;
21 parameter opcode_I_3 = 7'd103;
22
23 wire [31:0] ZAT; //Затычка в декодере
24 assign ZAT = 32'd1;
25
26 assign srcA = (opcode == opcode_R) ? 2'd0 :
27               (opcode == opcode_I_1) ? 2'd0 :
28               (opcode == opcode_I_2) ? 2'd0 :
29               (opcode == opcode_I_3) ? 2'd1 : ZAT[1:0];
30
31 assign srcB = (opcode == opcode_R) ? 3'd0 :
32               (opcode == opcode_I_1) ? 3'd1 :
33               (opcode == opcode_I_2) ? 3'd1 :
34               (opcode == opcode_I_3) ? 3'd4 : ZAT[2:0];
35
36 assign memi = (opcode == opcode_R) ? 5'd0 :
37               (opcode == opcode_I_1) ? 5'd0 :
38               (opcode == opcode_I_2) ? {1'b1, 1'b0, func3} :
39               (opcode == opcode_I_3) ? 5'd0 : ZAT[4:0];
40
41
42 assign aop = (opcode == opcode_R) ? {func7[6:5], func3} :
43               (opcode == opcode_I_1) ? {2'd0, func3} :
44               (opcode == opcode_I_2) ? {2'd0, func3} :
45               (opcode == opcode_I_3) ? 5'd0 : ZAT[4:0];
46
47 assign enpc = (opcode == opcode_R) ? 1'd1 :
48               (opcode == opcode_I_1) ? 1'd1 :
49               (opcode == opcode_I_2) ? 1'd1 :
50               (opcode == opcode_I_3) ? 1'd1 : ZAT[1];
51
52 assign ws = (opcode == opcode_R) ? 1'd0 :
53              (opcode == opcode_I_1) ? 1'd0 :
54              (opcode == opcode_I_2) ? 1'd1 :
55              (opcode == opcode_I_3) ? 1'd0 : ZAT[0];
56
57 assign mwe = (opcode == opcode_R) ? 1'd0 :
58              (opcode == opcode_I_1) ? 1'd0 :
59              (opcode == opcode_I_2) ? 1'd1 :
60              (opcode == opcode_I_3) ? 1'd0 : ZAT[0];
61
62 assign rfwe = (opcode == opcode_R) ? 1'd1 :
63               (opcode == opcode_I_1) ? 1'd1 :
64               (opcode == opcode_I_2) ? 1'd1 :
65               (opcode == opcode_I_3) ? 1'd1 : ZAT[0];
66
67 assign jalr = (opcode == opcode_R) ? 1'd0 :
68               (opcode == opcode_I_1) ? 1'd0 :
69               (opcode == opcode_I_2) ? 1'd0 :
70               (opcode == opcode_I_3) ? 1'd1 : ZAT[0];
71
72 assign jal = (opcode == opcode_R) ? 1'd0 :
73              (opcode == opcode_I_1) ? 1'd0 :
74              (opcode == opcode_I_2) ? 1'd0 :
75              (opcode == opcode_I_3) ? 1'd0 : ZAT[0];
76
77 assign b = (opcode == opcode_R) ? 1'd0 :
78             (opcode == opcode_I_1) ? 1'd0 :
79             (opcode == opcode_I_2) ? 1'd0 :
80             (opcode == opcode_I_3) ? 1'd0 : ZAT[0];
81 endmodule

```

Реализация инструкций типа S (opcode = 7'd35).

Что-ж, самый объемный кусок декодера мы написали, осталось только добить мелочь по типу инструкций S, J, U, B. Так-как читателю (надеюсь) уже относительно понятен алгоритм расставления значений на выходах декодера, то обойдусь минимальными пояснениями.

sb	Store Byte	Сохранить байт в память	$Mem[rs1 + imm][7:0] = rs2[7:0]$	S	0100011	0x0
sh	Store Half	Сохранить полуслово в память	$Mem[rs1 + imm][15:0] = rs2[15:0]$			0x1
sw	Store Word	Сохранить слово в память	$Mem[rs1 + imm][31:0] = rs2[31:0]$			0x2



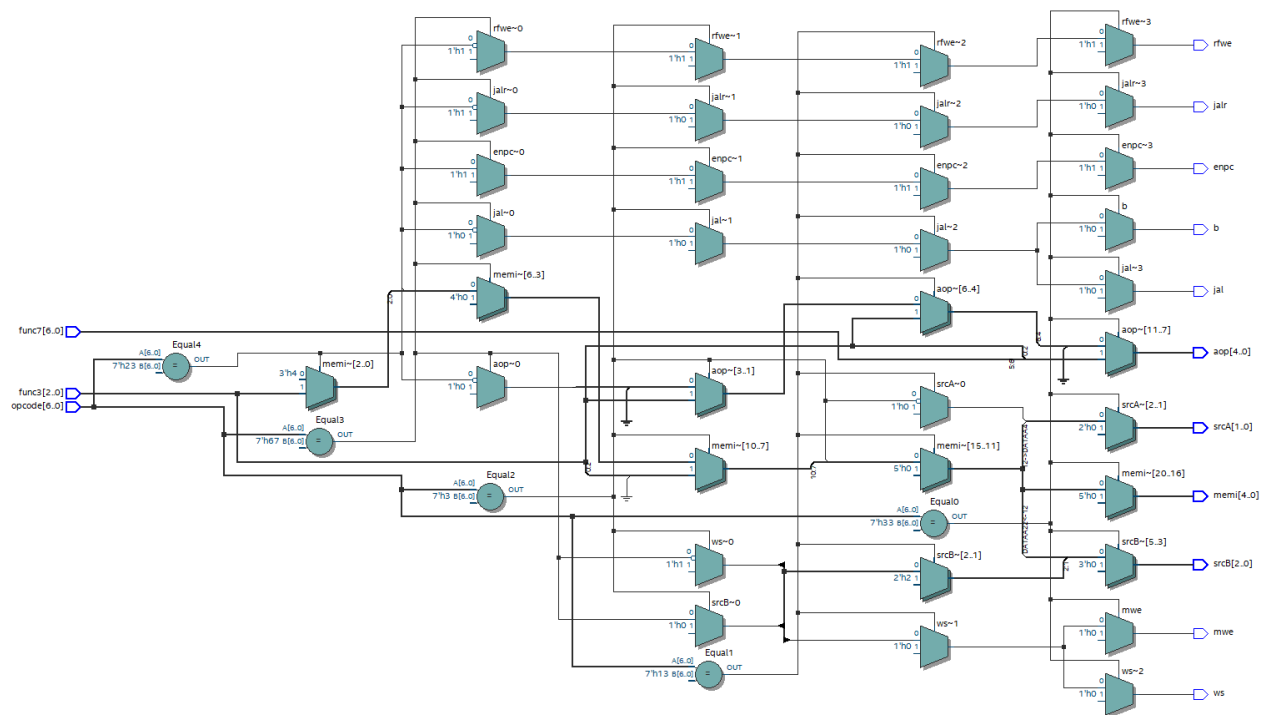
Очередная табличка про кодировку памяти

Биты	Название	Значения	Описание
0	mem_read	0 – нет чтения, 1 – чтение	Активирует чтение из памяти (для LW, LB, LH и т. д.).
1	mem_write	0 – нет записи, 1 – запись	Активирует запись в память (для SW, SB, SH).
2–3	mem_size	00 – byte (8 бит), 01 – half (16), 10 – word (32)	Определяет размер данных для загрузки/сохранения.
4 (опц.)	mem_unsigned	0 – знаковое, 1 – беззнаковое	Для загрузки (LBU, LHU – беззнаковое расширение).

Mwe = 1'd1, Memi = {1'd0, 1'd1, 3'df}, ws = 1'd1, rfwe = 1'd0, srcA = 2'd0, srcB = 3'd3, aop = 5'd0 (берем значение с RD1, складываем с константой и получаем адрес, по которому мы записываем значение с RD2 записываем в память, в RF ничего не записываем)

Jalr = 1'd0, Jal = 1'd0, b = 1'd0, enpc = 1'd1 – ничего экстраординарного с счетчиком не делаем

Схема декодера (R, S, I), как видно читателю, данная схема на самом деле является продвинутым мультиплексором.

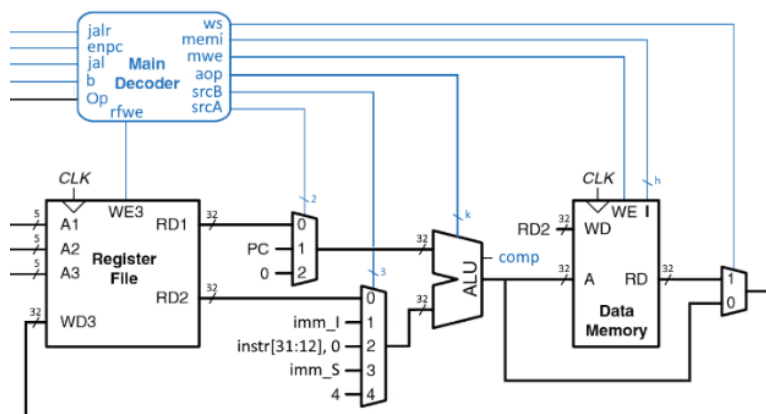


Реализация инструкций типа В (opcode = 7'd99).

beq	Branch if Equal	Перейти, если A == B	if (rs1 == rs2) PC += imm	B	1100011	0x0
bne	Branch if Not Equal	Перейти, если A != B	if (rs1 != rs2) PC += imm			0x1
blt	Branch if Less Than	Перейти, если A < B	if (rs1 < rs2) PC += imm			0x4
bge	Branch if Greater or Equal	Перейти, если A >= B	if (rs1 >= rs2) PC += imm			0x5
bltu	Branch if Less Than Unsigned	Перейти, если A < B беззнаковое	if (rs1 < rs2) PC += imm			0x6
bgeu	Branch if Greater or Equal Unsigned	Перейти, если A >= B беззнаковое	if (rs1 >= rs2) PC += imm			0x7

Про АЛУ (aop) и все “правые” биты декодера.

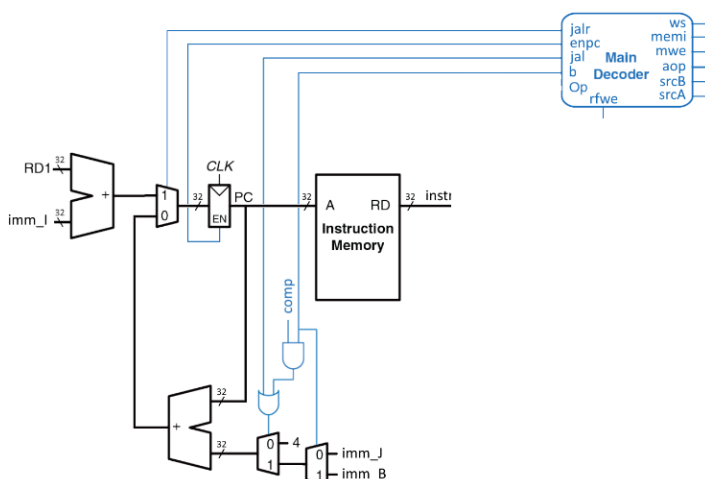
Операция	={cmp, mod, opcode}	Выражение	Действие
EQ	1 1 000	flag_o = (a_i == b_i)	Вывести флаг, если равны
NE	1 1 001	flag_o = (a_i != b_i)	Вывести флаг, если не равны
LTS	1 1 100	flag_o = a_i < b_i	Знаковое сравнение <
GES	1 1 101	flag_o = a_i ≥ b_i	Знаковое сравнение ≥
LTU	1 1 110	flag_o = a_i < b_i	Беззнаковое сравнение <
GEU	1 1 111	flag_o = a_i ≥ b_i	Беззнаковое сравнение ≥



Из-за нашей нумерации команд на АЛУ $aop = \{1, 1, func3\}$ (две единицы обозначают то, что на выходе АЛУ нам нужен только флаг сравнения, на OUT_ALU же будет просто 0). Также, так-как АЛУ проводит операцию с RD1 и RD2 и никуда результат не записывает, то: $rfwe = 1'd0$, $SrcA = 2'd0$, $SrcB = 1'd0$, $mwe = 1'd0$, $memi = 5'd0$, $ws = 1'd0$

Про счетчик.

Данный тип инструкций отличается от остальных тем, что мы работаем только с счетчиком и результатом сравнения на АЛУ, поэтому акцентируем внимание на левой части архитектуры RISC-V.



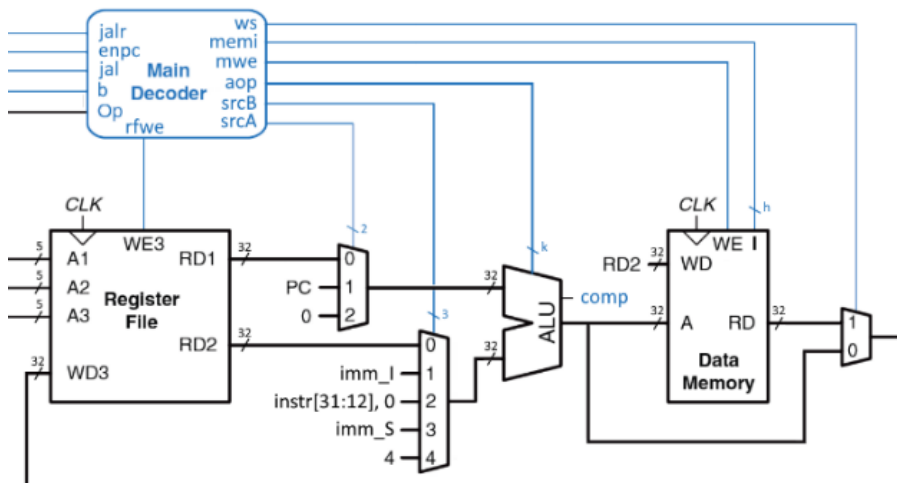
$B = 1'd1$ (переход при $C = 1$ на константу Imm_B), $jal = 1'd0$, $enpc = 1'd1$, $jalr = 1'd0$

Реализация инструкции типа J (opcode = 7'd111)

Да, для одной инструкции выделили целый opcode (в типе U на 2 инструкции также будет 2 opcode)

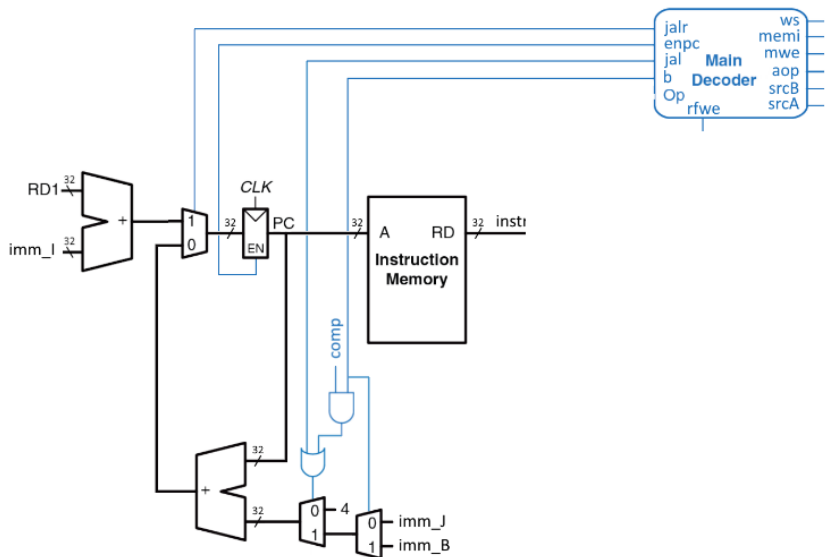
jal	Jump And Link	Переход с сохранением адреса возврата	$rd = PC + 4; PC += imm$	J	11011111
-----	---------------	---------------------------------------	--------------------------	---	----------

Отличием от $jalr$ является переход на какую-то константную величину относительно текущего значения счетчика (относительный переход), а не переход на строку "RA1+const", которая уже задается абсолютно.



Mwe = 1'd0, Memi = 5'd0, ws = 1'd0, rfwe = 1'd1, (работаем с регистровым файлом, а не с памятью)

srcA = 2'd1, srcB = 3'd4, aop = 5'd0 (берем значение с PC и 4, делаем сложение (0 операция на АЛУ) и записываем в RF)



Jal = 1'd1 (так-как надо перейти на значение в регистре 1 + константа Imm_J)

Enpc = 1'd1, jalr = 1'd0, b = 1'd0 (типичные значения на выходах декодера)

Реализация инструкции типа U (opcode = 7'd23, 7'd25)

imm[31:12]	rd	opcode	U-type
------------	----	--------	--------

Я объединил 2 opcode в одно описание, так-как для типа инструкций U opcode играет роль мультиплексора, а сами инструкции выполняют примерно одно и тоже, поэтому я сначала рассмотрю одинаковые параметры декодера для них, а потом уже про различия поговорим.

lui	Load Upper Immediate	Загрузить константу в сдвинутую на 12	rd = imm << 12	U	0110111
auipc	Add Upper Immediate to PC	Сохранить счетчик команд в сумме с константой << 12	rd = PC + (imm << 12)		0010111

Одинаково для всех инструкций типа U

Mwe = 1'd0, Memi = 5'd0, ws = 1'd0, rfwe = 1'd1, (работаем с регистровым файлом, а не с памятью)

srcB = 3'd2, enpc = 1'd1, b = 1'd0, jal = 1'd0, jalr = 1'd0, aop = 5'd0

Для lui (opcode = 7'd55)

srcA = 2'd2

Для auipc (opcode = 7'd23)

srcA = 2'd1

Декодер к простейшему RISC-V на Verilog (полностью). Надо затычку поменять на 0.

```
1 module Decoder_R (
2     input [6:0] opcode,
3     input [2:0] func3,
4     input [6:0] func7,
5     output jalr,
6     output enpc,
7     output jal,
8     output b,
9     output ws,
10    output [4:0] memi,
11    output mwe,
12    output rfwe,
13    output [4:0] aop,
14    output [2:0] srcB,
15    output [1:0] srcA
16 );
17
18 parameter opcode_R = 7'd51;
19 parameter opcode_I_1 = 7'd19;
20 parameter opcode_I_2 = 7'd3;
21 parameter opcode_I_3 = 7'd103;
22 parameter opcode_S = 7'd35;
23 parameter opcode_B = 7'd99;
24 parameter opcode_J = 7'd111;
25 parameter opcode_U_lui = 7'd55;
26 parameter opcode_U_auipc = 7'd23;
27
28 wire [31:0] ZAT; //затычка в декодере
29 assign ZAT = 32'd1;
30
31 assign srcA = (opcode == opcode_R) ? 2'd0 :
32              (opcode == opcode_I_1) ? 2'd0 :
33              (opcode == opcode_I_2) ? 2'd0 :
34              (opcode == opcode_I_3) ? 2'd1 :
35              (opcode == opcode_S) ? 2'd3 :
36              (opcode == opcode_B) ? 2'd0 :
37              (opcode == opcode_J) ? 2'd1 :
38              (opcode == opcode_U_lui) ? 2'd2 :
39              (opcode == opcode_U_auipc) ? 2'd1 : ZAT[1:0];
40
41
42
43
44 assign srcB = (opcode == opcode_R) ? 3'd0 :
45              (opcode == opcode_I_1) ? 3'd1 :
46              (opcode == opcode_I_2) ? 3'd1 :
47              (opcode == opcode_I_3) ? 3'd4 :
48              (opcode == opcode_S) ? 3'd3 :
49              (opcode == opcode_B) ? 3'd0 :
50              (opcode == opcode_J) ? 3'd4 :
51              ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 3'd2 : ZAT[2:0];
52
53 assign memi = (opcode == opcode_R) ? 5'd0 :
54              (opcode == opcode_I_1) ? 5'd0 :
55              (opcode == opcode_I_2) ? {1'b1, 1'b0, func3} :
56              (opcode == opcode_I_3) ? 5'd0 :
57              (opcode == opcode_S) ? {1'b0, 1'b1, func3} :
58              (opcode == opcode_B) ? 5'd0 :
59              (opcode == opcode_J) ? 5'd0 :
60              ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 5'd0 : ZAT[4:0];
61
62
63 assign aop = (opcode == opcode_R) ? {func7[6:5], func3} :
64              (opcode == opcode_I_1) ? {2'd0, func3} :
65              (opcode == opcode_I_2) ? {2'd0, func3} :
66              (opcode == opcode_I_3) ? 5'd0 :
67              (opcode == opcode_S) ? 5'd0 :
68              (opcode == opcode_B) ? {2'd3, func3} :
69              (opcode == opcode_J) ? 5'd0 :
70              ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 5'd0 : ZAT[4:0];
71
72 assign enpc = (opcode == opcode_R) ? 1'd1 :
73              (opcode == opcode_I_1) ? 1'd1 :
74              (opcode == opcode_I_2) ? 1'd1 :
75              (opcode == opcode_I_3) ? 1'd1 :
76              (opcode == opcode_S) ? 1'd1 :
77              (opcode == opcode_B) ? 1'd1 :
78              (opcode == opcode_J) ? 1'd1 :
79              ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd1 : ZAT[0];
80
81 assign ws = (opcode == opcode_R) ? 1'd0 :
82             (opcode == opcode_I_1) ? 1'd0 :
83             (opcode == opcode_I_2) ? 1'd1 :
84             (opcode == opcode_I_3) ? 1'd0 :
85             (opcode == opcode_S) ? 1'd1 :
86             (opcode == opcode_B) ? 1'd0 :
87             (opcode == opcode_J) ? 1'd0 :
88             ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd0 : ZAT[0];
89
90 assign mwe = (opcode == opcode_R) ? 1'd0 :
91             (opcode == opcode_I_1) ? 1'd0 :
92             (opcode == opcode_I_2) ? 1'd1 :
93             (opcode == opcode_I_3) ? 1'd0 :
94             (opcode == opcode_S) ? 1'd1 :
95             (opcode == opcode_B) ? 1'd0 :
96             (opcode == opcode_J) ? 1'd0 :
97             ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd0 : ZAT[0];
98
99 assign rfwe = (opcode == opcode_R) ? 1'd1 :
100             (opcode == opcode_I_1) ? 1'd1 :
101             (opcode == opcode_I_2) ? 1'd1 :
102             (opcode == opcode_I_3) ? 1'd1 :
103             (opcode == opcode_S) ? 1'd0 :
104             (opcode == opcode_B) ? 1'd0 :
105             (opcode == opcode_J) ? 1'd1 :
106             ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd1 : ZAT[0];
107
108 assign jalr = (opcode == opcode_R) ? 1'd0 :
109             (opcode == opcode_I_1) ? 1'd0 :
110             (opcode == opcode_I_2) ? 1'd0 :
111             (opcode == opcode_I_3) ? 1'd1 :
112             (opcode == opcode_S) ? 1'd0 :
113             (opcode == opcode_B) ? 1'd0 :
114             (opcode == opcode_J) ? 1'd0 :
115             ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd0 : ZAT[0];
116
117
118 assign jal = (opcode == opcode_R) ? 1'd0 :
119            (opcode == opcode_I_1) ? 1'd0 :
120            (opcode == opcode_I_2) ? 1'd0 :
121            (opcode == opcode_I_3) ? 1'd0 :
122            (opcode == opcode_S) ? 1'd0 :
123            (opcode == opcode_B) ? 1'd0 :
124            (opcode == opcode_J) ? 1'd1 :
125            ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd0 : ZAT[0];
126
127 assign b = (opcode == opcode_R) ? 1'd0 :
128           (opcode == opcode_I_1) ? 1'd0 :
129           (opcode == opcode_I_2) ? 1'd0 :
130           (opcode == opcode_I_3) ? 1'd0 :
131           (opcode == opcode_S) ? 1'd0 :
132           (opcode == opcode_B) ? 1'd1 :
133           (opcode == opcode_J) ? 1'd0 :
134           ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd0 : ZAT[0];
135
136 endmodule
```

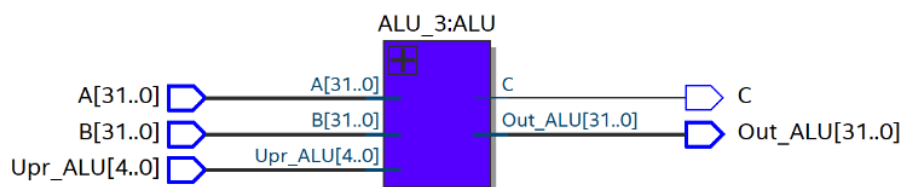
Лекция 8. RISC-V на Verilog

Введение

На самом деле, изначально не планировалось разбивать статью про RISC-V на 3(4, если считать декодер) лекции, но после того, как автор увидел примерное число страниц, которое выходит на описание данной темы (порядка 20-30), то было принято решение разделить лекция на несколько частей: архитектурную, декодер, реализацию архитектурной части на Verilog (то есть, в данной статье большая часть содержания – скрины кода и тесты получившегося устройства с некоторыми пояснениями) и прошивка (которая выполняет роль тестирования операций регистр/регистр, регистр/константа и условных переходов) (следующая статья) устройства

АЛУ к RISC-V.

АЛУ к данному процессору было реализовано в статье 1 про АЛУ (АЛУ под номером 3), но я напому некоторые моменты оттуда.



Для выхода АЛУ (у нас вместо result_0 выход Out_ALU

Операция	={cmp, mod, opcode}	Выражение	Действие
ADD	0 0 000	result_o = a_i + b_i	Сложение
SUB	0 1 000	result_o = a_i - b_i	Вычитание
SLL	0 0 001	result_o = a_i << b_i	Сдвиг влево
SLTS	0 0 010	result_o = a_i < b_i	Знаковое сравнение
SLTU	0 0 011	result_o = a_i < b_i	Беззнаковое сравнение
XOR	0 0 100	result_o = a_i ^ b_i	Побитовое исключающее ИЛИ
SRL	0 0 101	result_o = a_i >> b_i	Сдвиг вправо
SRA	0 1 101	result_o = a_i >>> b_i	Арифметический сдвиг вправо (операнд a_i — знаковый)
OR	0 0 110	result_o = a_i b_i	Побитовое логическое ИЛИ
AND	0 0 111	result_o = a_i & b_i	Побитовое логическое И

Роль флага сравнения у нас будет играть бит C

Операция	={cmp, mod, opcode}	Выражение	Действие
EQ	1 1 000	flag_o = (a_i == b_i)	Выставить флаг, если равны
NE	1 1 001	flag_o = (a_i != b_i)	Выставить флаг, если не равны
LTS	1 1 100	flag_o = a_i < b_i	Знаковое сравнение <
GES	1 1 101	flag_o = a_i ≥ b_i	Знаковое сравнение ≥
LTU	1 1 110	flag_o = a_i < b_i	Беззнаковое сравнение <
GEU	1 1 111	flag_o = a_i ≥ b_i	Беззнаковое сравнение ≥

Описание АЛУ(есть недочеты небольшие, исправленное АЛУ на гитхабе).

```
1 module ALU_3 (
2     input [4:0] Upr_ALU,
3     input [31:0] A,
4     input [31:0] B,
5     output C,
6     output [31:0] Out_ALU
7 );
8 parameter ADD = 5'b000000;
9 parameter SUB = 5'b010000;
10 parameter SLL = 5'b000001;
11 parameter SLTS = 5'b000010;
12 parameter SLTU = 5'b000011;
13 parameter XOR = 5'b000100;
14 parameter SRL = 5'b000101;
15 parameter SRA = 5'b001101;
16 parameter OR = 5'b000110;
17 parameter AND = 5'b000111;
18 parameter EQ = 5'b110000;
19 parameter NE = 5'b110001;
20 parameter LTS = 5'b111100;
21 parameter GES = 5'b111101;
22 parameter LTU = 5'b111110;
23 parameter GEU = 5'b111111;
24
25 wire [31:0] out_or;
26 wire [31:0] out_and;
27 wire [31:0] out_add;
28 wire [31:0] out_sub;
29 wire [31:0] out_xor;
30 wire [31:0] out_sll;
31 wire [31:0] out_slts;
32 wire [31:0] out_sltu;
33 wire [31:0] out_srl;
34 wire [31:0] out_sra;
35 wire out_A_menshe_B_zn;
36 wire out_A_ne_menshe_B_zn;
37 wire out_A_menshe_B;
38 wire out_A_ne_menshe_B;
39 wire out_equal;
40 wire out_notequal;
41
42 assign out_A_ne_menshe_B_zn = ~out_A_menshe_B_zn;
43 assign out_A_ne_menshe_B = ~out_A_menshe_B;
44 assign out_equal = (A==B) ? 1'b1 : 1'b0;
45 assign out_A_menshe_B = (A<B) ? 1'b1 : 1'b0;
46 assign out_notequal = ~out_equal;
47
48 assign out_add = A+B;
49 assign out_sub = A-B;
50 assign out_sll = A << B;
51 assign out_slts = {{31{1'b0}},out_A_menshe_B_zn};
52 assign out_sltu = {{31{1'b0}},out_A_menshe_B};
53 assign out_srl = A >> B;
54 assign out_sra = A >>> B;
55
56
57 assign Out_ALU = (Upr_ALU == ADD) ? out_add:
58                 (Upr_ALU == SUB) ? out_sub:
59                 (Upr_ALU == SLL) ? out_sll:
60                 (Upr_ALU == SLTS) ? out_slts:
61                 (Upr_ALU == SLTU) ? out_sltu:
62                 (Upr_ALU == XOR) ? out_xor:
63                 (Upr_ALU == SRL) ? out_srl:
64                 (Upr_ALU == SRA) ? out_sra:
65                 (Upr_ALU == OR) ? out_or:
66                 (Upr_ALU == AND) ? out_and:32'b0;
67
68 assign C = (Upr_ALU == EQ) ? out_equal:
69           (Upr_ALU == NE) ? out_notequal:
70           (Upr_ALU == LTS) ? out_A_menshe_B_zn:
71           (Upr_ALU == GES) ? out_A_ne_menshe_B_zn:
72           (Upr_ALU == LTU) ? out_A_menshe_B:
73           (Upr_ALU == GEU) ? out_A_ne_menshe_B:1'b0;
74
75
76 or_32_b or_ALU (
77     .A(A),
78     .B(B),
79     .out_or(out_or)
80 );
81
82
83 and_32_b and_ALU (
84     .A(A),
85     .B(B),
86     .out_and(out_and)
87 );
88
89 s1_mod2 mod2_ALU (
90     .A(A),
91     .B(B),
92     .out(out_xor)
93 );
94
95 comp_menshe zn_comp (
96     .A(A),
97     .B(B),
98     .out(out_A_menshe_B_zn)
99 );
100 endmodule
```

```

1 module sl_mod2 (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out
5 );
6
7 assign out = A ^ B;
8
9 endmodule

```

```

1 module or_32_b (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out_or
5 );
6
7
8 assign out_or = A | B;
9 endmodule

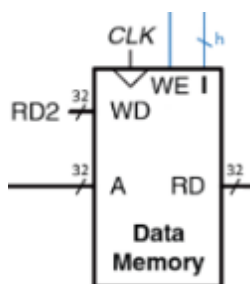
```

```

1 module and_32_b (
2     input [31:0] A,
3     input [31:0] B,
4     output [31:0] out_and
5 );
6
7 assign out_and = A & B;
8 endmodule

```

Data Memory

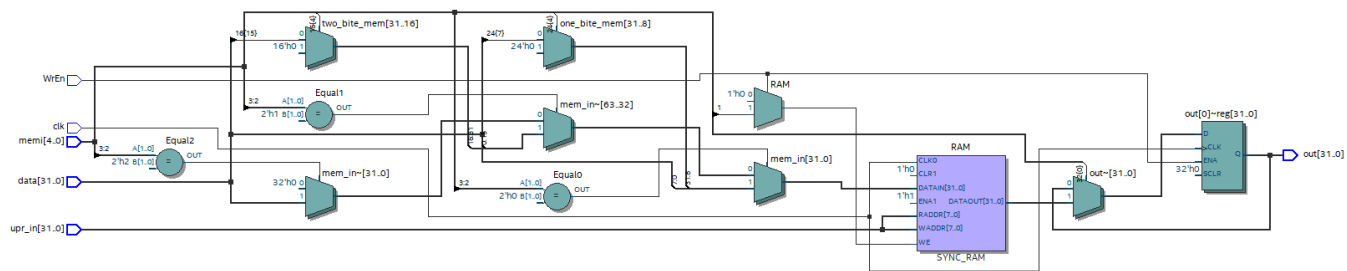


Данная штукавина практически полностью аналогична регистровому файлу кроме управляющих битов.

Биты	Название	Значения	Описание
0	mem_read	0 – нет чтения, 1 – чтение	Активирует чтение из памяти (для LW, LB, LH и т. д.).
1	mem_write	0 – нет записи, 1 – запись	Активирует запись в память (для SW, SB, SH).
2–3	mem_size	00 – byte (8 бит), 01 – half (16), 10 – word (32)	Определяет размер данных для загрузки/сохранения.
4 (опц.)	mem_unsigned	0 – знаковое, 1 – беззнаковое	Для загрузки (LBU, LHU – беззнаковое расширение).

То есть, залезаем в код для регистрового файла, ставим там пару-тройку мультиплексоров для реализации memi и получаем в итоге Data Memory.

RTL схема Data Memory



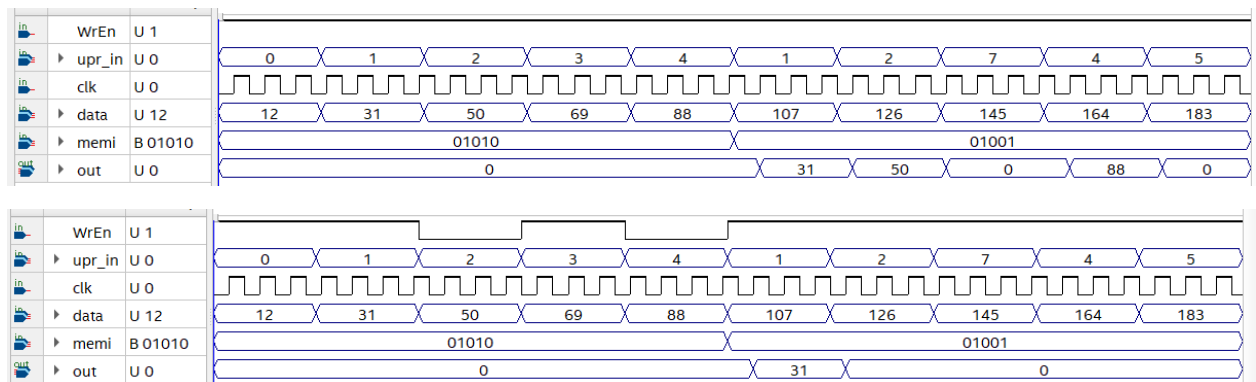
Описание Data Memory на Verilog.

```

1  module data_mem(
2      input [4:0] memi,
3      input [31:0] data,
4      input [31:0] upr_in,
5      output reg [31:0] out,
6      input clk,
7      input WrEn
8  );
9
10
11  reg [31:0] RAM [0:255];
12  wire [31:0] mem_in;
13  wire [31:0] one_bite_mem_zn;
14  wire [31:0] two_bite_mem_zn;
15  wire [31:0] one_bite_mem_bez;
16  wire [31:0] two_bite_mem_bez;
17  wire [31:0] one_bite_mem;
18  wire [31:0] two_bite_mem;
19  wire [31:0] full_bite_mem;
20
21  assign one_bite_mem = (memi[4]) ? one_bite_mem_bez : one_bite_mem_zn;
22  assign two_bite_mem = (memi[4]) ? two_bite_mem_bez : two_bite_mem_zn;
23
24  assign one_bite_mem_bez = {24'd0, data[7:0]};
25  assign one_bite_mem_zn = {{24 {data[7]}} , data[7:0]};
26  assign two_bite_mem_bez = {16'd0, data[15:0]};
27  assign two_bite_mem_zn = {{16 {data[15]}} , data[15:0]};
28  assign full_bite_mem = data;
29
30  assign mem_in = (memi[3:2] == 2'd0) ? one_bite_mem :
31                  (memi[3:2] == 2'd1) ? two_bite_mem :
32                  (memi[3:2] == 2'd2) ? full_bite_mem : 32'd0;
33
34
35
36  always @(posedge clk) begin
37      if (WrEn) begin
38          if (memi[1]) RAM[upr_in] <= mem_in;
39          if (memi[0]) out <= RAM[upr_in];
40      end
41  end
42
43  end
44
45  endmodule

```

Небольшое тестирование Data Memory



Декодер инструкций. Затычку надо поставить 32'd0 (иначе будет веселая константа)

```
1 module Decoder_R (
2     input [6:0] opcode,
3     input [2:0] func3,
4     input [6:0] func7,
5     output jalr,
6     output enpc,
7     output jal,
8     output b,
9     output ws,
10    output [4:0] memi,
11    output mwe,
12    output rfwe,
13    output [4:0] aop,
14    output [2:0] srcB,
15    output [1:0] srcA
16 );
17
18 parameter opcode_R = 7'd51;
19 parameter opcode_I_1 = 7'd19;
20 parameter opcode_I_2 = 7'd3;
21 parameter opcode_I_3 = 7'd103;
22 parameter opcode_S = 7'd35;
23 parameter opcode_B = 7'd99;
24 parameter opcode_J = 7'd111;
25 parameter opcode_U_lui = 7'd55;
26 parameter opcode_U_auipc = 7'd23;
27
28 wire [31:0] ZAT; //затычка в декодере
29 assign ZAT = 32'd1;
30
31 assign srcA = (opcode == opcode_R) ? 2'd0 :
32              (opcode == opcode_I_1) ? 2'd0 :
33              (opcode == opcode_I_2) ? 2'd0 :
34              (opcode == opcode_I_3) ? 2'd1 :
35              (opcode == opcode_S) ? 2'd3 :
36              (opcode == opcode_B) ? 2'd0 :
37              (opcode == opcode_J) ? 2'd1 :
38              (opcode == opcode_U_lui) ? 2'd2 :
39              (opcode == opcode_U_auipc) ? 2'd1 : ZAT[1:0];
40
41
42
43
44 assign srcB = (opcode == opcode_R) ? 3'd0 :
45              (opcode == opcode_I_1) ? 3'd1 :
46              (opcode == opcode_I_2) ? 3'd1 :
47              (opcode == opcode_I_3) ? 3'd4 :
48              (opcode == opcode_S) ? 3'd3 :
49              (opcode == opcode_B) ? 3'd0 :
50              (opcode == opcode_J) ? 3'd4 :
51              ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 3'd2 : ZAT[2:0];
52
53 assign memi = (opcode == opcode_R) ? 5'd0 :
54              (opcode == opcode_I_1) ? 5'd0 :
55              (opcode == opcode_I_2) ? {1'b1, 1'b0, func3} :
56              (opcode == opcode_I_3) ? 5'd0 :
57              (opcode == opcode_S) ? {1'b0, 1'b1, func3} :
58              (opcode == opcode_B) ? 5'd0 :
59              (opcode == opcode_J) ? 5'd0 :
60              ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 5'd0 : ZAT[4:0];
61
62
63 assign aop = (opcode == opcode_R) ? {func7[6:5], func3} :
64              (opcode == opcode_I_1) ? {2'd0, func3} :
65              (opcode == opcode_I_2) ? {2'd0, func3} :
66              (opcode == opcode_I_3) ? 5'd0 :
67              (opcode == opcode_S) ? 5'd0 :
68              (opcode == opcode_B) ? {2'd3, func3} :
69              (opcode == opcode_J) ? 5'd0 :
70              ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 5'd0 : ZAT[4:0];
71
72 assign enpc = (opcode == opcode_R) ? 1'd1 :
73              (opcode == opcode_I_1) ? 1'd1 :
74              (opcode == opcode_I_2) ? 1'd1 :
75              (opcode == opcode_I_3) ? 1'd1 :
76              (opcode == opcode_S) ? 1'd1 :
77              (opcode == opcode_B) ? 1'd1 :
78              (opcode == opcode_J) ? 1'd1 :
79              ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd1 : ZAT[0];
80
81 assign ws = (opcode == opcode_R) ? 1'd0 :
82             (opcode == opcode_I_1) ? 1'd0 :
83             (opcode == opcode_I_2) ? 1'd1 :
84             (opcode == opcode_I_3) ? 1'd0 :
85             (opcode == opcode_S) ? 1'd1 :
86             (opcode == opcode_B) ? 1'd0 :
87             (opcode == opcode_J) ? 1'd0 :
88             ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd0 : ZAT[0];
89
90 assign mwe = (opcode == opcode_R) ? 1'd0 :
91             (opcode == opcode_I_1) ? 1'd0 :
92             (opcode == opcode_I_2) ? 1'd1 :
93             (opcode == opcode_I_3) ? 1'd0 :
94             (opcode == opcode_S) ? 1'd1 :
95             (opcode == opcode_B) ? 1'd0 :
96             (opcode == opcode_J) ? 1'd0 :
97             ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd0 : ZAT[0];
98
99 assign rfwe = (opcode == opcode_R) ? 1'd1 :
100             (opcode == opcode_I_1) ? 1'd1 :
101             (opcode == opcode_I_2) ? 1'd1 :
102             (opcode == opcode_I_3) ? 1'd1 :
103             (opcode == opcode_S) ? 1'd0 :
104             (opcode == opcode_B) ? 1'd0 :
105             (opcode == opcode_J) ? 1'd1 :
106             ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd1 : ZAT[0];
107
108 assign jalr = (opcode == opcode_R) ? 1'd0 :
109             (opcode == opcode_I_1) ? 1'd0 :
110             (opcode == opcode_I_2) ? 1'd0 :
111             (opcode == opcode_I_3) ? 1'd1 :
112             (opcode == opcode_S) ? 1'd0 :
113             (opcode == opcode_B) ? 1'd0 :
114             (opcode == opcode_J) ? 1'd0 :
115             ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd0 : ZAT[0];
116
117
118 assign jal = (opcode == opcode_R) ? 1'd0 :
119             (opcode == opcode_I_1) ? 1'd0 :
120             (opcode == opcode_I_2) ? 1'd0 :
121             (opcode == opcode_I_3) ? 1'd0 :
122             (opcode == opcode_S) ? 1'd0 :
123             (opcode == opcode_B) ? 1'd0 :
124             (opcode == opcode_J) ? 1'd1 :
125             ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd0 : ZAT[0];
126
127 assign b = (opcode == opcode_R) ? 1'd0 :
128           (opcode == opcode_I_1) ? 1'd0 :
129           (opcode == opcode_I_2) ? 1'd0 :
130           (opcode == opcode_I_3) ? 1'd0 :
131           (opcode == opcode_S) ? 1'd0 :
132           (opcode == opcode_B) ? 1'd1 :
133           (opcode == opcode_J) ? 1'd0 :
134           ((opcode == opcode_U_lui) |(opcode == opcode_U_auipc)) ? 1'd0 : ZAT[0];
135 endmodule
```

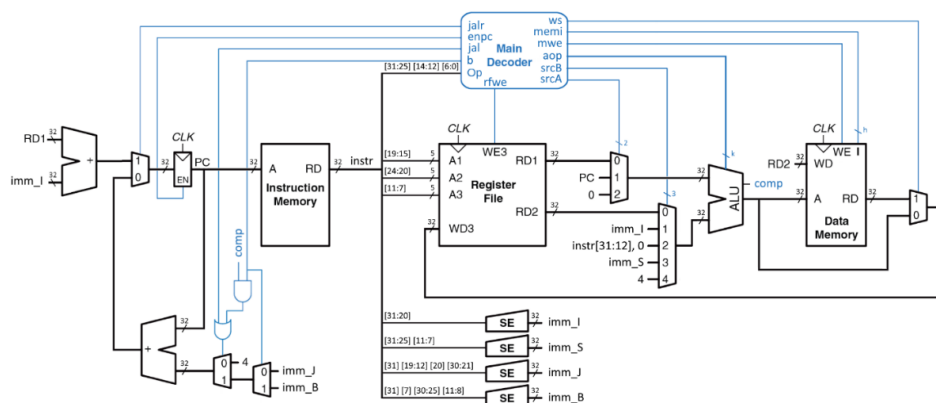
Я решил вывести его в отдельный модуль, так-как счетчик практически не связан с остальным процессором (не считая выходы декодера) и занимает довольно много места в коде, что делает его не очень хорошо читаемым.

```

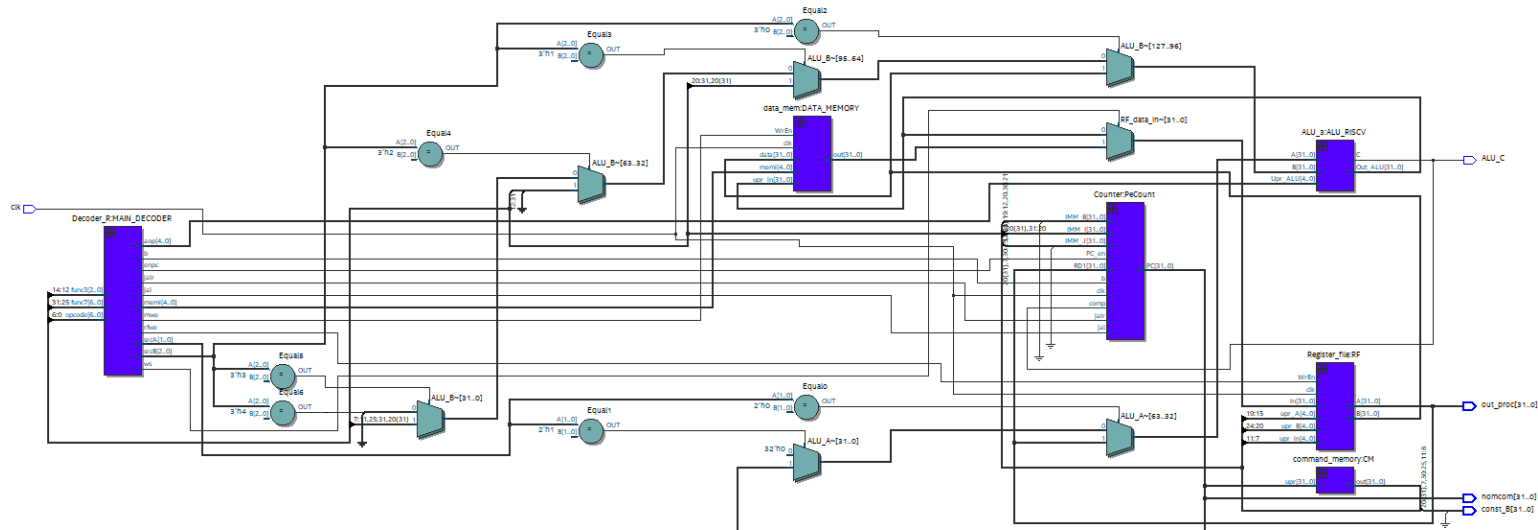
1 module Counter (
2     input  clk,
3     input  comp,
4     input  PC_en,
5     input  [31:0] RD1,
6     input  [31:0] IMM_I,
7     input  [31:0] IMM_J,
8     input  [31:0] IMM_B,
9     input  b,
10    input  jal,
11    input  jalr,
12    output reg [31:0] PC
13 );
14
15 wire [31:0] PC_PLUS;
16 wire b_comp;
17 wire upr_mux_1;
18 wire upr_mux_2;
19 wire [31:0] out_mux_2;
20 wire [31:0] PC_IN;
21 assign b_comp = b & comp;
22 assign upr_mux_1 = b_comp | jal;
23 assign upr_mux_2 = b;
24 assign PC_PLUS = (upr_mux_1 == 1'd0) ? 32'd4:out_mux_2;
25 assign out_mux_2 = (upr_mux_2 == 1'd0) ? IMM_J : IMM_B;
26 assign PC_IN = (jalr == 0) ? (PC+PC_PLUS) : (RD1+IMM_I);
27
28
29
30 always @(posedge clk) begin
31     if (PC_en == 1) PC <= PC_IN;
32     else PC <= PC;
33 end
34
35 endmodule

```

Схема RISC-V концептуальная



RTL схема RISC-V (что по описанию на Verilog синтезировалось).



В общем-то и все, процессор готов, осталось его протестировать (описание на Verilog будет на следующей страничке). Однако, как писалось выше, автор будет придерживаться “тестированием прошивкой”, где мы будем прошивать RISC-V, и если он на наших примерах работает, то и хорошо (такой поиск тривиальных ошибок), чем мы и займемся в следующей статье.

Как читатель может видеть, простейший одноктактный RISC-V весьма прост (да, тавтология) и занимает порядка 135(ядро)+100(АЛУ)+15(command_memory)+45(data memory)+135(декодер)+26(регистровый файл)+38(модули к АЛУ) = 464 строчек кода (порядка 500), что весьма подъемно для реализации за недельки так 2 с чашечкой чая (гикам можно меньше, разрешаю).

Описание на Verilog RISC-V (код есть на гитхабе).

```
1 module RISC_V(
2     input clk,
3     output [31:0] out_proc,
4     output ALU_C,
5     output [31:0] nomcom,
6     output [31:0] const_B
7 );
8
9     assign nomcom = PC;
10    assign const_B = IMM_B;
11
12    wire [31:0] PC;
13    wire [31:0] out_com;
14
15    wire [31:0] ALU_A;
16    wire [31:0] ALU_B;
17    wire [31:0] ALU_OUT;
18    wire [5:0] ALU_UPR;
19    wire comp;
20
21    wire WE_RF;
22    wire [4:0] RF_A;
23    wire [4:0] RF_B;
24    wire [4:0] RF_in;
25    wire [31:0] RF_data_A;
26    wire [31:0] RF_data_B;
27    wire [31:0] RF_data_in;
28    wire [31:0] data_mem;
29
30    wire [6:0] opcode_dec;
31    wire [2:0] func3;
32    wire [6:0] func7;
33    wire b_dec;
34    wire mem_WE_dec;
35    wire [4:0] memi;
36    wire jal;
37    wire jalr;
38    wire enpc;
39    wire ws;
40    wire [31:0] out_mem;
41
42    wire [1:0] UPR_MUX_A;
43    wire [2:0] UPR_MUX_B;
44
45    wire [31:0] IMM_I;
46    wire [31:0] IMM_S;
47    wire [31:0] IMM_J;
48    wire [31:0] IMM_B;
49
50    assign IMM_I = {{20{out_com[31]}},out_com[31:20]};
51    assign IMM_S = {{20{out_com[31]}},out_com[31:25],out_com[11:7]};
52    assign IMM_J = {{12{out_com[31]}}, out_com[19:12], out_com[20], out_com[30:21],1'd0};
53    assign IMM_B = {{20{out_com[31]}},out_com[], out_com[30:25], out_com[11:8], 1'd0};
54
55    assign func3 = out_com[14:12];
56    assign opcode_dec = out_com[6:0];
57    assign func7 = out_com[31:25];
58    assign RF_A = out_com[19:15];
59    assign RF_B = out_com[24:20];
60    assign RF_in = out_com[11:7];
61    assign data_mem = RF_data_B;
62
63    assign ALU_A = (UPR_MUX_A == 2'd0) ? RF_data_A :
64                  (UPR_MUX_A == 2'd1) ? PC : 32'd0;
65
66    assign ALU_B = (UPR_MUX_B == 3'd0) ? RF_data_B :
67                  (UPR_MUX_B == 3'd1) ? IMM_I :
68                  (UPR_MUX_B == 3'd2) ? {out_com[31:12],12'd0} :
69                  (UPR_MUX_B == 3'd3) ? IMM_S :
70                  (UPR_MUX_B == 3'd4) ? 32'd4 : 32'd0;
71
72    assign RF_data_in = (ws) ? out_mem : ALU_OUT;
73    assign out_proc = RF_data_A;
74    assign ALU_C = comp;
75
76    Counter PeCount (
77        .clk(clk),
78        .comp(comp),
79        .PC_en(enpc),
80        .RD1(RF_data_A),
81        .IMM_I(IMM_I),
82        .IMM_J(IMM_J),
83        .IMM_B(IMM_B),
84        .b(b_dec),
85        .jal(jal),
86        .jalr(jalr),
87        .PC(PC)
88    );
89
90    Decoder_R_MAIN_DECODER (
91        .opcode(opcode_dec),
92        .func3(func3),
93        .func7(func7),
94        .jalr(jalr),
95        .enpc(enpc),
96        .jal(jal),
97        .b(b_dec),
98        .ws(ws),
99        .memi(memi),
100       .mwe(mem_WE_dec),
101       .rfwe(WE_RF),
102       .aop(ALU_UPR),
103       .srcB(UPR_MUX_B),
104       .srcA(UPR_MUX_A)
105    );
106
107    data_mem DATA_MEMORY(
108        .memi(memi),
109        .data(data_mem),
110        .upr_in(ALU_OUT),
111        .out(out_mem),
112        .clk(clk),
113        .WEEn(mem_WE_dec)
114    );
115
116    ALU_3 ALU_RISCV(
117        .Upr_ALU(ALU_UPR),
118        .A(ALU_A),
119        .B(ALU_B),
120        .C(comp),
121        .Out_ALU(ALU_OUT)
122    );
123
124    Register_file RF(
125        .upr_A(RF_A),
126        .upr_B(RF_B),
127        .upr_in(RF_in),
128        .in(RF_data_in),
129        .A(RF_data_A),
130        .B(RF_data_B),
131        .clk(clk),
132        .WEEn(WE_RF)
133    );
134
135    Command_memory CM (
136        .upr(PC),
137        .out(out_com)
138    );
139
140
141 endmodule
```

Лекция 9. Прошивка RISC-V.

Введение

В данной статье мы будем сначала писать прошивку вручную (аналогично такой же статье про Cobra), а далее уже для более интересных программ пользоваться ассемблером, который получается из C (а затем переводить его в машинный код).

Ручное программирование RISC-V.

Как мы выяснили в статье про архитектуру, команды в RISC-V имеют такую кодировку:

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Для написания чего-то очень простого (например, сложить 8 и 9) нам хватит операции типа регистр/регистр(R) и регистр/константа(I)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type

Пример 1. Найти 8+9, данные снимаем с выхода регистрового файла (RA1)

addi	ADDition Immediate	Сложение с константой	rd = rs1 + imm	I	0x0 0x4 0x6 0x7
xori	eXclusive OR Immediate	Побитовое Исключающее ИЛИ с константой	rd = rs1 ^ imm		
ori	OR Immediate	Побитовое ИЛИ с константой	rd = rs1 imm		
andi	AND Immediate	Побитовое И с константой	rd = rs1 & imm		

Для решения данной простейшей задачи воспользуемся только типом I (opcode = 7'd19).

Строчка 1. Загрузка 8 в регистр 1 (прибавление 8 к 0, что лежит в регистре 1)

000000001000 00001 000 00001 0010011

0010011₂ = 19 – opcode данного типа команды

00001₂ = 1 – регистр, в который мы записываем константу

000₂ – funct3, показывает как бы операцию на АЛУ (сложение)

00001₂ – регистр, с которым складываем константу

000000001000₂ = 8 – с чем складываем значение в регистре (константа)

Строчка 2. Прибавление 9 к 8, что лежит в регистре 1)

000000001001 00001 000 00001 0010011

$0010011_2 = 19$ – *opcode* данного типа команды

$00001_2 = 1$ – регистр, в который мы записываем результат сложения

000_2 – *func3*, показывает как бы операцию на АЛУ (сложение)

00001_2 – регистр, с которым складываем константу

$000000001001_2 = 9$ – с чем складываем значение в регистре (константа)

Строчка 3. Вывод результата операции в RA1 (выводим RA1+0)

000000000000 00001 000 00001 0010011

$0010011_2 = 19$ – *opcode* данного типа команды

$00001_2 = 1$ – регистр, в который мы записываем результат сложения

000_2 – *func3*, показывает как бы операцию на АЛУ (сложение)

00001_2 – регистр, с которым складываем константу

$000000000000_2 = 0$ – с чем складываем значение в регистре (константа)

Итоговая прошивка:

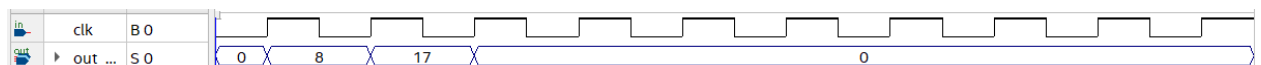
```
00000000100000001000000010010011
00000000100100001000000010010011
00000000000000001000000010010011
```

Что будет на выходе RA1 (out_proc) по event (clk из 0 в 1)

- 0) На выходе 0 (8 еще не запасалось)
- 1) На выходе 8 (результат операции на АЛУ (8+9) еще не записался)
- 2) На выходе 17 (Выводим значение в RA1+0)

Далее же на выходах будут 0

Результат симуляции RISC-V с такой прошивкой.



Как читатель видит, работает процессор (инструкции с *opcode* = 19 (I_1) так, как нам нужно

Пример 2. Реализация $(-15+3)$ с помощью инструкций типа R и типа I.

На самом деле, данную операцию опять-таки можно элементарно реализовать с помощью типа I_1 (opcode = 19), но мы не только прошиваем RISC-V, мы его еще и немного тестируем, поэтому совершаем относительно неоптимальные действия (как сейчас).

Табличка кодировки инструкций (opcode_R = 7'd51, opcode_I_1 = 7'd19)

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type

Строчка 1. Загрузка (-15) в регистр 1 (прибавление (-15) к 0, что лежит в регистре 1)

111111110001 00001 000 00001 0010011

$0010011_2 = 19$ – opcode данного типа команды

$00001_2 = 1$ – регистр, в который мы записываем константу

000_2 – funct3, показывает как бы операцию на АЛУ (сложение)

00001_2 – регистр, с которым складываем константу

$111111110001_2 = -15$ – с чем складываем значение в регистре (константа)

Строчка 2. Загрузка 3 в регистр 2 (прибавление 3 к 0, что лежит в регистре 0)

000000000011 00000 000 00010 0010011

$0010011_2 = 19$ – opcode данного типа команды

$00010_2 = 2$ – регистр, в который мы записываем константу

000_2 – funct3, показывает как бы операцию на АЛУ (сложение)

00000_2 – регистр, с которым складываем константу

$000000000011_2 = 3$ – с чем складываем значение в регистре (константа)

Строчка 3. Складываем 2 регистр(3) и 1 регистр(-15) и записываем это в 1 регистр

0000000 00001 00010 000 00001 0110011

$0110011_2 = 51$ – opcode данного типа команды (R)

$00001_2 = 1$ – регистр, в который мы записываем результат операции на АЛУ

000_2 – funct3, показывает как бы операцию на АЛУ (сложение)

00010_2 – регистр, с которым мы проводим операцию на АЛУ (RA1)

$00001_2 = 1$ – регистр, с которым мы проводим операцию на АЛУ (RA2)

$0000000_2 = 0$ – funct7

Строчка 4. Вывод результата операции в RA1 (выводим RA1+0)

000000000000 00001 000 00001 0010011

$0010011_2 = 19$ — *opcode* данного типа команды

$00001_2 = 1$ — регистр, в который мы записываем результат сложения

000_2 — *func3*, показывает как бы операцию на АЛУ (сложение)

00001_2 — регистр, с которым складываем константу

$000000000000_2 = 0$ — с чем складываем значение в регистре (константа)

Итоговая прошивка:

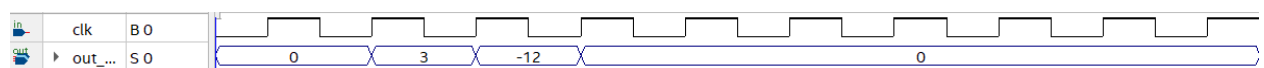
```
11111111000100001000000010010011
000000000001100000000000100010011
000000000000100010000000010110011
000000000000000001000000010010011
```

Что будет на выходе RA1 (out_proc) по event (clk из 0 в 1)

- 0) На выходе 0 (-15 еще не запасалось)
- 1) На выходе 0 (3 еще не записалось)
- 2) На выходе 3 (Выводит значение RA1 при операции на АЛУ)
- 3) На выходе -12 (результат операции на АЛУ, записанный в регистр 1).

Далее же на выходах будут 0

Результат симуляции RISC-V с такой прошивкой.



Как читатель видит, инструкции типа R работают как надо.

Пример 3. Инструкции типа Branch (умножение -15 на 3 путем сложения).

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type

Опкоды, которые нас понадобятся

Opcode_B = 7'd99, Opcode_I_1 = 7'd19, Opcode_R = 7'd51

Строчка 1. Загрузка 3 в регистр 2 (прибавление 3 к 0, что лежит в регистре 0)

000000000011 00000 000 00010 0010011

0010011₂ = 19 – opcode данного типа команды

00010₂ = 2 – регистр, в который мы записываем константу

000₂ – funct3, показывает как бы операцию на АЛУ (сложение)

00000₂ – регистр, с которым складываем константу

000000000011₂ = 3 – с чем складываем значение в регистре (константа)

Строчка 2. Переход на 4 строчки вниз (если счетчик равен 0)

0000000 00000 00010 000 10000 1100011

1100011₂ = 99 – opcode данного типа команды (B)

00000 = номер регистра 2 (ноль)

000₂ – funct3, показывает как бы операцию на АЛУ (регистр 1 = регистр 2)

00010₂ – номер RA1 (счетчик)

0000000010000₂ = {Imm_I, 1'd0} = 16 = 4 * (4)

Строчка 3. (прибавление (-15) к значению, что лежит в регистре 1)

111111110001 00001 000 00001 0010011

0010011₂ = 19 – opcode данного типа команды

00001₂ = 1 – регистр, в который мы записываем константу

000₂ – funct3, показывает как бы операцию на АЛУ (сложение)

00001₂ – регистр, с которым складываем константу

111111110001₂ = -15 – с чем складываем значение в регистре (константа)

Строчка 4. (прибавление (-1) к значению, что лежит в регистре 2)

111111111111 00010 000 00010 0010011

$0010011_2 = 19$ – *opcode* данного типа команды

$00010 = 2$ – регистр, в который мы записываем операцию с константой

$000_2 = func3$, показывает как бы операцию на АЛУ (сложение)

00010 – регистр, с которым складываем константу

$111111111111_2 = -1$ – с чем складываем значение в регистре (константа)

Строчка 5. Вывод значения счетчика в RA1 (выводим RA1+0)

000000000000 00010 000 00010 0010011

$0010011_2 = 19$ – *opcode* данного типа команды

$00010 = 2$ – регистр, в который мы записываем результат сложения

$000_2 = func3$, показывает как бы операцию на АЛУ (сложение)

00010_2 – регистр, с которым складываем константу

$000000000000_2 = 0$ – с чем складываем значение в регистре (константа)

Строчка 6. Переход на 4 строчки вверх (если счетчик больше 0)

1111111 00010 00000 100 10001 1100011

$1100011_2 = 99$ – *opcode* данного типа команды (B)

$00010 =$ номер регистра 2 (счетчик)

$100_2 = func3$, показывает как бы операцию на АЛУ ($0 < \text{регистр } 2$)

00000_2 – номер регистра RA1

$11111111110000_2 = \{Imm_I, 1'd0\} = -16 = 4 * (-4)$

Строчка 7. Вывод значения (-15*3) в RA1 (выводим RA1+0)

000000000000 00001 000 00001 0010011

$0010011_2 = 19$ – *opcode* данного типа команды

$00001 = 1$ – регистр, в который мы записываем результат сложения

$000_2 = func3$, показывает как бы операцию на АЛУ (сложение)

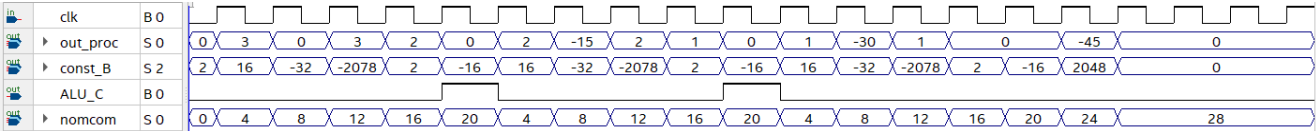
00001_2 – регистр, с которым складываем константу

$000000000000_2 = 0$ – с чем складываем значение в регистре (константа)

Итоговая прошивка.

```
0000000000011000000000000100010011
0000000000000000010000100001100011
111111110001000010000000010010011
111111111111000100000000100010011
0000000000000000010000000100010011
11111110001000000100100011100011
000000000000000001000000010010011
```

Результат симуляции RISC-V с такой прошивкой



Как читатель может увидеть, на симуляции у меня большее число выходных сигналов, это связано с тем, что я допустил ошибку в РС и искал ее, симулируя процессор с разными прошивками (думал, в декодере ошибся, но там все, как оказалось, было верно).

Полезные ссылки (+источники).

- 1) Гитхаб методички <https://github.com/YuraThomas/Methoda/tree/main>
- 2) АПС МИЭТ (хотя много формализма и воды, но вот для тех, кто частично шарит в теме данный ресурс будет полезен, основывался я большей частью на нем, но сделал в более последовательном и логичном по моей точке зрения стиле).
 - a. YouTube канал АПС https://www.youtube.com/@digital_machines
 - b. Гитхаб АПС <https://github.com/MPSU/APS>
- 3) Телеграм канал авторов, есть разные статьи (t.me/botdrakosha)
- 4) Харис-Харис, книжка по RISC-V (как бы я их не ругал за слишком формальный и одновременно полный воды рассказ, но основывался я в том числе и на них)